L'editor di rsedit

Architettura della libreria `rsedit_core` e del front-end TUI — manuale per lo sviluppatore

Indice

1 Introduzione 4
1.1 Come leggere questo manuale 4
1.2 Comportamento e difetti 4
1.3 Convenzioni 4
2 Panoramica 5
2.1 Gli strati 5
2.2 Dove sta cosa 5
2.3 La superficie pubblica 6
2.4 I thread 7
3 Il ciclo di vita del programma 8
3.1 Dal `main` al ciclo degli eventi 8
3.2 Il ciclo degli eventi della TUI 10
4 Lo stato: `EditorState` e i compartimenti 13
4.1 La facciata 13
4.2 Le regole dei compartimenti 14
4.3 I compartimenti 14
4.4 Le coppie con `editor/` 15
4.5 L'ordine dei lock 16
4.6 Accesso ai buffer 16
4.7 L'editor come contesto dell'interprete 16
4.8 Le decisioni 17
5 Dal tasto al comando 21
5.1 Il vocabolario dei tasti 21
5.2 Le keymap 21
5.3 `handle_key_event` 22
5.4 L'argomento prefisso 23
5.5 La risoluzione: `resolve_key_sequence` 24
5.6 `run_command_form`: l'esecuzione di un comando 25
5.7 La cattura di una sequenza 26
5.8 Le decisioni 27
6 Comandi e argomenti 28
6.1 Che cos'è un comando 28
6.2 Le specifiche degli argomenti 28
6.3 `call-interactively` e la catena degli argomenti 28
6.4 Le decisioni 29
7 Minibuffer e ricerca incrementale 31
7.1 Il minibuffer 31
7.2 La ricerca incrementale 31
8 I buffer 32

8.1 La struttura `Buffer` ... 32
8.2 `BufferTrait` e `GapBuffer` ... 32
8.3 Le due porte ... 33
8.4 Undo ... 34
8.5 Mark, regione, overlay e testo virtuale ... 36
8.6 Le cache del testo ... 37
8.7 File su disco ... 37
8.8 Le decisioni ... 37
9 Il testo: ricerca, sostituzione, kill ring, rettangoli e liste di risultati ... 40
9.1 Posizioni: sempre caratteri ... 40
9.2 `Pattern`: che cosa cercare ... 41
9.3 La sessione di ricerca incrementale: `Isearch` ... 42
9.4 La sessione di sostituzione: `Replace` ... 43
9.5 Il kill ring ... 45
9.6 I rettangoli ... 46
9.7 Le liste di risultati ... 47
9.8 Le decisioni ... 48
10 Modi ... 50
10.1 Il modo maggiore ... 50
10.2 Due lessici sullo stesso testo ... 50
10.3 Lo scanner delle espressioni ... 51
10.4 Facce e temi ... 53
11 Il lavoro in background ... 54
11.1 Il WorkManager ... 54
11.2 I lavori ... 56
11.3 Il nucleo ... 57
11.4 Gli argomenti ... 58
11.5 L'esecutore ... 59
11.6 Il registro ... 60
11.7 I thread dedicati ... 60
11.8 Il watchdog ... 61
11.9 Misure, segnalazioni e opzioni ... 61
11.10 Gli utenti del WorkManager ... 63
11.11 Come si prova ... 65
11.12 Le decisioni ... 65
12 Lo schermo ... 67
12.1 Finestre ... 67
12.2 `snapshot`: dalla memoria alla fotografia ... 67
12.3 Il renderer della TUI ... 68
12.4 Il mouse ... 68
12.5 Le decisioni ... 68
13 Macro di tastiera ... 70
13.1 Le decisioni ... 70
14 Le primitive dell'editor ... 71
14.1 Il registro ... 71
15 I moduli Lisp ... 73
16 I test ... 75
17 Ricette ... 76
17.1 Una primitiva o un comando in Rust ... 76
17.2 Un comando in Lisp ... 76

17.3 Un modo maggiore 76
17.4 Un lavoro di sfondo 76
17.5 Un modulo Lisp nuovo 77
17.6 Un nuovo front-end 77
18 Appendice 78
18.1 Glossario 78
18.2 Mantenere aggiornato questo manuale 78
18.3 Elenco delle primitive dell'editor 78

1 Introduzione

Questo manuale descrive la libreria `rsedit_core` (`core/src/`) e il front-end a terminale che la usa (`src/`). È il compagno di *L'interprete Lisp di rsedit* (`architettura-interprete.typ`), che descrive il linguaggio: qui l'interprete è dato per noto, e si spiega come l'editor lo ospita, come trasforma un tasto in un comando, come conserva il testo, come lavora in background e come descrive lo schermo a chi lo disegna.

La libreria è un editor *senza testa*. Non apre un terminale, non legge la tastiera e non disegna: riceve eventi già tradotti nel proprio vocabolario (`KeyEvent`, `MouseEvent`, testo incollato, dimensioni) e restituisce, quando le viene chiesto, una fotografia immutabile di ciò che andrebbe mostrato (`FrameSnapshot`). Tutto ciò che sta in mezzo — buffer, finestre, modi, comandi, lavori di sfondo — è in `rsedit_core`, e quasi tutto il comportamento visibile all'utente è scritto in Lisp nei moduli di `core/lisp/`.

> Due confini reggono l'architettura. L'interprete è un crate a sé, `risp`, e non può nominare alcun tipo dell'editor: questo permette di provarlo e riusarlo da solo. E `ui/` non nomina nulla di `editor/`, come verificano i test (`tests::layering_tests`): questo permette di scrivere un renderer contro una fotografia invece che contro gli interni della libreria.

1.1 Come leggere questo manuale

Il Sezione 2 e il Sezione 3 danno il quadro: gli strati, i thread, che cosa succede dall'avvio al ciclo degli eventi. Poi il manuale segue il percorso di un tasto: lo stato che attraversa (Sezione 4), la risoluzione e l'esecuzione del comando (Sezione 5, Sezione 6, Sezione 7), il testo che il comando modifica (Sezione 8, Sezione 9), i modi che lo interpretano (Sezione 10), il lavoro che continua in background (Sezione 11) e la fotografia che il front-end disegna (Sezione 12). Gli ultimi capitoli sono di riferimento.

Gli estratti di codice sono presi dal sorgente e semplificati: commenti tolti, parametri generici abbreviati, rami ripetitivi riassunti. Ogni riga di un estratto corrisponde a una riga del file indicato sopra di esso; il percorso è relativo a `core/src/`.

Ogni affermazione è stata letta nel codice; i comportamenti che dipendono dall'esecuzione (avvio, undo, comandi di shell, colorazione) sono stati osservati con piccoli programmi che usano la libreria come farebbe un front-end.

1.2 Comportamento e difetti

Questo manuale descrive il comportamento *attuale* del codice, anche dove quel comportamento è un difetto. I difetti sono descritti, con la riproduzione e una correzione proposta, nel *Rapporto sui problemi del codice* (`rapporto-problemi.typ`). Dove il manuale descrive un punto che il rapporto considera un difetto, un rimando come (→ BUF-1) indica la scheda corrispondente; quando il difetto viene corretto si aggiorna la descrizione qui e si toglie la scheda là.

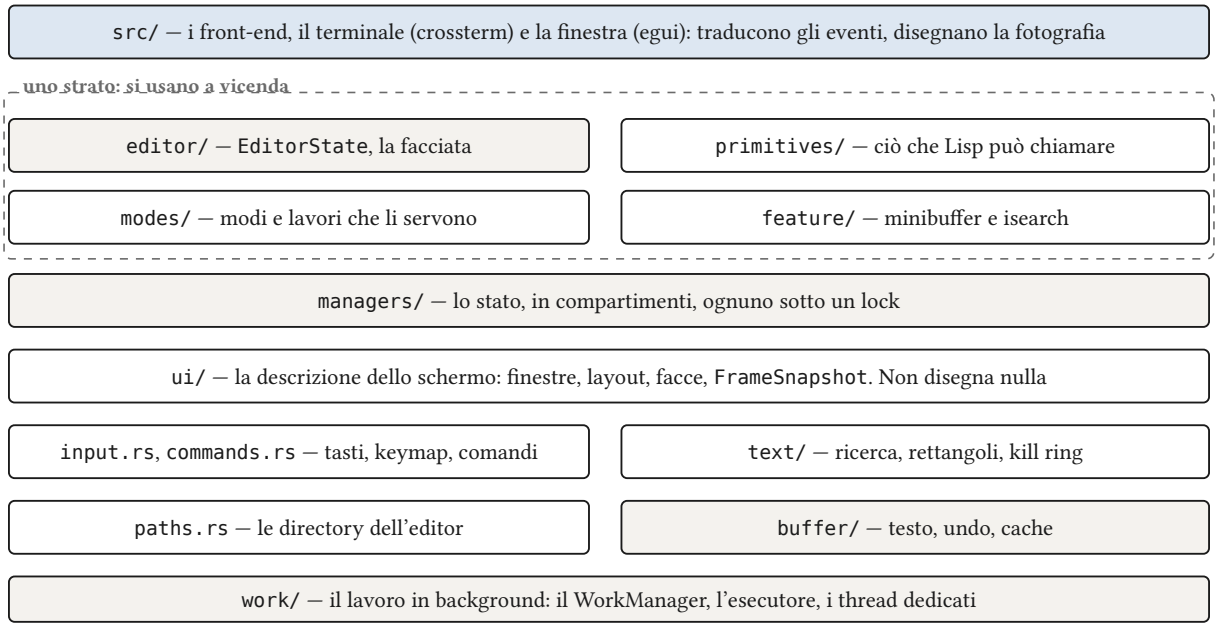
1.3 Convenzioni

- I nomi in `monospazio` sono identificatori: tipi e funzioni Rust (`run_command_form`), primitive e variabili Lisp (`call-interactively`, `mouse-mode`), file (`editor/keys.rs`, relativo a `core/src/` salvo indicazione diversa).
- Nei diagrammi le caselle grigie sono dati o stato, quelle bianche funzioni o passi, quelle azzurre appartengono a un altro thread o al front-end, quelle rosate sono errori. Nei diagrammi di sequenza le frecce continue sono chiamate e quelle tratteggiate risposte.
- «Primitiva» è una funzione Rust registrata nell'ambiente Lisp; «comando» è una funzione (Rust o Lisp) registrata anche nel registro dei comandi, e quindi raggiungibile con `M-x` e capace di farsi chiedere gli argomenti.

2 Panoramica

2.1 Gli strati

`lib.rs` elenca i moduli in ordine di dipendenza: ognuno usa quelli sopra di sé e, di quelli sotto, solo ciò che la sua voce dichiara. La figura li mostra dal basso verso l'alto, cioè nella direzione in cui si costruisce.



Fino a `managers/` le frecce sono omesse perché la regola è una sola: ogni livello può usare tutti quelli disegnati sotto di lui. `work/` non conosce l'editor: un buffer emette un suo argomento senza sapere chi lo ascolta (Sezione 11.4). I quattro moduli in alto sono uno strato solo, non un ordine: si usano a vicenda — `editor/` installa le primitive e avvia i lavori dei modi, che agiscono sull'editor — e la mappa li elenca per ciò che sono. L'interprete non è nella figura: è il crate `risp`, che non conosce l'editor; le sue espressioni compaiono da `input.rs` in su.

La regola ha quattro eccezioni, dichiarate. `buffer/` conserva alcuni tipi che vengono da più in alto, le facce di `ui/` (negli overlay e nel testo virtuale) e gli stati della colorazione e della scansione di `modes/` (nelle due cache del testo, Sezione 8.6). `input.rs` lega i tasti a espressioni Lisp, `ELispExp`, il cui tipo nomina `EditorState` di `editor/` (Sezione 5.2). `ui/` compone le finestre sulla tabella dei buffer, `Buffers` di `managers/`, e usa `ELispExp` per dare a Lisp l'id di una finestra e per leggere lo stato che un modo mette nella riga di stato. E `managers/` conserva i modi maggiori e le tabelle sintattiche di `modes/`, e le espressioni Lisp che i comandi, i legami e gli hook registrano (Sezione 4.3); le finestre, le facce e il tema che conserva vengono da `ui/`, che gli sta sotto. Per ogni modulo da `work/` a `managers/`, `layering_tests` tiene l'elenco di ciò che usa da più in alto: vuoto, tranne per questi quattro. Un uso nuovo fa fallire il test finché non lo si aggiunge lì e nella mappa di `lib.rs`.

2.2 Dove sta cosa

Percorso	Contenuto
`lib.rs`	Ordine dei moduli e superficie pubblica (`pub use`). Tutto il resto è `pub(crate)`.
`paths.rs`	Le directory dell'editor, configurazione e cache, secondo le convenzioni di ogni sistema: `app_dir` (Sezione 3.1.2).
`buffer/`	`Buffer`, `BufferTrait`, `GapBuffer`, undo, mark, overlay, testo virtuale, cache di colorazione (`syntax.rs`) e di scansione (`scan.rs`), timbro del file su disco (`disk.rs`).

Percorso	Contenuto
`text/`	Ciò che si può calcolare sul testo conoscendo solo `BufferTrait`: la ricerca (`search.rs`), le sessioni di ricerca incrementale e di sostituzione (`isearch.rs`, `replace.rs`), il kill ring, i rettangoli, le liste di posizioni (`results.rs`).
`input.rs`	`KeyEvent`, `KeyCode`, `MouseEvent`, `Keymap`, `TransientKeymap`, la lettura e la scrittura dei nomi dei tasti (`parse_key_sequence`, `describe_keys`), i legami scritti in Rust (`fill_default_keymaps`).
`commands.rs`	`ArgSpec`, `PrefixArg`, `Invocation`, `PendingCommand`, `CommandRegistry`: il vocabolario dei comandi.
`managers/`	I compartimenti dello stato: `Buffers`, `Windows`, `Modes`, `Commands`, `KillYank`, `Macros`, `History`, `Log`, `Runtime`.
`editor/`	`EditorState` e le porte dei compartimenti (`mod.rs`); la metà della facciata di ogni compartimento, nel file con il suo nome (`buffers.rs`, `windows.rs`, `modes.rs`, `commands.rs`, `kill_yank.rs`, `macros.rs`, `log.rs`, `runtime.rs`); ciò che coordina più cose, nei file che portano il nome di ciò che coordinano (`keys.rs`, `mouse.rs`, `frame.rs`, `boot.rs`, `settings.rs`, `replace.rs`, `results.rs`, `background.rs`, `work.rs`). Vedi Sezione 4.4.
`primitives/`	Le 304 primitive dell'editor, in 28 moduli, e `install_primitives`.
`modes/`	`MajorMode`, grammatica di colorazione (`syntax.rs`), tabella sintattica e scanner di espressioni (`sexp.rs`), i lavori di sfondo `Highlighter`, `Prescanner`, `FileWatcher`, `AutoSaver`.
`feature/`	Le due funzionalità che non possono dipendere da un file Lisp: il minibuffer e la ricerca incrementale.
`work/`	Il WorkManager: il nucleo dello scheduler, l'esecutore, il registro, i thread dedicati, il watchdog, gli argomenti e le opzioni (Sezione 11).
`ui/`	Albero delle finestre, finestre flottanti, layout delle righe, facce e temi, `FrameSnapshot`.
`tests/`	Circa ottanta file di test dell'editor, più `perf/`.
`../lisp/`	I moduli Lisp distribuiti con l'editor (venti file).
`../build.rs`	Copia i moduli Lisp in `target/<profilo>/data/lisp`, accanto all'eseguibile.
`../../src/`	I front-end: `main.rs` (avvio), `tui.rs` (il terminale: ciclo degli eventi e disegno) e `gui.rs` (una finestra grafica, con egui).

2.3 La superficie pubblica

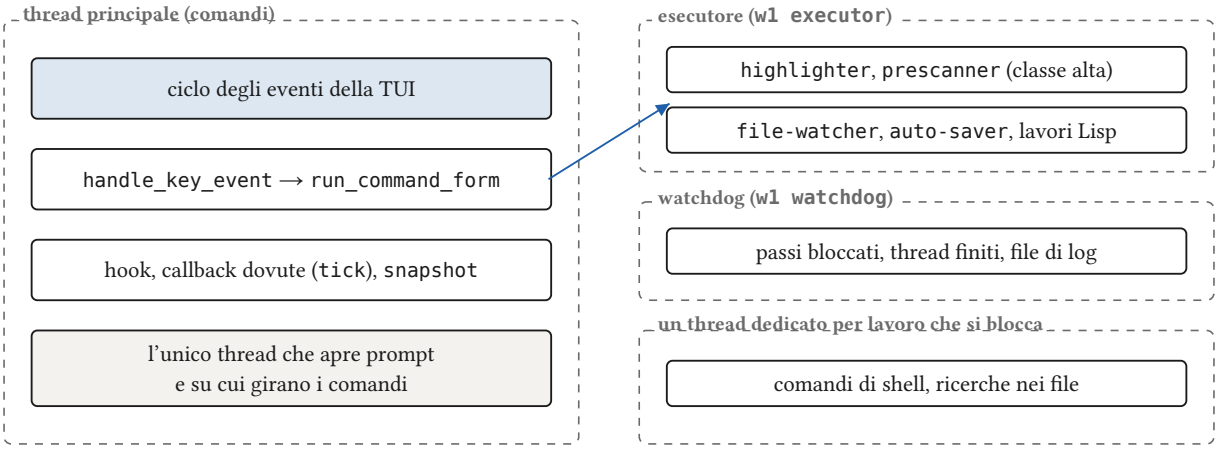
Un front-end vede soltanto ciò che `lib.rs` riesporta: `EditorState`, `create_global_env`, `BufferTrait`, `GapBuffer`, i tipi di input (`KeyEvent`, `KeyCode`, `KeyModifiers`, `MouseEvent`, `MouseKind`, `MouseButton`), i tipi della fotografia (`FrameSnapshot`, `RenderableWindowView`, `Highlight`, `GutterCell`, `Separator`, `Rect`, `Face`, `Style`, `Color`, `Theme`, `NAMED_COLORS`), dall'interprete `Env`, `LispContext`, `Parser` ed `eval`, i compartimenti `Buffers` e `Windows`, le directory dell'editor (`app_dir`, `AppDir`, `DirOptions`, `DirError` e i nomi delle variabili `CONFIG_DIR`, `CACHE_DIR`, `XDG_CONFIG_HOME`, `XDG_CACHE_HOME`, Sezione 3.1.2), più `mouse_mode`, `MOUSE_MODE`, `isolate_config_for_tests` e l'alias `ELispExp<B>` (`LispExp<EditorState<B>>`). I metodi di `EditorState` che un front-end usa sono pochi:

Metodo	Uso
`create_global_env::<B>()`	Costruisce editor e ambiente Lisp, carica la configurazione.

Metodo	Uso
`handle_key_event(ev, &env)`	Un tasto premuto.
`handle_paste(text, &env)`	Un incolla «bracketed», come un solo comando.
`handle_mouse_event(ev, &env)`	Un evento del mouse; risponde se ha cambiato qualcosa.
`resize(env, w, h)`	Nuove dimensioni del terminale.
`snapshot(&env, w, h)`	La fotografia da disegnare.
`next_redraw_in(&env, &frame)` `drag_scroll_in()`	Quanto si può aspettare prima di ridisegnare anche senza eventi.
`tick(&env)`	Il tempo è scaduto: esegue ciò che era in attesa (callback di sfondo, scorrimento durante il trascinamento).
`on_background_change(sveglia)`	La funzione che il lavoro di sfondo chiama quando ha cambiato lo schermo: sveglia il ciclo del front-end (Sezione 11.10).
`is_running()`	Falso dopo `quit`.
`shutdown(entro)`	All'uscita, anche per un panic: ferma il lavoro di sfondo e scrive nel file le ultime righe del log.
`enable_log_file`, `log_diagnostic`, `set_echo_message`, `take_backtrace_suffix`	Diagnostica.

2.4 I thread

L'editor usa più thread, ma uno solo esegue i comandi. La figura mostra chi esiste; tutti tranne il principale appartengono al WorkManager (Sezione 11), e portano il suo numero nel nome.

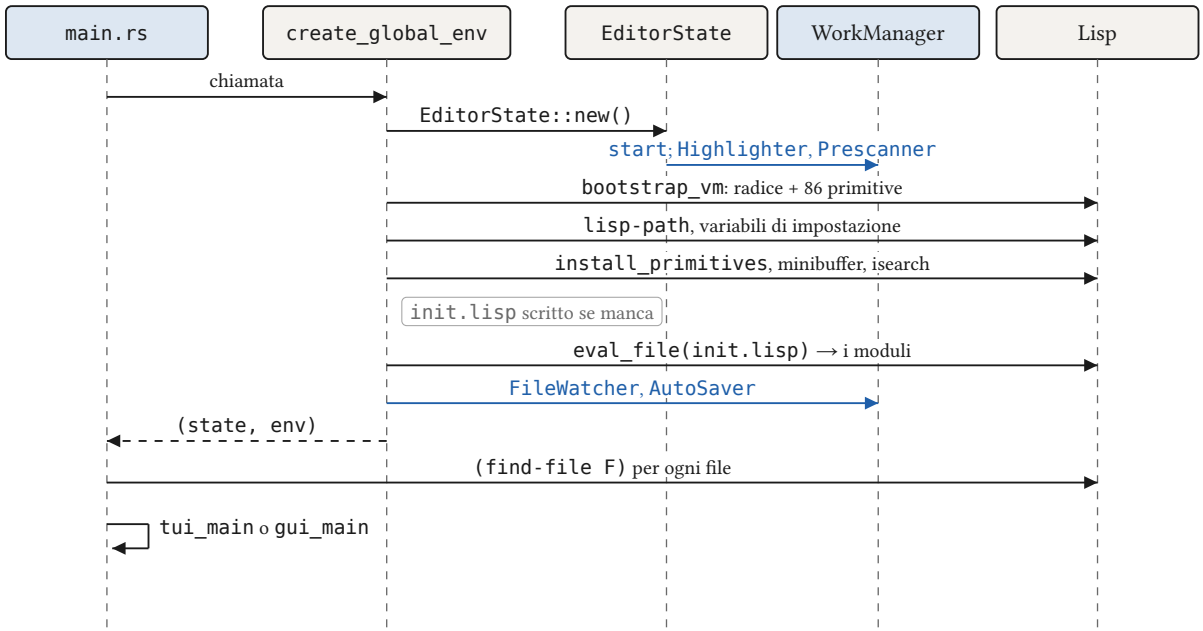


La freccia è la casella con cui il thread dei comandi consegna il lavoro all'esecutore. Tutti condividono lo stesso `EditorState` (clonato: ogni campo è un `Arc`) e, i lavori che lo usano, lo stesso ambiente Lisp. Il carburante, il permesso di `yield`, il contrassegno del comando per l'undo, i dati di `string-match` e il segno di `in_worker` sono invece per thread (variabili `thread_local!`). I lavori di sfondo non chiamano mai Lisp sul proprio thread quando l'effetto deve toccare l'interfaccia: lasciano una *callback dovuta*, che il thread principale esegue tra un comando e l'altro (Sezione 11.10). Fuori dal WorkManager restano i thread di `(spawn ...)`, che l'interprete avvia da sé, e il breve `stop shell` di un comando di shell fermato.

3 Il ciclo di vita del programma

3.1 Dal `main` al ciclo degli eventi

`src/main.rs` legge la riga di comando — i file da aprire, `--gui`, `--font FILE` e `--help` — e chiama `create_global_env::<GapBuffer>()`; poi attiva il file di log `rsedit.log` nella directory della cache (Sezione 3.1.2), apre ogni file della riga di comando valutando `(find-file PERCORSO)` ed entra nel ciclo di un front-end: `tui_main`, il terminale, se lo standard input è un terminale e `--gui` non è chiesto, altrimenti `gui_main`, una finestra grafica. Un errore di avvio dell'interprete viene stampato e il programma esce; un errore di `find-file` finisce nell'area dei messaggi. All'uscita, anche per un panic, `ShutDownOnExit` chiama `shutdown`, che ferma il lavoro di sfondo e scrive nel file le ultime righe del log.



3.1.1 I passi di `create_global_env`

1. `EditorState::new()` riempie la keymap globale con le poche associazioni scritte in Rust (`fill_default_keymaps`), crea ogni compartimento con il suo `Arc<RwLock<…>>`, avvia il WorkManager (`start`) e gli consegna i due lavori che girano sempre, `Highlighter` e `Prescanner`, che si svegliano quando un buffer o una grammatica cambiano.
2. `bootstrap_vm` crea l'ambiente radice con le 86 primitive di base e verifica l'interprete.
3. `lisp-path` diventa `("<directory dell'eseguibile>/data/lisp")`.
4. Ogni impostazione che l'editor legge (`editor/settings.rs`) riceve il suo valore iniziale, così che `(setq ...)` in una configurazione modifichi un valore che esiste già: per esempio `mouse-mode` (`nil`), `echo-message-timeout` (5), `minibuffer-width` (60), `minibuffer-height` (3), `window-separator` (`|`), `auto-save` e `watch-files` (`t`). `document_settings` dà a ognuna la documentazione che `C-h v` mostra. Viene creato il modo `fundamental-mode`.
5. `install_primitives`, `install_minibuffer` e `install_isearch` registrano le primitive, i comandi, il modo `minibuffer-mode` e il modo `isearch-mode`.
6. `rsedit-path` riceve il percorso dell'eseguibile.
7. Se manca, `init.lisp` viene creato con `DEFAULT_INIT_LISP` (vedi sotto).
8. `eval_file` valuta `init.lisp`, che carica i moduli Lisp. Un errore di valutazione ferma l'avvio, e `create_global_env` lo restituisce.
9. Il WorkManager riceve `FileWatcher` e `AutoSaver`, che a ogni passo leggono le loro impostazioni (`watch-files`, `watch-file-interval`, `auto-save`, `auto-save-interval`).

3.1.2 Le directory dell'editor

L'editor tiene due directory sue: quella della configurazione, con `init.lisp`, e quella della cache, con ciò che può rifare, come il file di log: cancellarla non perde nulla che conti. Le trova `app_dir`, in `paths.rs`: le si chiede una directory per ciò a cui serve (`AppDir::Config`, `AppDir::Cache`), e risponde con il suo posto su questo sistema, creandola se manca e se `DirOptions::create` lo chiede. La usano `create_global_env`, per `init.lisp`, e `main.rs`, per il log. Le due directory si trovano con la stessa regola:

Fonte, in ordine di precedenza	Directory
`RSEDIT_CONFIG_DIR`, `RSEDIT_CACHE_DIR`	usata così com'è, su ogni sistema
`XDG_CONFIG_HOME`, `XDG_CACHE_HOME` (solo Unix, e solo se assoluta)	la variabile, con `rsedit` aggiunto in fondo
il sistema	su Windows `%APPDATA%\rsedit` e `%LOCALAPPDATA%\rsedit\cache`; altrove, macOS compreso, `$HOME/.config/rsedit` e `$HOME/.cache/rsedit`

Una variabile impostata ma vuota conta come non impostata. Quando nessuna fonte risponde, `app_dir` dà `DirError::Undetermined`; gli altri errori sono una directory che manca senza che si chieda di crearla (`NotFound`), un file al suo posto (`NotADirectory`) e una creazione fallita (`CannotCreate`). Che la directory si possa scrivere non viene controllato. La scelta tra le fonti, `choose_dir`, riceve i valori invece di leggerli dall'ambiente: così i test ne provano la precedenza senza scrivere nell'ambiente del processo, che tutti i test vedono. `isolate_config_for_tests` dà invece a ogni processo di test due directory sue, attraverso le due variabili `RSEDIT_*`.

Quattro decisioni:

1. **Su macOS, `~/.config` e `~/.cache`.** È lì che guarda chi usa un editor da terminale su un Mac: gli editor da cui arriva, Emacs con XDG, Neovim e Helix, tengono lì configurazione e cache. Scartato: `~/Library`, la convenzione delle applicazioni impacchettate.
2. **Su Windows la cache è locale.** `%APPDATA%` segue la persona da una macchina all'altra e viene copiato a ogni accesso. Una cache è proprio ciò che si può rifare, e quindi ciò che non va copiato: sta sotto `%LOCALAPPDATA%`.
3. **Su Windows le variabili `XDG_*` non contano.** Le imposta solo chi si porta dietro un'abitudine di Unix; preferirle alla risposta del sistema metterebbe la configurazione dove nessun programma Windows la cerca.
4. **Ogni processo di test ha le sue directory.** Costruire un editor crea la directory della configurazione e valuta ciò che contiene: sotto la `HOME` di chi lancia i test scriverebbe lì la sua configurazione, e alcuni test spostano `HOME`, che vale per tutto il processo mentre i test girano in parallelo. `isolate_config_for_tests` punta configurazione e cache a due directory temporanee del processo, una volta sola e prima del primo editor, attraverso le due variabili `RSEDIT_*`: la porta di una persona, così i test passano da `app_dir` invece di aggirarla. È pubblica per i test del crate binario, che il `cfg(test)` di una libreria non raggiunge.

3.1.3 Il file `init.lisp` predefinito

Il file `init.lisp` predefinito carica, in quest'ordine: `commands`, `debug`, `common-keymaps`, `indent`, `minibuffer`, poi i moduli facoltativi `rust-mode`, `risp-mode`, `cc-mode`, `python-mode`, `toml-mode`, `bash-mode`, `dired`, `completion`, `preview`, `clipboard`, `electric-pair`, `buffer-list`, `shell`, `manpage`, `help`, `occur`, `compile`, `theme`, `vim`, `completion-at-point`, e imposta `dired-sort` a `'type` e `mouse-mode` a `t`. L'ordine conta: `commands` definisce `defcommand`, con cui sono scritti gli altri, e `debug` definisce `message` e `report-error`.

Il file viene scritto solo se manca: chi ha già un `init.lisp` deve aggiungere a mano le righe `(eval-file ...)` dei moduli nuovi (→ AVV-5).

3.1.4 `eval_file`

`eval_file(nome, env)` cerca prima `nome` come percorso. Se non esiste e il nome non contiene `/`, `\` o `.`, cerca `nome.lisp` in ogni directory di `lisp-path`, in ordine. Il contenuto viene avvolto in `(progn …)`, letto con *una sola* chiamata a `Parser::next` e valutato dentro uno scope di carburante (`begin_command`), alla radice della catena di ambienti (`top_level`). Un file mancante produce una riga di log e il valore `nil`.

da `editor/boot.rs` — estratto semplificato

```
let content = format!("(progn {})", /* il testo del file, o di nome.lisp in lisp-path */);
let ast = if let Ok(ast) = Parser::new(&content).next() { ast } else {
    return Ok(ELispExp::nil());           // errore di sintassi: nessun messaggio
};
let _command = self.begin_command();             // uno scope di carburante
let value = eval(&ast, top_level(&env), self)?;  // alla radice dell'ambiente
Ok(ELispExp::proper_list(vec![value]))
```

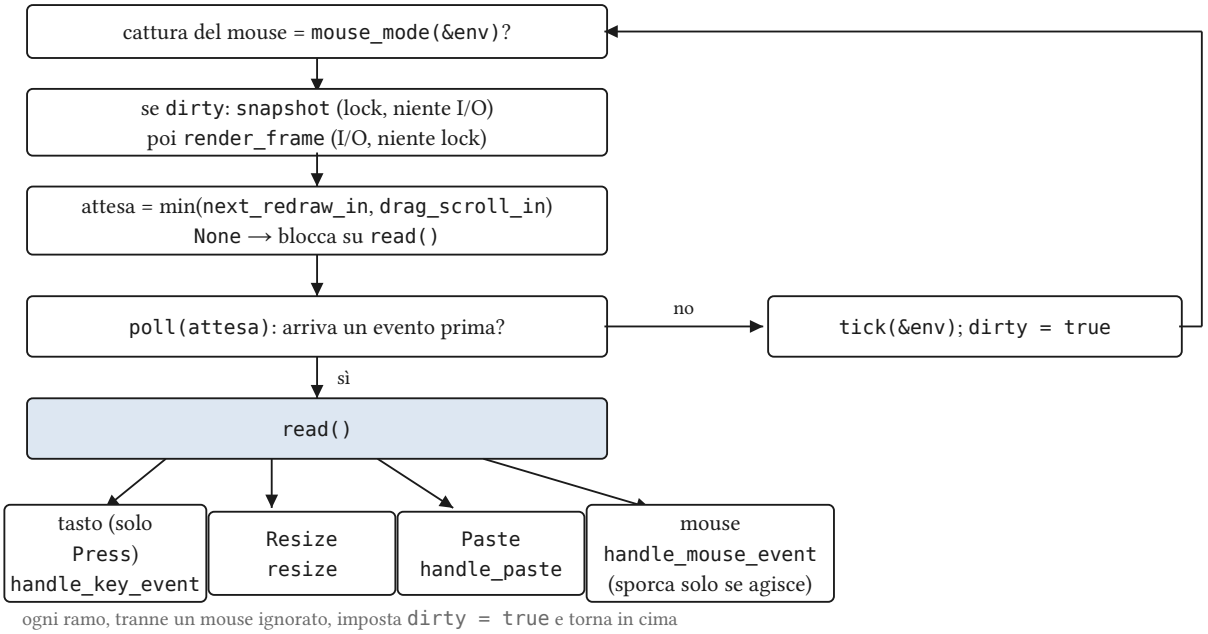
L'involucro `(progn …)` fa del file un'unica espressione, e questo ha tre conseguenze. Una parentesi chiusa di troppo chiude la `progn` in anticipo e il resto del file non viene letto; una parentesi mancante, o un commento sull'ultima riga senza a capo finale (la parentesi aggiunta finisce dentro il commento), lascia la `progn` aperta e il lettore fallisce. In entrambi i casi `eval_file` restituisce `nil` senza messaggi (→ AVV-2). Il primo errore di *valutazione*, invece, interrompe il file da quel punto e risale come errore.

3.1.5 Le decisioni

1. **I due lessici partono con l'editor.** L'evidenziatore e la prescansione sono sottomessi in `EditorState::new`, prima che esista un modo: un lavoro che aspetta un cambiamento non costa nulla, e uno mai avviato costerebbe un file senza colori e senza una ragione detta. Sono sottomessi una volta sola (→ BKG-2).
2. **I due osservatori partono dopo `init.lisp`.** Il controllo dei file e il salvataggio automatico leggono le loro impostazioni a ogni passo, e sono sottomessi dopo la configurazione: ciò che dice conta dalla prima occhiata.
3. **Il file predefinito carica moduli e imposta variabili, niente di più.** È valutato con `?`, e i moduli che carica non sono sempre presenti — un binario di test non ha `data/lisp` accanto —: senza di loro `defcommand` non è una macro, una forma che la usa diventa una chiamata qualunque, e l'avvio si ferma. Un comando sta in un modulo o in Rust. È una stringa grezza, dove una barra rovesciata è una barra rovesciata; ed è una costante con un nome, perché un test la valuti in un ambiente nudo.
4. **Un file si valuta alla radice dell'ambiente.** Un `defun`, un `defvar` o il `setq` di un nome nuovo legano nell'ambiente in cui sono valutati, e dentro una funzione se ne vanno con lei. `eval_file` valuta alla radice (`top_level`), da dovunque venga la chiamata: un modulo caricato da una funzione, o con `M-:`, definisce ciò che definirebbe `init.lisp`.

3.2 Il ciclo degli eventi della TUI

`tui_main` (`src/tui.rs`) attiva la modalità raw e l'incolla «bracketed», rileva la profondità di colore (`COLORTERM`, `TERM`), comunica le dimensioni iniziali in `frame-width` e `frame-height`, e poi gira finché `is_running()` è vero.



ogni ramo, tranne un mouse ignorato, imposta `dirty = true` e torna in cima

Il cuore del ciclo, senza la gestione della cattura del mouse:

da `../../src/tui.rs` — estratto semplificato

```
while state.is_running() {
    if dirty {
        let (cols, rows) = terminal::size()?;
        frame = state.snapshot(&env, cols as usize, rows as usize);  // lock, niente I/O
        render_frame(&frame, depth)?;                                // I/O, niente lock
        dirty = false;
    }
    let wake = [state.next_redraw_in(&env, &frame), state.drag_scroll_in()]
        .into_iter().flatten().min();
    if let Some(remaining) = wake && !poll(remaining)? {
        state.tick(&env);         // l'attesa è scaduta: callback dovute, scorrimento
        dirty = true;
        continue;
    }
    match read()? {
        Event::Key(k) => if let Some(ev) = translate_key(k)
                         && k.kind == KeyEventKind::Press {
            state.handle_key_event(ev, &env); dirty = true;
        },
        Event::Resize(w, h) => {
            state.resize(env.clone(), w as usize, h as usize); dirty = true;
        }
        Event::Paste(text)  => { state.handle_paste(text, &env); dirty = true; }
        Event::Mouse(m) => if let Some(ev) = translate_mouse(m)
                           && state.handle_mouse_event(ev, &env) { dirty = true; },
        _ => (),
    }
}
```

Tre dettagli contano. Il ridisegno avviene solo se qualcosa è cambiato (`dirty`): un terminale che riporta il mouse manda decine di movimenti al secondo, e ridisegnare per ognuno farebbe sfarfallare lo schermo. L'attesa è guidata dall'editor: se nulla è in sospeso `next_redraw_in` risponde `None` e il ciclo si blocca su `read()` senza consumare CPU. E `tick` viene chiamato solo quando l'attesa scade, non a ogni giro.

Un tasto che `translate_key` non sa tradurre — `Delete`, `Home`, `End`, `PageUp`, `PageDown`, `Insert`, i tasti funzione — viene soltanto registrato nel log: `KeyCode` non ha varianti per loro (→ CMD-2). Shift con un carattere diventa la maiuscola; `BackTab` diventa `Tab` con Shift.

All'uscita normale la TUI disattiva la cattura del mouse e l'incolla, pulisce lo schermo e lascia la modalità raw. Lo stesso ripristino non avviene se il ciclo esce per un errore di I/O o per un panic (→ TUI-2). La TUI disegna sullo schermo principale del terminale, non su quello alternativo (→ TUI-8).

4 Lo stato: `EditorState` e i compartimenti

4.1 La facciata

`EditorState<B: BufferTrait>` è una struttura di `Arc`: clonarla costa poco e ogni clone vede lo stesso stato. I suoi campi sono privati, e ognuno è un *compartimento* di `managers/` sotto un `RwLock`, un dato piccolo con un lock o un atomico proprio, o il WorkManager. La tabella dà i tipi senza l'`Arc` che li condivide.

Campo	Tipo	Contenuto
`buffers`	`RwLock<Buffers<B>>`	Tabella nome → `Arc<RwLock<Buffer>>`, buffer corrente, ordine di recente uso.
`windows`	`RwLock<Windows>`	Albero delle finestre affiancate, finestre flottanti, focus, prossimo id, trascinamento, ultimo clic.
`modes`	`RwLock<Modes<B>>`	Registro dei modi maggiori, keymap globale, completamenti globali, modi per estensione, keymap transitoria, tasti di ripetizione, hook globali.
`commands`	`RwLock<Commands<B>>`	Registro dei comandi, pila dei comandi in attesa di argomenti, argomento prefisso, ultimo comando e sua forma.
`kill_yank`	`RwLock<KillYank>`	Kill ring, ultimo yank, flag «il comando precedente ha ucciso/incollato», ultimo rettangolo ucciso, testo dovuto agli appunti di sistema.
`macros`	`RwLock<Macros>`	Macro in registrazione, ultima macro, profondità delle riproduzioni, contatore.
`history`	`RwLock<History>`	Storia dei prompt per chiave (100 voci per chiave predefinite).
`log`	`RwLock<Log>`	Righe diagnostiche e file a cui sono copiate.
`runtime`	`RwLock<Runtime>`	Colonna obiettivo, isearch e sostituzione in corso, `FuelMeter`, pila delle chiamate per i backtrace, tema in uso e tema di fabbrica, i lavori avviati da Lisp per nome.
`echo_message`	`RwLock<EchoMessage>`	Testo dell'area messaggi con l'istante in cui è stato scritto.
`pending_keys`	`RwLock<Vec<KeyEvent>>`	Tasti di una sequenza non ancora completa.
`owed`	`RwLock<Vec<OwedCallback>>`	Callback chieste dai lavori di sfondo e non ancora eseguite (Sezione 11.10.4).
`current_results`	`RwLock<Option<String>>`	Il buffer di risultati che `next-error` percorre.
`grammar_epoch`	`AtomicU64`	Quante volte è cambiata una grammatica: il timbro di una colorazione.
`syntax_changed`	`Topic`	L'argomento `syntax-changed`, risolto una volta (Sezione 11.4).
`work`	`WorkManager<EditorState<B>>`	Il lavoro asincrono: i lavori dell'esecutore e i thread dedicati (Sezione 11).

Campo	Tipo	Contenuto
`background_change`	`BackgroundChange`	Se un lavoro ha cambiato lo schermo dall'ultima fotografia, e come dirlo al front-end (Sezione 11.10).
`running`	`AtomicBool`	Falso dopo `quit`.
`shell_commands`	`AtomicUsize`	Quanti comandi di shell stanno girando.
`command_errors`	`AtomicUsize`	Quanti comandi sono falliti: la riproduzione di una macro si ferma al primo.

`EditorState` implementa `Debug` e `PartialEq` solo perché `LispContext` li richiede; nessuno dei due è pensato per essere usato (→ DOC-2).

4.2 Le regole dei compartimenti

Ogni compartimento le segue; la documentazione di `managers/mod.rs` le riassume.

Regola	Perché
Un compartimento non raggiunge un altro compartimento.	Riceve i valori che gli servono e restituisce risposte (`Windows::scroll_focused` riceve un numero di righe, non un buffer). Così solo la facciata tiene due lock insieme.
Non consegna guardie.	Si accede allo stato solo con una closure (`windows(|w| …)`), e il lock non sopravvive alla domanda.
Non valuta mai.	Nessun compartimento riceve un `Env`. Può conservare valori Lisp (il corpo di un hook), mai eseguirli: Lisp può rientrare nell'editor da qualunque primitiva.
Risponde con un verdetto, non con un effetto.	Quando un'operazione ha conseguenze altrove (un buffer da ripuntare, una finestra da rifocalizzare) restituisce un enum come `BufferRemoved` o `WindowRemoved`, e la facciata agisce.
Non fa nulla di lento con il lock preso.	Niente disco, rete o attese. Registrare una diagnostica aggiunge una riga alla lista e basta: il file lo scrive il watchdog con `Log::flush`, che tiene il lock della lista solo per copiare le righe.

Le prime quattro regole servono alla correttezza: ognuna toglie un modo in cui due pezzi dello stato si possono vedere in disaccordo. L'ultima serve alla latenza, ed è quella che si rompe per sbaglio, perché romperla non costa nulla finché il disco è veloce e l'albero piccolo. Il segnale è un lock tenuto attraverso una chiamata di cui non si controlla la durata: una lettura o una scrittura, un'allocazione grande quanto un file, una callback. Se non si sa dire quanto dura il corpo, si copia fuori ciò che serve e si rilascia prima.

La regola che nasce da tutte queste, e che vale anche fuori dai compartimenti, è una sola: *mai tenere un lock attraverso una chiamata all'interprete.* `run_hook` copia la lista degli hook e rilascia `modes` prima di valutarli; `command_specs` restituisce dati posseduti perché `call-interactively` valuta Lisp subito dopo. Un `RwLock` non è rientrante: un hook che chiama `add-hook` mentre lo stesso thread tiene il lock di lettura del registro si blocca per sempre, ogni volta.

4.3 I compartimenti

Ogni compartimento è un tipo di `managers/`, sotto un `RwLock` della facciata, e tiene un'invariante che i suoi campi presi uno per uno non terrebbero.

Compartimento	Che cosa garantisce
`Buffers`	La tabella non è mai vuota, e `current` nomina sempre un buffer della tabella: `make_current` e `bind_current` rifiutano un nome assente, `rename` e `remove` spostano `current` nello stesso passo.
`Windows`	Il focus nomina sempre una finestra che esiste, e l'ultima finestra affiancata non si chiude.
`Modes`	Le metà globali e quelle dei modi — keymap, sorgenti di completamento, hook — si leggono sotto lo stesso lock, nell'ordine in cui la risoluzione le consulta.
`Commands`	L'argomento prefisso appartiene a un comando solo, e i comandi in attesa di argomenti si chiudono in ordine inverso a quello in cui sono cominciati.
`KillYank`	I flag «il comando precedente ha ucciso, ha incollato» valgono rispetto all'anello e si spostano insieme, una volta per comando.
`Macros`	I tasti di una registrazione diventano l'ultima macro in un passo solo, e nessun tasto riprodotto viene registrato, a qualunque profondità.
`History`	Un cammino nella storia vale solo nell'anello e nel prompt in cui è cominciato.
`Log`	Ogni riga arriva al file una volta e nell'ordine in cui è stata registrata, salvo le scritture che falliscono e le righe uscite dalla lista prima di essere scritte.
`Runtime`	Nessuna: raggruppa ciò che appartiene alla sessione e non a un documento (Sezione 4.8).

La facciata raggiunge un compartimento con una *porta* che esegue una closure: `windows(|w| …)` per leggere, `windows_mut(|w| …)` per cambiare. Il lock non sopravvive alla domanda, e dentro la closure non si richiama la facciata: un `RwLock` non promette che una seconda lettura sullo stesso thread riesca; si può chiamare ciò che non prende lock, come fa `set_mode`, che sposta il contatore delle grammatiche dentro la porta di `modes`. `Log` è il solo che prende il lock da sé: `Log::append` e `Log::flush` ricevono il `RwLock<Log>`, perché il WorkManager registra senza tenere un editor, che lo contiene.

4.4 Le coppie con `editor/`

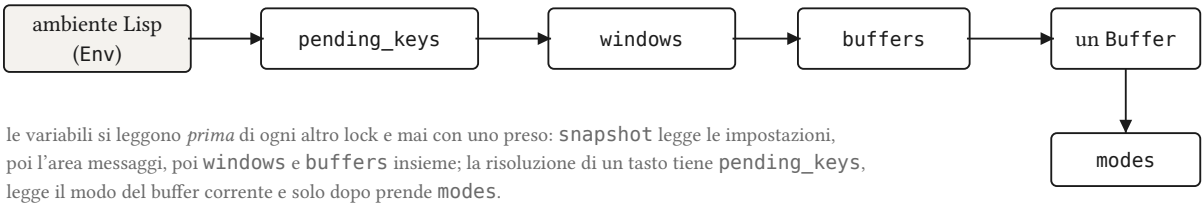
`managers/` ed `editor/` si accoppiano per nome, con una regola sola.

- `managers/X` è il compartimento: i campi sotto un lock e le operazioni che non hanno bisogno d'altro.
- `editor/X` è la sua metà della facciata: i metodi di `EditorState` che danno alle operazioni di X un nome da editor (`command_specs` per `Commands::specs`, perché la facciata ha anche nomi di buffer e di finestre) e ciò che usa X insieme a un altro compartimento, al testo di un buffer o all'interprete.
- Le porte dei lock stanno tutte in `editor/mod.rs`, accanto ai campi: è il solo file che prende i lock dei compartimenti. Una porta sola ne tiene due, `windows_mut_with_buffers`, per `snapshot`, che compone ogni finestra sul buffer che mostra (Sezione 4.5).
- Un metodo che inoltra una chiamata sola al compartimento X sta in `editor/X.rs`, chiunque lo chiami: i cinque del tema in `runtime.rs`, quelli che il mouse chiede alle finestre in `windows.rs`.
- Un compartimento senza metodi di facciata non ha un file in `editor/`: oggi `history`, le cui porte bastano.
- Chi ha bisogno di un compartimento solo, primitive comprese, usa la porta: `ctx.modes(|m| …)` in una primitiva è previsto.

Gli altri file di `editor/` portano il nome di ciò che coordinano — `keys.rs`, `mouse.rs`, `frame.rs`, `boot.rs`, `settings.rs`, `replace.rs`, `results.rs`, `background.rs` — e tengono solo ciò che mette insieme più cose: un compartimento e il testo di un buffer, l'interprete o un altro compartimento. `work.rs` è la facciata del WorkManager, che non è un compartimento: lo si raggiunge intero (`work()`), perché non ha un lock che si possa prendere fuori ordine o tenere attraverso l'interprete. L'implementazione di `LispContext`, il contratto dell'editor con l'interprete, sta in `mod.rs`.

4.5 L'ordine dei lock

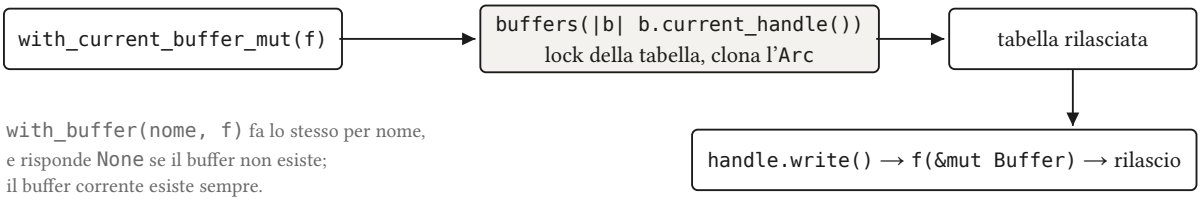
Quando un'operazione deve tenere più lock insieme, li prende in quest'ordine. Sono due a farlo. `snapshot` tiene `windows`, in scrittura, `buffers` e i buffer che compone, attraverso la sola porta che apre due compartimenti, `windows_mut_with_buffers`. La risoluzione di un tasto tiene `pending_keys` mentre legge il buffer corrente e poi interroga `modes`, e lo prende in scrittura per togliere una mappa `Release`. Ogni altro percorso copia fuori ciò che gli serve e rilascia prima di prendere il lock seguente; `echo_message` e gli altri compartimenti si prendono da soli.



le variabili si leggono *prima* di ogni altro lock e mai con uno preso: `snapshot` legge le impostazioni, poi l'area messaggi, poi `windows` e `buffers` insieme; la risoluzione di un tasto tiene `pending_keys`, legge il modo del buffer corrente e solo dopo prende `modes`.

`buffers` ha una particolarità: nel percorso comune non viene tenuto mentre si blocca un buffer. `with_buffer` e `with_current_buffer` clonano l'`Arc` del buffer dalla tabella, *rilasciano* la tabella e solo allora bloccano il buffer. Così chi scrive in un buffer non chiude fuori dalla tabella gli altri thread — l'evidenziatore e lo scanner la percorrono mentre l'utente scrive. Soltanto `snapshot` tiene la tabella e i singoli buffer insieme, nell'ordine indicato.

4.6 Accesso ai buffer



`with_buffer(nome, f)` fa lo stesso per nome, e risponde `None` se il buffer non esiste; il buffer corrente esiste sempre.

`Buffers` non è mai vuoto: nasce con `*scratch*` corrente. Le funzioni che cambiano quale buffer è corrente passano tutte da `make_current` o, per `set-buffer`, che agisce su un buffer senza mostrarlo, da `bind_current`, che non tocca l'ordine dei buffer recenti: tutte e due rifiutano un nome assente dalla tabella (Sezione 4.8).

4.7 L'editor come contesto dell'interprete

`EditorState<B>` è il `T: LispContext` dell'interprete:

Metodo del trait	Implementazione
`consume_fuel(n)`	`FuelMeter` di `Runtime` (clonato fuori dal lock); `Exhausted` diventa `OutOfFuel`.
`log_diagnostic(msg)`	`Log::append`: la riga va nella lista; il file lo scrive il watchdog (`Log::flush`).
`with_unwind(cleanup)`	Esegue la pulizia di un `unwind-protect` con il carburante che resta e, finito quello, con una riserva di 100.000 passi, che non passa alla valutazione ordinaria e non si ricarica fino al prossimo scope esterno.
`begin_thread_evaluation()`	`arm_thread()`: il nuovo thread riceve il budget configurato.

Metodo del trait	Implementazione
`push_call_frame` / `pop_call_frame` / `call_frame_depth` / `truncate_call_frames`	La pila `call_stack: Vec<String>` di `Runtime`, che `report_error` legge come backtrace.

Lo scope di carburante dell'editor è `begin_command()`: lo aprono `run_command_form`, ogni hook, `report_error`, ogni callback dovuta, ogni funzione di `after-resize-hook` ed `eval_file`. Solo l'apertura più esterna ricarica il budget (10.000.000 di passi), quindi un `eval-file` dentro un comando consuma il budget del comando, e così `pre-command-hook`, che gira dentro lo scope del comando. `post-self-insert-hook` e `post-command-hook`, invece, girano dopo che lo scope si è chiuso: ogni loro funzione riparte con il budget pieno.

4.8 Le decisioni

1. **Lo stato è diviso in compartimenti, ognuno sotto un lock.** Un compartimento raccoglie i campi che sono un fatto in più pezzi. Sotto lock separati si vedrebbero a metà di un cambiamento — il focus che nomina una finestra che il layout ha già tolto — e ogni operazione che ne tocca due dovrebbe prenderli in un ordine scritto altrove. Sotto un lock solo non c'è un ordine da rispettare, e il disaccordo non si può vedere.
2. **`current` non nomina mai un buffer assente.** Nella tabella, `current` è una chiave e `recency` una lista di chiavi: due pezzi su tre valgono solo rispetto al terzo. `current` si cambia solo con `make_current` e `bind_current`, che rifiutano un nome assente, e `rename` e `remove` lo spostano nello stesso passo, non lasciandolo al chiamante. Un nome assente diventato corrente farebbe fallire la ricerca successiva con un panic, e con l'editor andrebbe perso ciò che non era salvato nelle altre finestre.
3. **`Buffers` non legge il testo e non conosce le finestre.** Ciò che un buffer contiene è di `Buffer`, e chi lo mostra sono le finestre, che ne tengono il nome. Il rapporto è a senso unico, e i due compartimenti non hanno bisogno l'uno del lock dell'altro: quando un nome cambia o un buffer sparisce, `BufferRenamed` e `BufferRemoved` lo dicono e la facciata ripunta le finestre. `rename` non tocca neppure il campo `name` del buffer: bloccare un buffer tenendo la tabella metterebbe la tabella in coda a chi sta scrivendo nel buffer.
4. **I campi di `Commands` sono la vita di un comando.** Il registro dice che cosa può eseguire `M-x` e con quali argomenti, l'argomento prefisso è il conteggio scritto prima, la pila è ciò che resta da raccogliere, gli ultimi due campi sono ciò che resta al comando successivo. Si scrivono tutti nel corso di un tasto, e l'ordine in cui lo sono è il protocollo. Che il prefisso sia ancora in lettura è un campo a sé: «nessun argomento» e «un argomento finito» sono stati diversi, e una cifra significa cose diverse nei due.
5. **Gli hook stanno con i modi, non con i comandi.** Sembrano macchinario dei comandi, ma un hook appartiene a un modo: quelli registrati per tutti i modi sono la metà globale di `MajorMode::hooks`, come la keymap globale lo è di `MajorMode::keymaps`.
6. **Guardare indietro nella storia non fa perdere ciò che si scriveva.** Il primo passo indietro mette da parte il testo del prompt, e andare avanti oltre la voce più recente lo restituisce. Altrimenti, per tornare a ciò che si stava scrivendo, bisognerebbe annullare il prompt e ricominciare.
7. **La storia non ricorda tutto.** Non ricorda la stringa vuota, che è come si esce da un prompt, né la ripetizione della voce più recente: lo stesso comando eseguito quattro volte lascia una voce, o tornare indietro di un passo costerebbe quattro tasti. Una ripetizione più vecchia resta dov'è e si aggiunge di nuovo in testa: spostarla riordinerebbe la storia sotto l'utente. Il tetto, cento voci come in Emacs, è per prompt, così un prompt usato di continuo non spinge fuori le voci di un altro. Ai due capi di un anello il prompt resta com'è, invece di ricevere una copia di sé: il tasto deve poter dire «sei alla fine».
8. **Anello e flag di `KillYank` stanno insieme.** L'anello da solo sarebbe un campo. Ne fanno un compartimento le due domande che Emacs fa a ogni kill e a ogni yank: il comando precedente ha ucciso? ha incollato? Una serie di `C-k` deve fare una voce sola, e `yank-pop` deve sapere quale testo ha appena

inserito uno yank. Sotto lo stesso lock «accodare o aggiungere» e «flag ed estensione dello yank» sono ciascuno un'operazione sola.

9. **Due flag per domanda.** Un comando alza `killed_this` mentre gira, e `roll_over` lo sposta in `killed_last` quando è finito; lo stesso per lo yank. Con un flag solo, un comando non potrebbe chiedere che cosa ha fatto il precedente senza vedere anche ciò che ha appena fatto lui. Le due coppie si spostano insieme, perché un comando chiede di tutte e due.
10. **Il rettangolo ucciso non sta nell'anello.** Un rettangolo è una forma, e l'anello contiene testo: con un rettangolo nell'anello ogni yank dovrebbe chiedersi che forma ha, e un blocco copiato da un terminale non si distinguerebbe dalle righe di cui è fatto. È un posto e non un anello suo, perché non c'è uno `yank-pop` dei rettangoli; e due kill di rettangolo di seguito sono due rettangoli, non un blocco che sullo schermo non c'è mai stato.
11. **La lista del log e il suo file sono una cosa sola.** Il file è lo specchio della lista: abilitarlo scrive tutto ciò che è stato registrato fino a quel momento, così il file contiene la storia intera e non solo ciò che è venuto dopo. Funziona solo se lista e file si sistemano insieme, sotto lo stesso lock: altrimenti una riga registrata in mezzo da un altro thread arriverebbe troppo tardi per la storia e troppo presto per il file.
12. **Il file del log non lo scrive chi registra.** Registrare aggiunge una riga e la conta come non scritta, e basta: niente disco, quindi registrare costa lo stesso con il disco pieno o con la directory di casa in rete, e nessun comando, nessun passo dello scheduler aspetta una scrittura. Scrive `Log::flush`, che il watchdog chiama a ogni giro, un quarto di secondo per impostazione: su un thread che non è bloccato quando il resto lo è, che è quando un log serve. Un'ultima volta alla chiusura.
13. **`Log` prende il suo lock da sé e non va mai in panic.** È il solo compartimento con funzioni che ricevono il proprio `RwLock`, perché il WorkManager registra senza tenere un editor (Sezione 11). Una scrittura che fallisce si perde, lasciando un buco nel file, e un lock avvelenato si usa così com'è: `flush` gira sul watchdog, che riporta i panic degli altri e non deve averne di suoi — con il disco pieno, salvare un file registra una diagnostica la cui scrittura fallisce a sua volta.
14. **Una macro registra tasti, non comandi.** Non ogni tasto diventa un comando: `C-u` e le cifre li prende il lettore dell'argomento prefisso, e `zap-to-char` legge il suo carattere con una cattura di tasti, senza un secondo comando. Un registratore che guardasse i comandi riprodurrebbe `zap-to-char` come l'attesa di un carattere che non arriva. I tasti seguono poi ciò che significano *ora*: si cambia un legame e la macro fa la cosa nuova, come in Emacs. E si scrivono gratis: `describe_keys` e `parse_key_sequence` sono inversi, quindi una macro è una stringa nella sintassi di `define-key`.
15. **Registrazione e riproduzione stanno insieme.** Finire una registrazione sposta i tasti nel posto dell'ultima macro: separati, un lettore in mezzo vedrebbe due copie o nessuna. E se un tasto si registri è una domanda su registrazione e riproduzione insieme: una macro che ne chiama un'altra registra la chiamata, non ciò in cui si espande. I comandi del registratore (`NOT_RECORDED`) non si registrano, perché i tasti che chiudono una registrazione si premono mentre gira: l'elenco è per nome e sta accanto allo stato, perché quali comandi siano del registratore è una domanda sua.
16. **Le riproduzioni annidate hanno un tetto.** Una riproduzione è una chiamata Rust attraverso il gestore dei tasti, quindi una macro che chiama se stessa fa crescere la pila, che finisce molto prima del carburante — se ne spendono pochi passi per livello — e una pila finita chiude il processo invece di dare un errore. Sedici livelli bastano: l'annidamento legittimo, una macro che chiama una macro con nome che ne chiama un'altra, ne usa due o tre.
17. **Le metà globali stanno con il registro dei modi.** La keymap di un modo e quella globale si consultano in quest'ordine a ogni tasto, e le sorgenti di completamento del modo e quelle globali si fondono a ogni completamento. Separate, chi le legge dovrebbe sapere che i posti sono due e prenderli nell'ordine giusto. La keymap transitoria e i tasti di ripetizione stanno sotto lo stesso lock, così risolvere un tasto prende un lock solo.
18. **Gli hook e le sorgenti si copiano fuori prima di chiamarli.** Le liste di `Modes` contengono espressioni Lisp, e niente in `Modes` le chiama: ogni lettura le copia e rilascia il lock. Un hook che chiama

`add-hook`, `define-key`, `make-mode` o `add-syntax-rule` vuole questo lock in scrittura, e con il lock tenuto si bloccherebbe ogni volta, su un thread solo, con un Lisp qualunque.

19. **Il cammino tra le keymap si scrive una volta.** `binding_for`, `is_prefix` e `bindings` rispondono sui legami in effetto accanto alla risoluzione che l'editor esegue, e ne condividono il cammino. Un secondo cammino scritto altrove sarebbe una seconda risposta a «che cosa fa questo tasto», e le due divergerebbero la prima volta che l'ordine cambiasse. Ogni legame porta con sé la mappa da cui viene (`BindingSource`): «`q` esegue `quit-window` qui, perché il buffer è in `dired-mode`» è quasi tutto ciò che vuole sapere chi cerca un tasto.
20. **I modi per nome di file sono una lista ordinata.** I modelli si sovrappongono — `\.rs$` e `^Cargo\.` corrispondono entrambi a `Cargo.rs` — e quale vince deve averlo deciso qualcuno, non un hash: vince il primo dichiarato.
21. **I tasti di ripetizione si dichiarano.** La regola che verrebbe in mente, «dopo una sequenza che finisce con K, un K da solo ripete», farebbe di `f` dopo `C-x C-f` un altro `find-file`.
22. **`Runtime` raggruppa per durata, non per un'invariante.** Gli altri compartimenti tengono insieme un fatto in più pezzi; questi campi no: nessuna operazione ne tocca due, e metterli insieme non chiude nessuna finestra di disaccordo. Il guadagno è più piccolo: la facciata non porta otto lock indipendenti, e chi cerca lo stato dell'editor, distinto da quello di un documento, ha un posto solo. Il tema sta qui in mancanza di meglio: è dell'interfaccia, e si sposterà quando ci sarà un compartimento suo.
23. **Il carburante non sta sotto il lock di `Runtime`.** `FuelMeter` si sincronizza da sé: il budget vivo è una variabile del thread, e l'atomico dietro si legge solo quando si apre uno scope. Si tiene come `Arc` e si dà per clone, così chi apre uno scope, che dura un comando intero, non tiene il lock attraverso l'interprete.
24. **Le finestre non leggono i buffer.** Una finestra nomina un buffer e non sa altro. Dove un'operazione ha bisogno di qualcosa del buffer — lo scorrimento il numero di righe, `C-l` la riga del punto — riceve il numero: i due compartimenti non si tengono mai insieme, e le operazioni si provano senza un editor. Per lo stesso motivo un clic risponde con una colonna dello schermo, che `editor/mouse.rs` traduce in una colonna del buffer: una riga con testo virtuale è più larga della sua riga di testo.
25. **I doppi clic li conta l'editor.** Un terminale manda una pressione per volta e nient'altro: «questa è la seconda nello stesso punto» lo sa solo chi tiene la prima, e deve essere l'editor, perché in un front-end sarebbe un calcolo da rifare in ogni front-end. Due clic sono una coppia nella stessa cella entro 400 ms, un po' meno del `double-click-time` di Emacs. La stessa cella e non una vicina, perché un clic subito accanto al precedente sposta il cursore di un carattere, e leggerlo come doppio sarebbe l'errore più fastidioso. Dopo una coppia il primo clic si dimentica: tre pressioni sono una coppia e una singola.
26. **Un trascinamento si ricorda, e scorre a tempo.** È l'unico gesto del mouse che non è un evento solo: dove è cominciato decide che cosa significano gli eventi dopo, ovunque sia andato il puntatore. Un puntatore fermo fuori dalla finestra non manda eventi, ma la selezione deve continuare: per questo lo scorrimento durante il trascinamento è un timer di 60 ms, non una risposta al movimento.
27. **Una striscia divide il frame e non prende il focus.** `open_bottom` apre una finestra alta esattamente quanto chiesto sotto tutte le altre, che cedono ciascuna una parte dello spazio: una striscia è del frame, non di una finestra tagliata a metà. Il focus resta dov'è, perché la striscia si mostra mentre si scrive altrove — i completamenti sotto un prompt — e il focus che si spostasse renderebbe corrente il suo buffer: il tasto successivo finirebbe nella lista dei suggerimenti.
28. **I float sono nel giro delle finestre, ma non nel loro conto.** `all_ids` mette i float dopo le finestre affiancate, dal più vecchio, perché un float da cui il focus è uscito sarebbe altrimenti irraggiungibile. `count` conta solo le affiancate, perché decide se una finestra si può chiudere e che cosa annulla `C-x 1`, e un prompt disegnato sopra non divide nulla.
29. **Scorrere sposta la vista, e il punto la segue.** `C-v` sposta la vista e il punto la segue solo se ne è uscito, come in Emacs: spostare il punto e lasciare che il disegno lo insegua lo porterebbe al bordo invece di tenergli il posto sullo schermo. Due righe restano in comune tra una schermata e l'altra. La rotella segue la stessa regola nella finestra con il focus, e nelle altre lascia il punto dov'è: il punto è del buffer, e un'altra

finestra può mostrare lo stesso buffer. La rotella porta l'ultima riga fino in cima, `C-v` solo fino in fondo: la prima guarda la fine di un file, il secondo lo legge.

30. **La facciata dà ai compartimenti nomi da editor.** `is_command` inoltra a `Commands::is_command`, e sembra un doppione; ma per lo più rinomina: `Commands::specs` è chiaro perché il tipo dice di che cosa parla, mentre `EditorState`, che ha anche nomi di buffer e id di finestre, ha bisogno di `command_specs`. Togliere gli inoltri metterebbe in ogni chiamata quale compartimento tiene che cosa, e spostare un campo tra due compartimenti cambierebbe ogni chiamante invece di un inoltro.
31. **Un inoltro sta con il suo compartimento.** Chi cerca che cosa l'editor offre su un compartimento lo trova in un file, e chi scrive un inoltro nuovo sa dove metterlo, chiunque sia a usarlo. I file di coordinamento restano fatti di coordinamento.
32. **Una porta sola tiene due compartimenti.** `snapshot` compone ogni finestra contro la tabella dei buffer, e ne sposta lo scorrimento: ha bisogno di `windows` in scrittura e di `buffers` insieme. La porta che li apre entrambi li prende nell'ordine dei lock, scritto così in un posto solo (Sezione 4.5).
33. **Il contatore delle grammatiche sta accanto al registro, non dentro.** Una colorazione porta il timbro della grammatica sotto cui è stata calcolata, e aggiungere una regola non cambia il testo: senza il contatore nessuno se ne accorgerebbe. Ogni fotografia chiede a ogni buffer se la sua colorazione è indietro, e un atomico fuori dal registro risponde senza prenderne il lock. Si sposta solo sotto il lock in scrittura del registro, dopo il cambio della grammatica, e lo si legge sotto quel lock insieme alla grammatica: le due cose non si vedono mai in disaccordo. Una tabella sintattica che cambia emette `syntax-changed` e non sposta il contatore: è dell'altro lessico.
34. **Il messaggio e il suo istante stanno sotto un lock solo.** Con due lock, chi legge potrebbe accoppiare un messaggio nuovo all'istante del vecchio e nasconderlo appena scritto. Ogni messaggio fa ripartire il suo tempo, e un messaggio scaduto resta nello stato: è la vista che lo dimentica.
35. **Le callback dovute non sono un compartimento.** Ogni compartimento non sa nulla del Lisp, e quella lista ne è fatta: metterla dietro un compartimento romperebbe la regola, o chiederebbe una seconda rappresentazione di una chiamata, inventata per non dire che cos'è. Sta accanto a `current_results`, come stato che la facciata tiene da sé.
36. **Finestra con il focus e buffer corrente cambiano insieme.** Lo fanno in due modi soli: `show_in_focused_window` cambia il buffer e lascia il focus, `set_focused_window_id` sposta il focus e rende corrente ciò che la finestra mostra, e ogni spostamento del focus finisce in `follow_focus`, anche quando è il compartimento a spostarlo. Se i due andassero alla deriva, l'editor disegnerebbe il cursore in una finestra e scriverebbe in un'altra.
37. **In un buffer in sola lettura si scrive sotto il suo lock.** Una trascrizione — l'uscita di un comando di shell, i risultati di una ricerca — è in sola lettura perché nessuno ci scriva. `append_to_buffer` abbassa e rialza il flag dentro un solo lock in scrittura: non c'è un momento in cui il buffer è visibile e scrivibile. `dired` fa lo stesso dal Lisp, dove non può tenere un lock e deve fidarsi che nulla giri in mezzo.
38. **Un file che non c'è si apre vuoto e non modificato.** `new_buffer` con un percorso legge il file e fallisce se non può; un file che non esiste non è un errore, è il modo comune di crearne uno. `new_file_buffer` fa un buffer vuoto che ricorda dove andrà, non modificato: lasciarlo non perde nulla, e `C-x C-s` crea il file anche se non si è scritto niente.
39. **Un buffer chiuso con modifiche non salvate lascia la sua copia.** La copia del salvataggio automatico si butta quando ciò che teneva è nel file: a un salvataggio, e alla chiusura di un buffer senza modifiche. `close_buffer` non chiede nulla, e non chiedono `close-buffer` e `kill-buffer-without-saving` che lo chiamano: la copia può essere tutto ciò che resta del lavoro.

5 Dal tasto al comando

5.1 Il vocabolario dei tasti

Un `KeyEvent` è un `KeyCode` (`Char(c)`, `Backspace`, `Enter`, `Esc`, `Tab`, le quattro frecce, e `None`, che nessuno costruisce) più i modificatori `ctrl`, `alt`, `shift`. Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione non hanno un `KeyCode` (→ CMD-2). Un `MouseEvent` dice che cosa ha fatto il mouse (`MouseKind`: un pulsante premuto, rilasciato o trascinato, la rotella), in quale cella del frame e con quali modificatori. Sono i tipi che un front-end costruisce e consegna a `handle_key_event` e a `handle_mouse_event`.

Nelle associazioni i tasti si scrivono come in Emacs: `C-x`, `M-x`, `C-M-f`, `S-<tab>`, con i nomi `<ret>`, `<esc>`, `tab`, `<backtab>`, `<backspace>`, `<space>`, `<left>`, `<right>`, `<up>`, `<down>`. Un tasto ha al più un prefisso di modificatori, quindi `M-C-x` e `C-S-a` non si possono scrivere (→ CMD-4). Una sequenza è una stringa di tasti separati da spazi: `"C-x C-f"`. `parse_key_sequence` legge queste stringhe e `describe_keys` le riscrive nello stesso modo; stanno in `input.rs`, con i tipi.

La keymap globale scritta in Rust (`fill_default_keymaps`) contiene pochissimo: `M-x` (`command-execute-prompt`), le frecce, Invio e i caratteri stampabili da `' '` a `'~'`, legati a `(self-insert "c")`. Tutto il resto — `C-x C-f`, `C-s`, `C-k`, … — viene da `common-keymaps.lisp` e dagli altri moduli.

Solo i caratteri ASCII stampabili sono legati a `self-insert`. Il ripiego per gli altri caratteri esiste — una keymap può dare un comando a ogni carattere che non lega, con `(define-key MAPPA "<char>" COMANDO)` — ma la keymap globale non lo usa: un carattere come «è» arriva alla risoluzione senza un comando e produce `è is undefined` (→ CMD-1).

5.2 Le keymap

Una `Keymap` associa sequenze di tasti a espressioni Lisp: un nome di comando (`forward-char`), una forma che porta già i suoi argomenti (`(self-insert "x")`) o una lambda. Il tipo è `ELispExp`, che `run_command_form` esegue così com'è; ed è l'unica cosa che `input.rs` prende da più in alto, perché il suo tipo nomina `EditorState`.

da `input.rs` — estratto semplificato

```
pub struct Keymap<B: BufferTrait> {
    bindings: HashMap<Vec<KeyEvent>, ELispExp<B>>,  // la sequenza intera
    prefixes: HashSet<Vec<KeyEvent>>,               // ogni prefisso proprio
    character_handler: Option<ELispExp<B>>,         // il ripiego dei caratteri
}
pub fn insert(&mut self, keys: Vec<KeyEvent>, ast: ELispExp<B>) {
    for len in 1..keys.len() {
        self.prefixes.insert(keys[..len].to_vec());
    }
    self.bindings.insert(keys, ast);
}
```

Ogni tasto chiede due cose a una keymap: se la sequenza è un legame (`get`) e se è l'inizio di uno (`is_prefix`). Tutte e due sono la ricerca in una tabella hash: `insert` registra i prefissi propri della sequenza insieme al legame, e `remove` ricostruisce l'insieme dai legami che restano. `binding` aggiunge il ripiego: un carattere senza modificatori che non è un legame né l'inizio di uno va al `character_handler`, se c'è; un simbolo diventa `(simbolo "c")`, ogni altra funzione `(funcall 'f "c")`. `set_character_handler` lo installa, ed è ciò che fa `(define-key MAPPA "<char>" ...)`; con `nil` lo toglie.

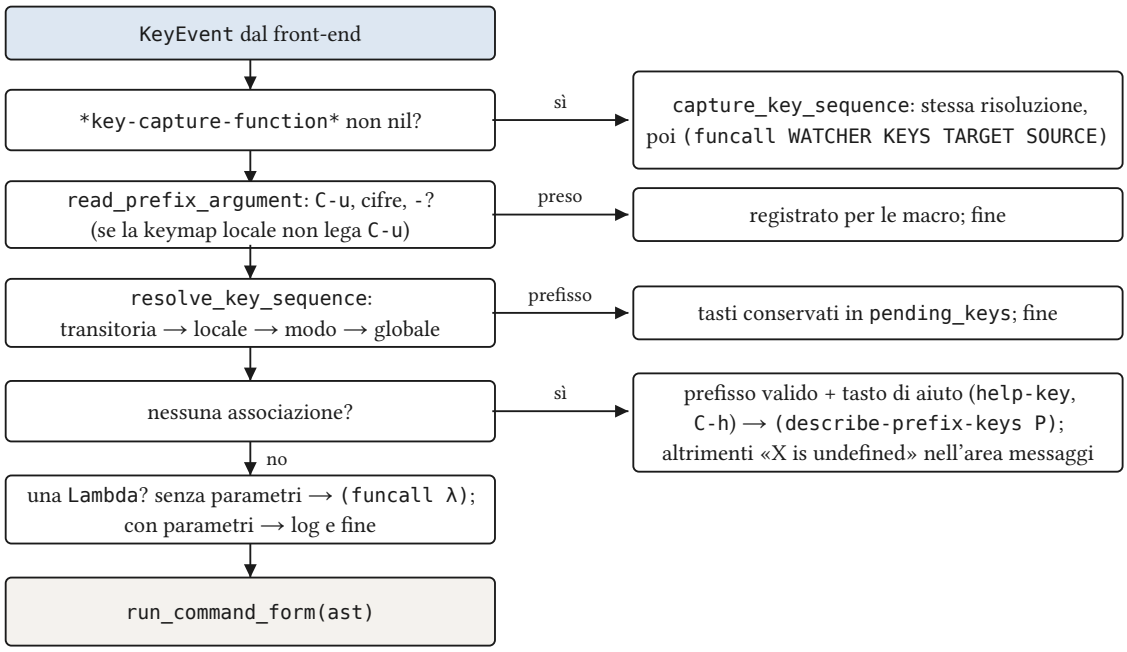
Le keymap stanno nel compartimento `Modes`: la globale, quella di ogni modo e la *transitoria*, consultata prima delle altre finché resta installata. Una `TransientKeymap` è una `Keymap`, il messaggio da mostrare mentre è attiva e che cosa fare di un tasto che non lega (`OnUnbound`, nella tabella della risoluzione). La installano le domande a un tasto (`y-or-n`, `save-some-buffers`), la vista predefinita di `query-replace`,

la striscia dei completamenti e la ripetizione: `TransientKeymap::repeating` costruisce la mappa `[o]`, che `install_repeat_keymap` rimette dopo ogni comando con un tasto di ripetizione.

5.2.1 Le decisioni

1. **I prefissi si conservano, non si cercano.** Ogni tasto chiede se la sequenza è l'inizio di un legame. Rispondere scorrendo i legami vorrebbe dire scorrere tutta la mappa, centinaia di voci per i soli caratteri, sul percorso tra il tasto e il carattere che compare. L'insieme cambia solo quando si definisce un legame, all'avvio o quando l'utente cambia la sua configurazione; la domanda si fa a ogni tasto.
2. **Legame e prefissi si registrano insieme.** Un legame senza i suoi prefissi sarebbe irraggiungibile: `C-x` da solo risulterebbe non definito, e la sequenza verrebbe buttata prima di arrivare a `C-x C-f`. Per questo i campi sono privati e `insert` li scrive tutti e due: lo stato sbagliato non si può costruire, invece di essere solo sconsigliato. Togliere un legame è raro, e `remove` ricostruisce i prefissi invece di contarli.
3. **Domande e offerte sono lo stesso meccanismo.** «Premi `y` o `n`» e «premi ancora `o`» sembrano due funzioni diverse, ma sono una keymap consultata per prima che poi se ne va: cambia solo `OnUnbound`. Essere una keymap, invece di un ciclo che legge le risposte, fa delle risposte comandi normali: si legano come tutto il resto, l'aiuto li descrive, e ognuno è un comando a sé per l'undo.
4. **Una mappa `Release` non ha un'uscita.** Se ne va dentro la risoluzione del tasto, mentre la sequenza è ancora sotto lock, dove non si può chiamare l'interprete. Quindi non deve portare con sé uno stato da ripulire: una mappa con una sessione dietro deve essere una `Refuse`, che finisce con i suoi tasti, dove un comando può ripulire. La vista predefinita di `query-replace` non segue la regola (→ RIC-8).

5.3 `handle_key_event`



I tasti vengono registrati per la macro di tastiera in corso anche quando non eseguono nulla: una macro deve riprodurre ciò che è stato premuto, compresi `C-u 5` e i tasti non associati.

Il front-end chiama `handle_key_event` solo per gli eventi di pressione; l'incolla arriva da `handle_paste` come un unico comando `(insert-pasted-text TESTO)`, che passa anch'esso da `run_command_form`.

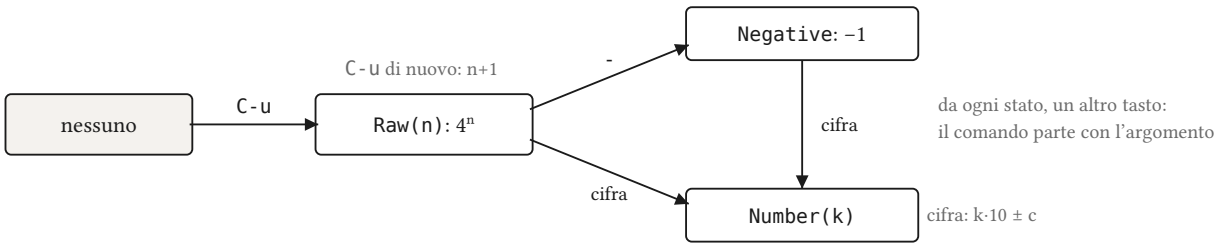
In codice, la funzione è una sequenza di uscite anticipate:

da `editor/keys.rs` — estratto semplificato

```
pub fn handle_key_event(&self, event: KeyEvent, env: &Arc<Env<EditorState<B>>>) {
    if let Some(watcher) = env.get_variable(KEY_CAPTURE_FUNCTION) && watcher.is_truthy() {
        self.record_keys(&[event.clone()], None);
        self.capture_key_sequence(event, watcher, env);     // read-key-sequence
        return;
    }
    if !(Commands::is_prefix_argument_key(&event) && self.local_key_claims(&event))
        && self.read_prefix_argument(&event)                // C-u, cifre, -
    {
        self.record_keys(&[event], None);
        return;
    }
    let mut sequence = self.pending_key_events();
    sequence.push(event.clone());
    let mut ast = match self.resolve_key_sequence(event.clone()) {
        Dispatch::Run(ast) => { self.record_keys(&sequence, command_named_by(&ast)); ast }
        Dispatch::Nothing  => return,                       // un prefisso: si aspetta
        Dispatch::Unbound { described, prefix, last } => {
            /* prefisso + tasto di aiuto → describe-prefix-keys; altrimenti
               "X is undefined" nell'area messaggi */
            return;
        }
    };
    if let ELispExp::Lambda(ref lambda) = ast {             // un tasto legato a una lambda
        if !lambda.params.is_empty() { /* log */ return; }
        ast = ELispExp::form(vec![ELispExp::symbol("funcall"), ast]);
    }
    self.run_command_form(ast, env);
}
```

5.4 L'argomento prefisso

`C-u` e le cifre non arrivano alle keymap: `Commands::read_prefix_argument` li prende per primo, a meno che la keymap locale del buffer (`set-local-keymap`) non leghi `C-u` lei stessa (`local_key_claims`). Lo stato è la coppia `prefix_arg: Option<PrefixArg>` e `reading_prefix: bool`.



Un tasto qualunque che non fa parte dell'argomento chiude la lettura (`reading_prefix = false`) ma lascia l'argomento al comando che segue. Il comando lo legge tramite `call-interactively` (le specifiche `p` e `P`); dopo *ogni* comando, riuscito o fallito, `clear_prefix_argument` lo cancella. Non esistono le forme `M-4` o `C-4` di Emacs. Mentre l'argomento e la sequenza sono a metà, `pending_input()` li descrive (`C-u 4 C-x-`) e la TUI li mostra a destra nella riga dei messaggi.

I caratteri sono legati a forme che portano già il loro argomento, `(self-insert "x")`; una forma così non passa da `call-interactively`, e quindi non riceve l'argomento prefisso: `C-u 3 x` inserisce una sola «x» (→ CMD-5).

5.5 La risoluzione: `resolve_key_sequence`

Il tasto si aggiunge a `pending_keys`; poi, con un solo lock di lettura su `modes`, la sequenza viene cercata in quattro keymap, in ordine:

1. la *keymap transitoria*, se installata;
2. la keymap locale del buffer corrente, se `set-local-keymap` gliene ha data una;
3. la keymap del modo maggiore del buffer corrente;
4. la keymap globale.

da `editor/keys.rs` — estratto semplificato

```
let mut pending = self.pending_keys.write().unwrap();
pending.push(event);
let names = self.keymap_modes();     // la keymap locale, se c'è, e il modo
let current_modes: Vec<&str> = names.iter().map(String::as_ref).collect();
let bound = self.modes(|modes| {
    if let Some(map) = modes.transient() {
        match (
            map.keymap.get(&pending).cloned(),
            map.keymap.is_prefix(&pending),
            map.on_unbound,
        ) {
            (Some(ast), _, _) => return Bound::Command(ast),
            (None, true, _)   => return Bound::Prefix,
            (None, false, OnUnbound::Refuse)  => return Bound::Refused,
            (None, false, OnUnbound::Pass)    => {}
            (None, false, OnUnbound::Release) => release_transient = true,
        }
    }
    // la keymap locale, poi il modo, poi la keymap globale
    let hit = modes.binding_below_transient(&current_modes, &pending);
    let prefix = modes.is_prefix_below_transient(&current_modes, &pending);
    match (hit, prefix) {
        (Some(binding), _) => Bound::Command(binding.target),
        (None, true)  => Bound::Prefix,
        (None, false) => Bound::Unbound,
    }
});
if !matches!(bound, Bound::Prefix) { pending.clear(); }
```

Una sequenza è cercata *intera*: `pending_keys` contiene già i tasti precedenti, e la tabella di una keymap associa sequenze di tasti, non tasti singoli. Un risultato `Prefix` lascia la sequenza in `pending_keys`, in attesa del tasto successivo; ogni altro risultato la svuota. Un modo che lega un prefisso tiene viva la sequenza anche se solo la globale la completa. Le stesse due funzioni (`binding_below_transient`, `is_prefix_below_transient`) rispondono anche a `key-binding` e ai comandi di aiuto, così ciò che l'aiuto riporta e ciò che l'editor esegue non possono divergere.

La keymap transitoria (`TransientKeymap`) è installata da una primitiva o dal meccanismo di ripetizione e dice che cosa fare di un tasto che non conosce:

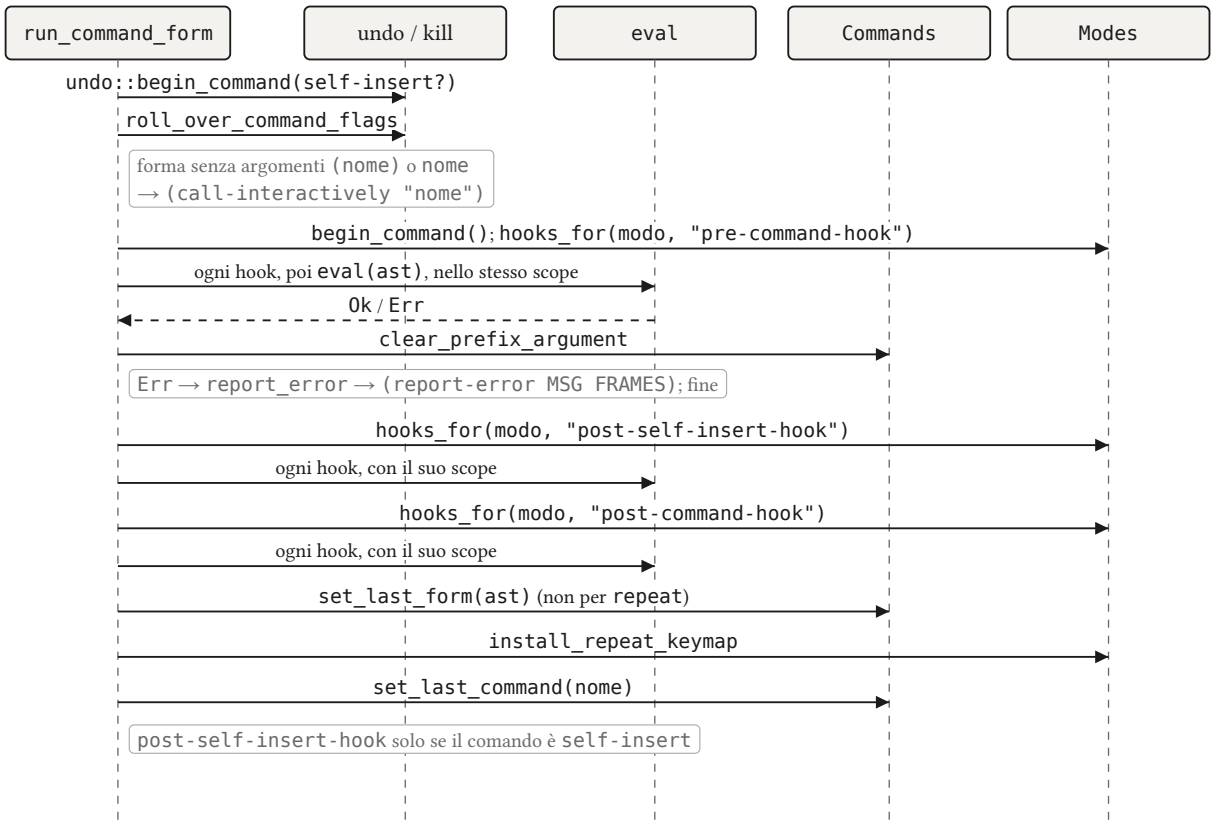
`OnUnbound`	Comportamento	Uso tipico
`Refuse`	Il tasto è rifiutato in silenzio; la mappa resta. Il messaggio della mappa occupa la riga dei messaggi.	Una domanda (`y-or-n`, `save-some-buffers`): si risponde solo con i suoi tasti.
`Pass`	Il tasto prosegue verso le altre keymap; la mappa resta.	La striscia dei completamenti: si può continuare a scrivere.

`OnUnbound`	Comportamento	Uso tipico
`Release`	Il tasto prosegue; la mappa viene tolta.	La ripetizione (`C-x o o o`): un'offerta che si può ignorare. Anche la vista predefinita di `query-replace` (→ RIC-8).

Dopo ogni comando, `install_repeat_keymap` controlla se il comando ha un *tasto di ripetizione* dichiarato (`define-repeat-key`) e in tal caso installa una mappa `Release` che lo associa al comando stesso.

5.6 `run_command_form`: l'esecuzione di un comando

Tutto ciò che il tasto, l'incolla, il mouse, la cattura e la riproduzione di macro fanno eseguire passa da qui.



da `editor/commands.rs` — estratto semplificato

```
pub(super) fn run_command_form(&self, mut ast: ELispExp<B>, env: &Arc<Env<EditorState<B>>>) {
    let bound_command = /* Some(nome) se ast è `nome` o `(nome)`, cioè senza argomenti */;
    let this_command  = /* il simbolo in testa, se c'è */;
    undo::begin_command(this_command == Some("self-insert"));     // nuova epoca
    self.roll_over_command_flags();                                // kill ring
    if let Some(name) = bound_command {
        ast = ELispExp::form(vec![
            ELispExp::symbol("call-interactively"), ELispExp::string(name),
        ]);
    }
    let outcome = {
        let _command = self.begin_command();
        let mode = self.with_current_buffer(|buf| buf.current_mode.clone());
        self.run_hook(&mode, "pre-command-hook", env);
        eval(&ast, env.clone(), self)
    };
    self.clear_prefix_argument();
    if let Err(e) = outcome {
        self.report_error(&format!("{:?} {:?}", ast, e), env); return;
    }
    let mode = self.with_current_buffer(|buf| buf.current_mode.clone());
    if this_command == Some("self-insert") {
        self.run_hook(&mode, "post-self-insert-hook", env);
    }
    self.run_hook(&mode, "post-command-hook", env);
    if this_command != Some("repeat") { self.commands_mut(|c| c.set_last_form(Some(ast))); }
    self.install_repeat_keymap(this_command);
    self.set_last_command(this_command);
}
```

1. *Undo.* `undo::begin_command` assegna al thread una nuova epoca; tutte le modifiche del comando finiranno in un solo gruppo (Sezione 8.4).
2. *Flag del kill ring.* «Il comando precedente ha ucciso» diventa vero o falso *prima* che il comando parta, così una serie di `C-k` si accumula in una sola voce del kill ring e un comando che fallisce non lascia il flag acceso.
3. *Instradamento.* Una forma senza argomenti diventa una chiamata a `call-interactively` con il nome del comando, così l'editor raccoglie gli argomenti dichiarati. Una forma che porta già i suoi argomenti — `(self-insert "a")` — resta com'è: il percorso della digitazione non paga nulla.
4. *Esecuzione.* `pre-command-hook`, poi `eval`, dentro lo stesso scope di carburante: l'hook spende il budget del comando.
5. *Argomento prefisso* cancellato, anche se il comando è fallito.
6. *Errore.* `report_error` incrementa `command_errors`, prende la pila delle chiamate come backtrace, la svuota e chiama la funzione Lisp `report-error` (definita in `debug.lisp`: messaggio nell'area messaggi e, se `debug-on-error`, una finestra `*Backtrace*`). Senza quella funzione scrive nel log. Dopo un errore non girano gli hook e `last-command` non cambia.
7. *Hook.* Prima `post-self-insert-hook` (solo per `self-insert`: qui si aggancia `electric-pair`), poi `post-command-hook`. Ogni lista è la concatenazione degli hook del modo e di quelli globali, copiata con un solo lock; un hook che fallisce viene registrato nel log e non ferma gli altri.
8. *Memoria.* La forma eseguita diventa quella di `repeat`; si offre il tasto di ripetizione; `last-command` riceve il nome.

5.7 La cattura di una sequenza

`read-key-sequence` arma `*key-capture-function*`. Il tasto successivo segue la stessa risoluzione di un tasto normale — la sequenza a metà si vede come sempre — ma al posto di eseguire il comando viene chiamata la funzione con la sequenza, la forma che sarebbe stata eseguita (citata) e il nome della keymap

che la fornisce, o `"prefix-argument"` per `C-u`. È così che `describe-key` e `zap-to-char` leggono un tasto: non esiste in rsedit un «leggi un tasto» che blocchi, perché i tasti arrivano uno per giro del ciclo degli eventi.

5.8 Le decisioni

1. **Un percorso solo esegue un comando.** Il tasto, l'incolla, il mouse, la cattura e la macro passano tutti da `run_command_form`: un secondo percorso andrebbe tenuto in passo con il primo a mano, per l'undo, gli hook e `last-command`.
2. **Un tasto che allunga una sequenza non è un comando.** `resolve_key_sequence` risponde invece di eseguire. `C-x` non deve chiudere il gruppo di undo in cui si sta scrivendo, né spezzare in due voci una serie di `C-k`: sono cose che succedono tutte dopo il punto in cui la risoluzione restituisce.
3. **La sequenza a metà sta nell'editor, non in un modo.** Una keymap di modo si consulta prima della globale, e una sequenza cominciata sotto l'una non deve restare a metà nell'altra. `pending_keys` è un vettore che si riempie e si svuota, così il tasto che è da solo un legame non alloca nulla.
4. **La cattura segue la risoluzione vera.** `read-key-sequence` non intercetta i tasti prima della risoluzione: li accumula nella stessa sequenza, così un `C-x` a metà si vede come sempre, e devia solo al posto dell'esecuzione. Ciò che fa un tasto ha una sola risposta vera, quella su cui l'editor agirebbe; una seconda implementazione delle precedenze riporterebbe quella vecchia il giorno in cui l'ordine cambia.
5. **Armare la cattura è una variabile.** `*key-capture-function*` è un fatto in un posto solo, che il Lisp legge e imposta come ogni variabile, al prezzo di una ricerca per tasto; nil quando nessuno cattura. Si disarma prima di chiamare la funzione: una funzione che apre un buffer o fallisce non deve lasciare catturato anche il tasto dopo.
6. **Il tasto di aiuto dopo un prefisso è una domanda.** Una sequenza non associata, il cui ultimo tasto è l'ultimo di `help-key` e il cui resto è un prefisso vivo, chiama `describe-prefix-keys`; senza un modulo di aiuto è indefinita come ogni altra. La variabile si legge solo su una sequenza già non associata, mai su un tasto digitato.
7. **Un incolla è un comando solo.** Il terminale dice che un incolla è un incolla. Digitato tasto per tasto sarebbe annullato un carattere alla volta, e `electric-pair` accoppierebbe di nuovo ogni parentesi: arriva invece come `insert-pasted-text`, con un gruppo di undo, un `post-command-hook` e nessun `post-self-insert-hook`.
8. **Dopo un comando senza tasto di ripetizione non si toglie nulla.** Una mappa `Release` l'ha già tolta la risoluzione, passando il tasto oltre; una `Refuse`, una domanda, risponde con i suoi comandi, e `install_repeat_keymap` gira dopo ognuno: toglierla lì la toglierebbe appena ha avuto risposta.

6 Comandi e argomenti

6.1 Che cos'è un comando

Per l'interprete un comando non esiste: è una funzione qualunque. Diventa un comando quando ha una voce nel registro di `Commands` (`register_command`), che associa al nome l'elenco degli argomenti da chiedere. Una primitiva diventa comando registrandosi con `into.command(nome, puntatore, specifiche, DOC)` invece che con `into.function`; una funzione Lisp con la macro `defcommand` di `commands.lisp`:

```
(defcommand greet (who) ("sGreet whom: ")
  "Say hello to WHO."
  (message "Hello, %s" who))
```

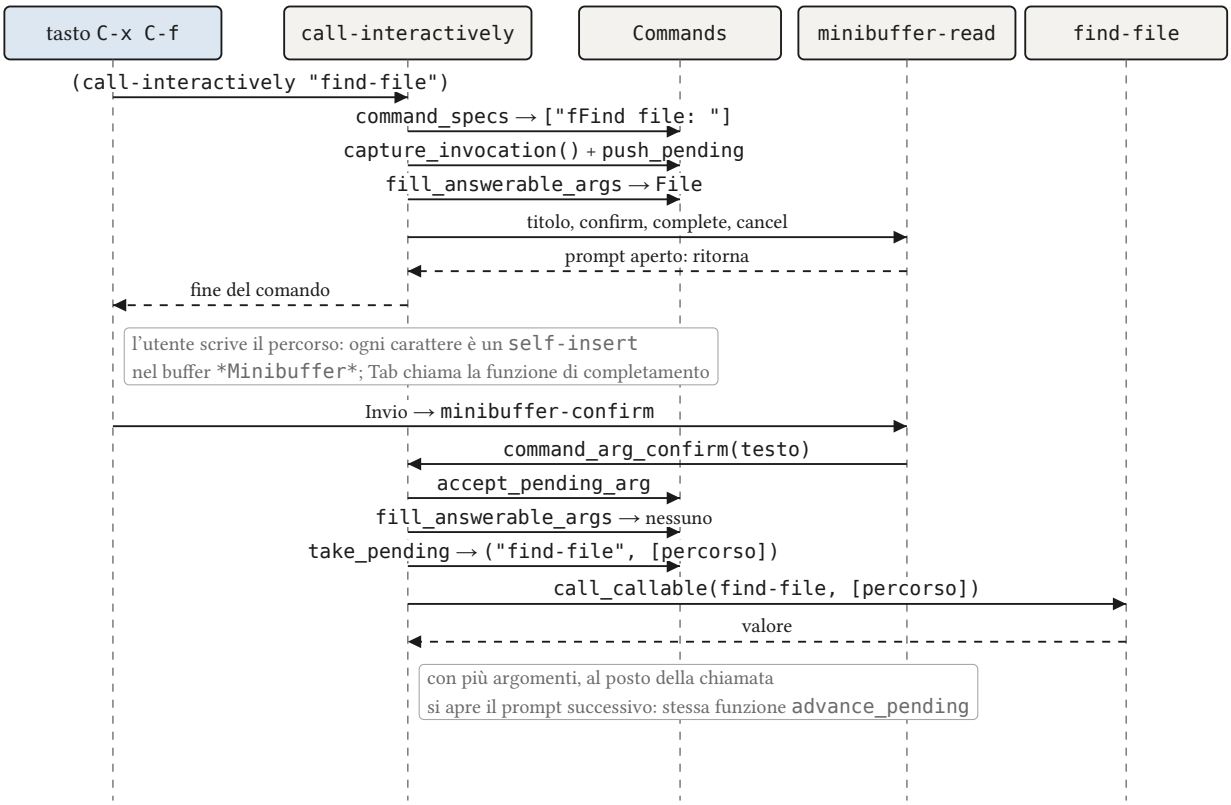
`M-x` completa solo sui nomi registrati; una funzione non registrata si può comunque legare a un tasto, e viene chiamata senza argomenti. Il registro, `CommandRegistry`, è una tabella dal nome del comando alle sue specifiche, già analizzate (`ArgSpec::parse`).

6.2 Le specifiche degli argomenti

Codice	`ArgSpec`	Valore passato al comando
`s`	`String`	Il testo digitato nel prompt.
`n`	`Number`	Il testo convertito in numero; un testo non numerico diventa 0 (→ CMD-6).
`b`	`Buffer`	Un nome di buffer, con completamento sui buffer vivi (escluso `*Minibuffer*`).
`f`	`File`	Un percorso, completato un componente alla volta.
`p`	`Count`	L'argomento prefisso come numero, 1 se assente. Non chiede nulla.
`P`	`RawCount`	L'argomento grezzo: `nil`, `(4)` per `C-u`, un numero, o il simbolo `-`. Non chiede nulla.
`r`	`Region`	*Due* valori, inizio e fine della regione attiva. Senza regione il comando è rifiutato prima di aprire qualunque prompt.

6.3 `call-interactively` e la catena degli argomenti

Il minibuffer non blocca: apre un prompt e ritorna, e la risposta arriva con un tasto successivo. Raccogliere due argomenti non può quindi essere un ciclo: è una catena, e lo stato tra un anello e l'altro sta nella pila dei `PendingCommand` di `Commands`.



`Invocation` — argomento prefisso e regione — è catturata quando il comando inizia (`capture_invocation`), non quando l'ultimo argomento arriva, e risponde agli argomenti `p`, `P` e `r` di tutta la catena.

Il titolo del prompt dice chi sta chiedendo: `find-file - Find file:` o, con più argomenti, `query-replace (2/2) - With:`. Il prompt viene aperto chiamando `minibuffer-read` per nome, così un minibuffer alternativo serve anche gli argomenti dei comandi. Annullare il prompt (`command_arg_cancel`) toglie il comando dalla pila: un elenco di argomenti incompleto non arriva mai a un comando. Un nuovo `call-interactively` senza minibuffer aperto svuota la pila dagli orfani rimasti da prompt chiusi in altro modo.

Il completamento degli argomenti `b` e `f` è fatto in Rust; una funzione in `*command-arg-completion-function*`, se impostata, lo sostituisce per tutti i tipi ricevendo il tipo come simbolo (`buffer`, `file`, …) e il prefisso.

6.4 Le decisioni

1. **Un comando è un'idea dell'editor, non dell'interprete.** Un comando è qualcosa che si raggiunge con `M-x` o con un tasto, e di cui l'editor sa chiedere gli argomenti: un nome di file con il completamento, un buffer, un numero. Per un interprete che non sa di stare in un editor non significa nulla. Così l'interprete non ne sa niente: un comando è una funzione qualunque con una voce nel registro, `(find-file "/tmp/x")` da Lisp è una chiamata come le altre, e né il tipo `Lambda` né `eval` devono sapere che i comandi esistono.
2. **Le specifiche si analizzano alla registrazione.** Un codice sbagliato, come `"zFind file: "`, diventa un errore all'avvio dell'editor invece di una sorpresa la prima volta che qualcuno esegue il comando. Per la stessa ragione un testo dopo `p`, `P` o `r`, che non chiedono nulla, è un errore e non un dettaglio da ignorare.
3. **Prefisso e regione si fissano quando il comando comincia.** Gli argomenti non arrivano tutti insieme: uno chiesto arriva qualche tasto dopo, da un minibuffer che per aprirsi ha cambiato buffer. Quando arriva l'ultimo, la regione appartiene a un altro buffer e l'argomento prefisso è già stato cancellato per il comando

successivo. All'inizio sono invece ciò che l'utente intendeva: `C-u 4 M-x foo` passa quattro a `foo`, per quanto a lungo resti aperto il prompt.

4. **I comandi in attesa sono una pila.** Un comando può cominciare mentre un altro sta ancora chiedendo i suoi argomenti (`M-x` dentro un prompt), e i due devono chiudersi nell'ordine inverso a quello in cui sono cominciati. Un posto solo perderebbe il primo.

7 Minibuffer e ricerca incrementale

7.1 Il minibuffer

Il meccanismo del minibuffer è in Rust (`feature/minibuffer.rs`, `primitives/minibuffer.rs`) perché troppe cose ne dipendono — `M-x`, `M-:`, gli argomenti dei comandi — per affidarlo a un file Lisp che potrebbe mancare. L'aspetto, invece, è sostituibile: `minibuffer-read` chiama la funzione contenuta in `*minibuffer-read-function*`, che per impostazione predefinita è `default-minibuffer-prompt`.

```
(minibuffer-read PROMPT ON-CONFIRM ON-CHANGE ON-CANCEL [MODE [HISTORY-KEY]])
```

Argomento	Significato
`ON-CONFIRM`	Chiamata con il testo finale quando si preme Invio, *dopo* la chiusura del prompt.
`ON-CHANGE`	Nonostante il nome, è la funzione di completamento: riceve il testo e restituisce i candidati. È chiamata al primo Tab dopo ogni modifica; i Tab successivi scorrono i candidati, o li consegnano a `*completion-read-function*` se impostata (è ciò che fa `completion.lisp`).
`ON-CANCEL`	Chiamata senza argomenti quando si preme Esc.
`MODE`	Il modo del buffer del prompt (predefinito `minibuffer-mode`); `isearch-mode` per la ricerca.
`HISTORY-KEY`	La chiave della storia (`M-p`, `M-n`); per default il testo del prompt, così ogni prompt ha una storia senza chiederla. Fino a `history-length` voci, 100 per default.

`default-minibuffer-prompt` salva le callback in variabili globali (`*minibuffer-on-confirm*`, `*minibuffer-on-change*`, `*minibuffer-on-cancel*`, `*minibuffer-previous-buffer*`, …) e apre una finestra flottante con bordo sul buffer `*Minibuffer*`, *al centro* del frame, larga `minibuffer-width` (60) e alta `minibuffer-height` (3) e ridotta se il terminale è più piccolo. Il centro è scelto perché il fondo dello schermo è occupato da altre strisce (i completamenti, la compilazione); alcuni commenti descrivono ancora un prompt agganciato in basso (→ DOC-1). Ogni prompt usa lo stesso buffer, `*Minibuffer*`: due prompt aperti insieme lo condividono (→ MIN-1). `C-g` nel prompt resta `keyboard-quit`, che non lo chiude; lo chiude Esc (→ MIN-2).

Un worker di sfondo non può aprire un prompt: `minibuffer-read` controlla `in_worker()` e rifiuta con un errore, perché il tasto che risponderebbe arriverebbe in mezzo a ciò che l'utente sta facendo.

7.2 La ricerca incrementale

`C-s` apre un prompt in `isearch-mode` e ricorda dove era il punto. Da lì il lavoro lo fa il `post-command-hook` di quel modo: ogni comando eseguito mentre il prompt è aperto — ogni carattere scritto, ogni cancellazione — provoca un giro di `isearch-update`, che ripete la ricerca e sposta il punto. Non serve un ciclo «leggi un tasto», che in rsedit non esiste.

> Mentre il prompt è aperto il buffer corrente è il minibuffer. Tutto ciò che riguarda la ricerca lavora quindi sul buffer cercato *per nome*, preso dallo stato della sessione (`Isearch`) in `Runtime`. Una modifica che usasse il buffer corrente cercherebbe nella riga del prompt; il test `an_isearch_searches_the_file_not_the_prompt` esiste per questo. Lo stato della sessione e l'algoritmo di ricerca sono descritti nel Sezione 9.

8 I buffer

8.1 La struttura `Buffer`

Campo	Contenuto
`name`, `file_path`	Nome e file visitato.
`text: B`	Il testo, tramite `BufferTrait` (oggi `GapBuffer`). Contiene anche il punto (il cursore).
`is_modified`, `read_only`	Modificato rispetto al disco; rifiuta le modifiche (controllato alle due porte).
`current_mode`	Il nome del modo maggiore; decide keymap, hook, grammatica e tabella sintattica.
`undo`	`UndoHistory`: gruppi fatti e disfatti (Sezione 8.4).
`mark: Option<Mark>`	Posizione e flag `active`; la regione è tra mark attivo e punto.
`overlays`	`OverlayTable`: facce su intervalli, per categoria, che seguono le modifiche.
`virtual_text`	Testo mostrato in una posizione senza essere nel buffer (suggerimenti, anteprime).
`version`	Incrementata da ogni modifica: un risultato calcolato su un'altra versione viene rifiutato.
`changed`	L'argomento emesso a ogni modifica del testo: il `buffer-changed` dell'editor, da quando il buffer vi entra (Sezione 11.4).
`syntax`, `scan`	Cache della colorazione e della scansione delle espressioni.
`file_stamp`, `stale`	Timbro (istante di modifica e lunghezza) del file all'ultima lettura o scrittura; flag «cambiato sotto di noi».
`auto_saved_at`	Versione scritta nell'ultimo salvataggio automatico.
`data`	`HashMap<String, Box<dyn Any>>`: dati che altre parti appendono al buffer e che muoiono con lui.
`writer`	Il lavoro che scrive nel buffer da un thread suo (un comando di shell, una ricerca); chiudere il buffer lo ferma.

8.2 `BufferTrait` e `GapBuffer`

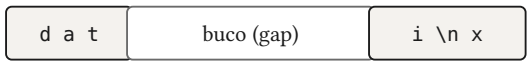
`BufferTrait` è l'unica cosa che il resto dell'editor può assumere sul testo: lunghezza, carattere in una posizione, punto in coordinate lineari e in (riga, colonna), conversioni tra le due, spostamenti del punto, inserimento e cancellazione di un carattere *al punto*, righe, `clear`. Gli spostamenti e le conversioni lavorano in caratteri, non in byte. Ciò che ogni implementazione deve fare è scritto una volta sola, per un `B` qualunque, in `buffer_conformance_tests`: una seconda implementazione è una riga in fondo a quel file, e arriva già provata. Il trait chiede anche `Clone`, ma il `clone` di `GapBuffer` è un `todo!()` (→ BUF-3).

`GapBuffer` tiene un `Vec<char>` con un buco al punto: inserire e cancellare lì costa O(1), spostare il punto costa la distanza percorsa. Accanto al testo tiene un indice delle posizioni fisiche dei `\n`, diviso in prima e dopo il buco: poiché le posizioni fisiche non cambiano quando il buco cresce o si restringe, scrivere un carattere aggiorna l'indice in O(1).

da `buffer/gap_buffer.rs` — estratto semplificato

```
pub struct GapBuffer {
    data: Vec<char>,                // testo e buco nello stesso vettore
    gap_start: usize,               // primo indice del buco = posizione del punto
    gap_end: usize,                 // primo indice dopo il buco
    newlines_before: Vec<usize>,    // posizioni fisiche dei \n prima del buco
    newlines_after: VecDeque<usize>, // posizioni fisiche dei \n dopo il buco
}
fn insert(&mut self, c: char) {
    if self.gap_start == self.gap_end { self.gap_grow(); }   // buco esaurito: raddoppia
    self.data[self.gap_start] = c;
    if c == '\n' { self.newlines_before.push(self.gap_start); }
    self.gap_start += 1;
}
```

Spostare il punto vuol dire spostare il buco: `move_gap` copia con `copy_within` i caratteri che il buco deve scavalcare dall'altra parte, e sposta tra le due metà dell'indice i ritorni a capo che attraversano il buco, correggendone la posizione fisica della larghezza del buco. Un ritorno a capo non cambia mai posizione fisica per un inserimento o una cancellazione al punto, ed è questo che rende O(1) l'aggiornamento dell'indice durante la digitazione. La posizione *logica* del k-esimo ritorno a capo è `newlines_before[k]` se k cade nella prima metà, altrimenti la posizione fisica nella seconda metà meno la larghezza del buco; `line_start(riga)` è la posizione del ritorno a capo precedente più uno. La conversione da posizione lineare a (riga, colonna) del punto, `cursor_pos`, conta invece i caratteri dall'inizio del buffer (→ BUF-2).



`gap_start` = punto

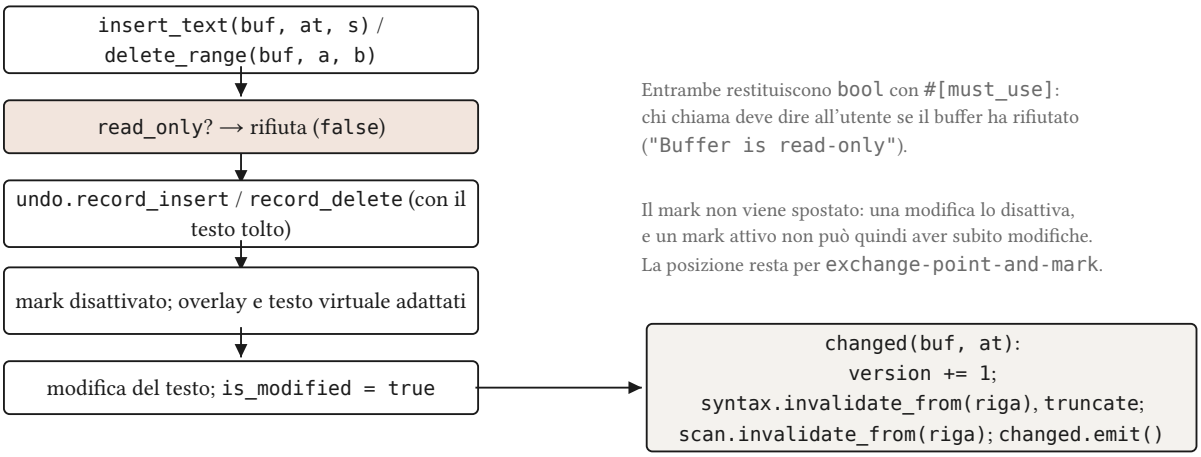
un carattere costa 4 byte (`char`); il buco cresce raddoppiando quando si esaurisce

`newlines_before: Vec<usize>`

`newlines_after: VecDeque<usize>` (posizioni fisiche)

8.3 Le due porte

Ogni primitiva che cambia il testo passa da `edits::insert_text` o da `edits::delete_range`. È la proprietà su cui si reggono undo, overlay, cache e sola lettura: una nuova primitiva di modifica li ottiene tutti per il solo fatto di non poter evitare le porte.



Entrambe restituiscono `bool` con `#[must_use]`: chi chiama deve dire all'utente se il buffer ha rifiutato (`"Buffer is read-only"`).

Il mark non viene spostato: una modifica lo disattiva, e un mark attivo non può quindi aver subito modifiche. La posizione resta per `exchange-point-and-mark`.

da `primitives/edits.rs` — estratto semplificato

```rust
pub(crate) fn insert_text<B: BufferTrait>(
    buf: &mut Buffer<B>, at: usize, content: &str,
) -> bool {
    if buf.read_only { return false; }
    if content.is_empty() { return true; }
    let at = at.min(buf.text.len());
    let point = buf.text.cursor_pos_1d();
    let inserted = content.chars().count();
    buf.undo.record_insert(at, inserted, point);          // 1. la storia
    buf.mark = deactivated(buf.mark);                     // 2. la regione
    buf.overlays.adjust_for_insert(at, inserted);         // 3. ciò che è ancorato al testo
    buf.virtual_text.adjust_for_insert(at, inserted);
    undo::apply_insert(&mut buf.text, at, content);       // 4. il testo
    buf.is_modified = true;
    changed(buf, at);                                     // 5. versione e cache
    true
}
fn changed<B: BufferTrait>(buf: &mut Buffer<B>, at: usize) {
    buf.version = buf.version.wrapping_add(1);
    let line = buf.text.cursor_1d_to_2d(at.min(buf.text.len())).0;
    buf.syntax.invalidate_from(buf.version, line);
    buf.syntax.truncate(buf.text.line_count());
    buf.scan.invalidate_from(buf.version, line);
    buf.changed.emit();                                   // sveglia chi aspetta
}
```

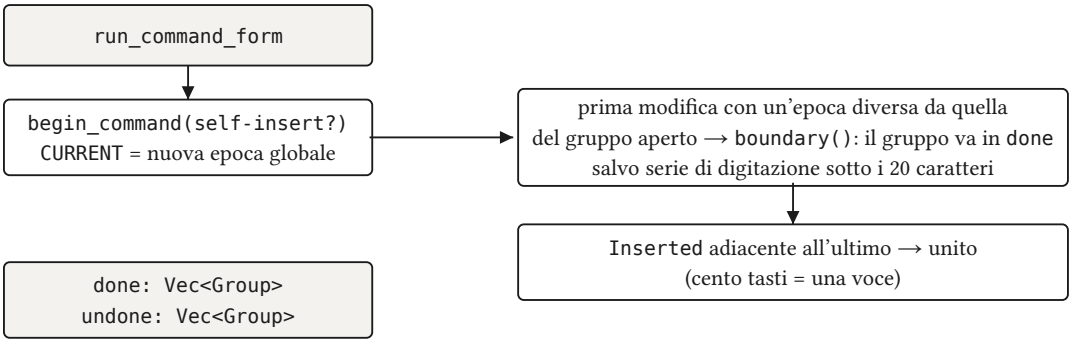
`delete_range` ha la stessa forma. Ordina e limita gli estremi, copia il testo che sta per togliere (tutto il buffer con `to_string`, una porzione con `slice`), lo consegna a `record_delete` — è l'unico modo in cui l'undo potrà rimetterlo — e poi adatta overlay e testo virtuale, cancella e chiama `changed` dalla posizione di partenza. L'ordine conta: la storia registra lo stato *prima* della modifica, gli overlay vengono spostati con gli stessi numeri che descrivono la modifica, e la versione cambia per ultima, così nessun lettore può vedere la versione nuova insieme al testo vecchio (tutto avviene sotto lo stesso lock di scrittura del buffer).

Tre percorsi oggi scavalcano le porte:

- `adopt_text` (lettura di un file, `revert-buffer`, ricarica dal watcher) sostituisce tutto il testo e fa a mano ciò che le porte farebbero: incrementa la versione, invalida entrambe le cache dalla riga 0, svuota overlay, testo virtuale, mark e storia dell'undo (conservandone il limite), riporta il punto alla stessa riga e colonna;
- `set_minibuffer_content`, usata dallo scorrimento dei completamenti, scrive direttamente nel testo del minibuffer e poi chiama `changed` (→ MIN-3)
- `undo` e `redo`, che applicano l'inverso di un gruppo direttamente sul testo con `UndoHistory::undo(text)` e poi chiamano `changed`: versione e cache sono a posto, ma `read_only` non viene controllato e overlay e testo virtuale non si spostano (→ BUF-1).

8.4 Undo

La storia registra *modifiche*, non fotografie: un inserimento è `Inserted { at, len }`, una cancellazione `Deleted { at, text }`. Un gruppo è l'insieme delle modifiche di un comando, con la posizione del punto prima della prima modifica. Disfare applica l'inverso del gruppo e produce il gruppo che lo rifarebbe; rifare è la stessa operazione sull'altra pila.



una nuova modifica svuota undone: nessun albero dei rami

Le due funzioni che decidono i gruppi sono queste:

da `buffer/undo.rs` — estratto semplificato

```
pub(crate) fn begin_command(amalgamating_kind: bool) {      // chiamata da run_command_form
    if PINNED.get() > 0 { return; }      // dentro una macro: un gruppo solo
    CURRENT.with(|current| {
        let previous = current.get();
        current.set(Command {
            epoch: NEXT_EPOCH.fetch_add(1, Ordering::Relaxed),
            amalgamating_kind,
            joins_previous: amalgamating_kind && previous.amalgamating_kind,
        });
    });
}

fn push(&mut self, change: Change, point: usize) {            // ogni modifica passa da qui
    self.undone.clear();                                       // niente albero dei rami
    let command = current_command();
    if self.open.is_some() && command.epoch != self.open_epoch
        && !(command.joins_previous && self.open_insert_len() < AMALGAMATION_LIMIT)
    {
        self.boundary();                // il gruppo aperto va in `done`
    }
    self.open_epoch = command.epoch;
    self.bytes += change.weight();                             // byte di testo cancellato
    let group = self.open.get_or_insert_with(|| Group { changes: Vec::new(), point });
    if let Change::Inserted { at, len } = change
        && let Some(Change::Inserted { at: prev_at, len: prev_len }) =
            group.changes.last_mut()
        && *prev_at + *prev_len == at
    {
        *prev_len += len;               // inserimento contiguo: si allunga
        return;
    }
    group.changes.push(change);
}
```

Disfare un gruppo vuol dire applicare le sue modifiche al contrario, dall'ultima alla prima, e costruire intanto il gruppo che rifarebbe ciò che si è appena disfatto:

da `buffer/undo.rs` — estratto semplificato

```rust
fn apply_inverse<B: BufferTrait>(&mut self, text: &mut B, group: Group) -> Group {
    let mut inverse = Vec::with_capacity(group.changes.len());
    for change in group.changes.iter().rev() {
        match change {
            Change::Inserted { at, len } => {
                // si toglie, ricordando che cosa
                let removed = text.slice(*at, at + len);
                apply_delete(text, *at, at + len);
                inverse.push(Change::Deleted { at: *at, text: removed });
            }
            Change::Deleted { at, text: content } => {
                apply_insert(text, *at, content);              // si rimette il testo salvato
                inverse.push(Change::Inserted { at: *at, len: content.chars().count() });
            }
        }
    }
    Group { changes: inverse, point: /* posizione della prima modifica */ }
}
```

`undo` chiude il gruppo aperto, toglie l'ultimo da `done`, lo applica al contrario e mette l'inverso in `undone`; `redo` fa l'operazione simmetrica. Restituiscono la posizione in cui riportare il punto.

- *Il confine è pigro.* Chiamare `boundary` a ogni tasto costava un lock di scrittura sul buffer anche per i tasti che non modificano nulla (circa 120 dei 890 ns di un tasto). Ora un comando timbra un'epoca in una variabile del thread (`CURRENT`), e il gruppo aperto si chiude alla prima modifica che arriva con un'epoca diversa. Le epoche vengono da un contatore globale, così due thread non possono confondere le loro.
- *Digitazione.* `joins_previous` è vero se il comando e il precedente sono entrambi `self-insert`; la serie continua nello stesso gruppo finché il gruppo non ha inserito `AMALGAMATION_LIMIT` (20) caratteri.
- *Macro.* `OneGroup` (RAII, annidabile) blocca il timbro: tutti i comandi di una riproduzione finiscono in un gruppo solo.
- *Fuori dai comandi.* Una modifica fatta da Lisp senza un tasto premuto usa l'epoca rimasta nel thread, quindi si unisce al gruppo dell'ultimo comando; `undo-boundary` chiude il gruppo a mano.
- *Limite.* `DEFAULT_UNDO_LIMIT` è 1 MiB di *testo cancellato* (solo le cancellazioni portano testo); oltre, i gruppi più vecchi vengono scartati. `set-undo-limit` lo cambia per buffer. Il gruppo più recente non viene mai scartato, anche se da solo supera il limite.

8.5 Mark, regione, overlay e testo virtuale

Il mark è una posizione con un flag `active`. La regione esiste solo se il mark è attivo (`region_bounds` la ordina e la limita alla lunghezza del testo), e ogni modifica la disattiva, come in Emacs. Il flag `rectangle` dice che la regione si disegna come un rettangolo; il rettangolo tra due posizioni lo calcola `Rectangle::between_offsets`, in `text/rectangle.rs` (Sezione 9). Gli *overlay* sono intervalli con una faccia e una categoria (le corrispondenze di una ricerca, una diagnostica, l'enfasi di una pagina di manuale); sono, con il testo virtuale, le sole posizioni conservate che seguono le modifiche: le porte li adattano con `adjust_for_insert` e `adjust_for_delete`. Nessuna delle due estremità cresce, e un overlay che si riduce a lunghezza zero viene cancellato. Il *testo virtuale* è ancorato a un punto, non a un intervallo, ed è per questo una tabella a parte: il layout delle righe lo inserisce nel testo mostrato (`ui/layout.rs`), e un clic su di esso viene riportato alla posizione vera del buffer.

8.6 Le cache del testo

Cache	Che cosa conserva	Chi la riempie
`SyntaxCache`	Gli intervalli colorati di ogni riga e lo stato del lessico (la pila delle regioni aperte) all'inizio di ogni riga, per versione, modo ed edizione delle grammatiche.	L'evidenziatore, `LINES_PER_TURN` (500) righe per passo, svegliato da `buffer-changed` e `syntax-changed`.
`ScanCache`	Un punto di ripresa ogni 64 righe per la scansione delle espressioni bilanciate, per versione, modo ed edizione della tabella sintattica.	Lo scanner preventivo, 500 righe per passo, svegliato dagli stessi argomenti. `forward-sexp`, l'indentazione e `syntax-ppss` la leggono e partono dal punto di ripresa più vicino invece che dall'inizio.

Entrambe sono invalidate dalla riga modificata in giù, dalla stessa chiamata (`changed`); la colorazione vecchia resta disegnata finché la nuova non la sostituisce. Una cache che non corrisponde a versione, modo ed edizione correnti non viene usata: il caso peggiore è una scansione dall'inizio, la stessa che si farebbe senza cache.

8.7 File su disco

`find-file` legge il file, imposta `file_path` e il timbro (`FileStamp`: istante di modifica e lunghezza, un solo `stat`), e sceglie il modo secondo le espressioni regolari di `add-auto-mode`. La lettura usa `read_to_string`, che rifiuta un file non UTF-8; il motivo va nel log (→ FIL-3). Salvare scrive l'intero testo con `std::fs::write` (sul posto, senza file temporaneo (→ FIL-2)) e aggiorna il timbro, altrimenti ogni salvataggio sembrerebbe la scrittura di qualcun altro. `save-buffer` scrive l'esito, riuscito o fallito, nel log e non nell'area messaggi (→ FIL-1) `write-file` lo scrive anche nell'area messaggi.

Il *watcher* (`FileWatcher`, ogni `watch-file-interval` secondi, 3 per default, al massimo 8 file per turno; spento da `(setq watch-files nil)`) confronta i timbri: un buffer pulito il cui file è cambiato viene ricaricato con `adopt_text`; uno modificato, o il cui file è sparito, viene marcato `stale`, e il salvataggio successivo chiede conferma. L'*autosalvataggio* (`AutoSaver`, ogni `auto-save-interval`, 30 secondi per default, al massimo 4 buffer per turno; spento da `(setq auto-save nil)`) scrive i buffer modificati la cui versione è cambiata dall'ultima copia in `#nome#`, nella directory del file o in `auto-save-directory`; `recover-file` riporta la copia nel buffer. La versione salvata viene registrata anche quando la scrittura fallisce (→ FIL-5).

8.8 Le decisioni

1. **La sola lettura sta sul buffer e si controlla alle porte.** Un buffer che *presenta* qualcosa — un elenco di directory, l'elenco delle facce, un backtrace — è la vista di uno stato che sta altrove: scriverci non cambia quello stato, fa solo dire allo schermo una cosa falsa. I tasti da soli non bastano a proteggerlo: ogni tasto stampabile è legato a `self-insert`, e `M-x kill-line` o una riga di Lisp passerebbero comunque. Dalle porte invece passa ogni modifica, tranne `undo` e `redo` (→ BUF-1). Il punto si muove liberamente: leggere è lo scopo di un buffer in sola lettura.
2. **I dati appesi vivono quanto il buffer.** Ciò che una funzionalità ricorda *di un buffer* — i risultati di una ricerca mostrati lì, una tabella, lo stato di una vista — sta in `data`, sotto un nome, come `Box<dyn Any>`. Uccidere il buffer distrugge la mappa, e ogni valore con il suo `Drop`; un `downcast` del tipo sbagliato risponde `None`. Due funzionalità appendono i loro dati allo stesso buffer senza sapere l'una dell'altra. Scartati: una tabella globale per nome di buffer, sbagliata dopo un cambio di nome e ancora lì dopo la chiusura; il testo del buffer, che va bene solo per ciò che si mostrerebbe comunque; un puntatore grezzo, che non ha distruttore e che con un cast sbagliato è un comportamento indefinito invece di un rifiuto.

3. **`adopt_text` sostituisce il testo in un metodo solo.** Sei istruzioni sparse nei chiamanti sono sei occasioni di dimenticarne una, e quelle facili da dimenticare non si vedono subito: senza la versione l'evidenziatore salverebbe i colori del testo vecchio sul testo nuovo, senza svuotare gli overlay una diagnostica resterebbe su una riga che si è spostata. Butta via undo, mark, overlay e testo virtuale, che descrivono posizioni di un testo che non c'è più: disfare verso uno stato che il file non ha mai avuto è peggio che non poter disfare. Tiene il modo, il limite dell'undo, e la riga e la colonna del punto, perché tornare circa dove si era è lo scopo di ricaricare un file che si sta leggendo.
4. **Il flag del rettangolo sta sul mark.** È una proprietà della selezione, non del buffer: ogni modifica disattiva il mark, e così chiude anche il modo rettangolo, senza una seconda cosa da ricordare di spegnere. Cambia solo ciò che si disegna: i comandi dei rettangoli lavorano da punto e mark in ogni caso.
5. **Gli overlay seguono le modifiche, alle porte.** Le regole di un modo sono espressioni regolari sul testo: dicono «una parola di questa lista è una parola chiave», non «*questo* tratto, per qualcosa che è successo». Per dire la seconda cosa ci sono gli overlay, che quindi devono seguire il testo: le porte chiamano `adjust_for_insert` e `adjust_for_delete`, come per l'undo e la versione. Una posizione che si sposta senza i suoi overlay è un'evidenziazione sulle parole sbagliate, minuti dopo, senza niente che dica perché.
6. **La tabella degli overlay è ordinata per inizio.** Una modifica sposta gli overlay che stanno dopo di lei: con l'ordine sono una ricerca binaria e una coda, invece di tutta la tabella. Le due regole di aggiustamento sono monotone, quindi l'ordine sopravvive e non si ricostruisce mai. Il caso più comune, una modifica dopo l'ultimo overlay, costa un confronto con `furthest_end`.
7. **Nessuna estremità di un overlay cresce.** `start` si sposta quando l'inserimento è prima o proprio lì, `end` solo quando è strettamente prima: un testo inserito all'inizio o alla fine resta fuori, uno inserito dentro entra a farne parte. Una corrispondenza evidenziata resta esattamente il testo che corrispondeva, qualunque cosa si scriva accanto.
8. **Il testo virtuale è un punto, in una tabella sua.** Non è un overlay: un overlay colora caratteri che ci sono, e uno senza larghezza viene cancellato. Non è nemmeno testo inserito davvero: il buffer risulterebbe modificato, l'undo si riempirebbe di cose mai scritte e il salvataggio le scriverebbe nel file. È una posizione che si sposta con il testo e non sparisce mai. La tabella non sa a che cosa serve: suggerimenti, anteprime e il resto sono categorie scelte da chi li produce.
9. **Due cache, perché due lessici.** La grammatica decide che aspetto ha il testo, e il suo stato è la pila delle regioni aperte; la tabella sintattica decide che cosa significa — dove sono stringhe, commenti e liste — e il suo stato è un `Resume`. `forward-sexp`, l'indentazione, `syntax-ppss` ed electric-pair leggono la seconda, e nessuno di loro si può servire dalla prima (Sezione 10).
10. **La colorazione tiene uno stato per riga, la scansione un punto ogni 64 righe.** Quasi ogni riga di quasi ogni file comincia nello stesso stato vuoto, e gli stati della colorazione, internati, costano poco. Uno stato della scansione contiene invece le *posizioni* delle liste aperte, così due righe alla stessa profondità hanno stati diversi e internarli non risparmia nulla. Un punto di ripresa ogni 64 righe costa un sessantaquattresimo della memoria, e una risposta al più 64 righe di scansione.
11. **Le cache stanno sul buffer.** Ciò che le invalida è una modifica, e le modifiche passano dalle porte; ciò che descrivono è un testo, a cui non devono sopravvivere. Una tabella a parte nell'editor andrebbe ripulita a ogni chiusura di un buffer, e sarebbe sbagliata, invece che assente, il giorno in cui non lo fosse.
12. **Chi legge la cache della scansione non la scrive.** La riempie solo lo scanner preventivo, e i movimenti la leggono con il lock di lettura che hanno già. Riempirla leggendo vorrebbe dire prendere il lock di scrittura del buffer sul percorso di un tasto, nel mezzo della valutazione di Lisp, per far funzionare una cache. Il prezzo: un file appena aperto si scandisce dall'inizio finché lo scanner preventivo non l'ha percorso.
13. **Una modifica tiene ciò che sta sopra.** Una modifica alla riga L non cambia il testo prima di L: i punti di ripresa e gli stati che entrano nelle righe fino a L restano esatti, e si scarta solo da L in giù, senza dover contare le righe aggiunte o tolte.

14. **Il colore vecchio resta disegnato.** Dopo una modifica nulla di ciò che sta sotto è più affidabile, ma gli intervalli restano e si disegnano finché l'evidenziatore non li sostituisce. Cancellarli farebbe passare il resto del file al testo semplice a ogni tasto, che è molto peggio di qualche millisecondo di colore un po' sbagliato.
15. **La versione non basta.** Un cambio di modo, o una regola aggiunta alla grammatica di un modo, non tocca il testo. Per questo le due cache ricordano anche il modo e l'edizione della grammatica o della tabella con cui sono state calcolate, e una cache che ne descrive un'altra ricomincia dall'inizio. Così resta fuori dal lessico anche lo stato di un'altra grammatica, che nomina le regioni con il loro posto in quella.
16. **La colorazione non si risincronizza.** Dopo una modifica si ricalcola fino alla fine del file, invece di fermarsi dove lo stato calcolato torna uguale a quello di prima. Fermarsi lì costerebbe le righe fino alla convergenza invece del resto del file, ma richiede di tenere gli stati della generazione precedente e di contare di quante righe la modifica ha spostato il resto: senza quel conto si confronterebbero le righe sbagliate. Il lavoro a passi limitati tiene comunque l'editor reattivo, e la risincronizzazione riguarda quanto lavoro c'è in tutto: si aggiunge se le misure la chiedono.
17. **Undo scrive sotto le porte.** Rifare la storia non deve registrarsi come storia nuova, quindi `undo` e `redo` applicano le modifiche con `apply_insert` e `apply_delete`, sotto lo strato che registra: non c'è un flag da ricordare. Il chiamante fa poi ciò che le porte farebbero, con `changed` da `Applied::from` (→ BUF-1).
18. **Il testo da rifare si cattura disfacendo.** Disfare un inserimento toglie il testo e lo ricorda nel gruppo inverso, invece di conservarlo prima per un `redo` che può non venire mai.
19. **Una macro è un gruppo solo.** Rieseguire una macro passa ogni comando per la via normale, e lascerebbe cinquanta gruppi dietro una macro di cinquanta comandi: chi la esegue non vuole disfarne un cinquantesimo. `OneGroup` tiene fermo il timbro per tutta la riproduzione, e la regola che c'è già — un gruppo si chiude alla prima modifica sotto un altro timbro — fa il resto.
20. **Il file si riconosce da un timbro, non da un hash.** La domanda «il file è cambiato?» si fa per ogni buffer aperto, ogni pochi secondi. Un hash risponde esattamente ma legge tutto il file; un timbro — istante di modifica e lunghezza — risponde con un solo `stat`. Sfugge una scrittura che cada nella granularità del filesystem e lasci identica la lunghezza: il buffer resta vecchio fino al cambiamento successivo, senza danni.
21. **Il timbro si confronta per differenza, non per «più recente».** Gli istanti tornano indietro nell'uso normale: `git checkout` rimette istanti vecchi, `touch -d` scrive quello che gli si dice, un filesystem di rete non è d'accordo con l'orologio locale. «Diverso da ciò che ho registrato» è una domanda a cui il filesystem risponde; «più recente del mio» è una domanda su due orologi. Il timbro si registra anche a ogni scrittura dell'editor, altrimenti ogni salvataggio sembrerebbe la scrittura di qualcun altro.
22. **`stale` è un flag, non una domanda.** Il momento in cui fermare la sovrascrittura del lavoro di un altro è il salvataggio, non l'istante in cui il cambiamento si nota: il watcher marca il buffer, e il salvataggio successivo chiede.
23. **L'autosalvataggio confronta versioni.** «Modificato» resta vero dal primo tasto fino al salvataggio, quindi un lavoro che scrivesse a ogni turno ogni buffer modificato riscriverebbe per sempre un buffer fermo. La versione si muove solo quando si muove il testo: `auto_saved_at` la ricorda, e la domanda «c'è qualcosa da scrivere?» diventa un confronto tra due numeri. Un editor lasciato aperto di notte non costa nulla.

9 Il testo: ricerca, sostituzione, kill ring, rettangoli e liste di risultati

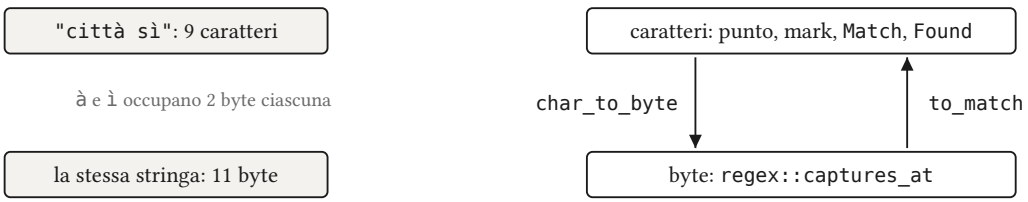
La directory `core/src/text/` raccoglie ciò che si può calcolare su un testo senza sapere che cosa sia un editor. Il criterio di ammissione è scritto in `text/mod.rs`: un file sta qui se lo potrebbe usare un programma che non è un editor. In pratica nessun file della directory importa `editor` o `lisp`, e nessuno riceve un `Buffer`: ricevono un `BufferTrait` (o una `&str`), cioè qualcosa che sa dare caratteri per posizione, e nient'altro. Niente punto, mark, undo, lock o finestre.

File	Che cosa contiene	Chi lo guida
`search.rs`	`Pattern` (che cosa cercare) con `search_forward`, `search_backward` e `scan`; le espressioni compilate di `string-match` (`compiled`).	`feature/isearch.rs`, `editor/replace.rs`, `primitives/scan.rs`, `primitives/match_data.rs`
`isearch.rs`	La sessione di ricerca incrementale: `Isearch` e `Direction`.	`feature/isearch.rs`, `primitives/isearch.rs`
`replace.rs`	La sessione di sostituzione, `Replace`, con `Casing` ed `expand_replacement`.	`editor/replace.rs`
`kill_ring.rs`	`KillRing`: un anello di stringhe con la regola di rotazione e di accodamento.	`managers/kill_yank.rs` (`KillYank`)
`rectangle.rs`	`Rectangle` e `Span`: l'aritmetica di due angoli e delle righe tra loro.	`primitives/rectangle.rs`, `ui/windows.rs`
`results.rs`	`Entry` e `Results`: una lista di posti e un cursore che la percorre.	`primitives/results.rs`, `editor/results.rs`

Le sessioni (`Isearch`, `Replace`) tengono lo *stato* di una ricerca o di una sostituzione tra un tasto e l'altro. Chi le guida le conserva in `Runtime` e chiama un metodo per ogni risposta dell'utente; la sessione conosce il buffer solo per nome (Sezione 9.8).

9.1 Posizioni: sempre caratteri

Tutte le posizioni del modulo sono offset in *caratteri*, come il punto, il mark e la storia dell'undo. Il motore delle espressioni regolari (`regex`) lavora invece in *byte* su una `&str`. La conversione avviene in un posto solo: `char_to_byte` all'ingresso e `to_match` (che usa `byte_to_char`) all'uscita. Un errore di conversione non si vede su testo ASCII e rompe il primo carattere accentato, per questo non è ripetuta nei chiamanti.



9.2 `Pattern`: che cosa cercare

da `text/search.rs` — estratto semplificato

```
pub enum Pattern {
    Literal { chars: Vec<char>, fold: bool },   // confronto carattere per carattere
    Regex(Regex),                               // compilata una volta sola
}
pub fn new(source: &str, regexp: bool, fold: bool) -> Result<Self, String> {
    if source.is_empty() {
        return Err("Searching for the empty string would never finish".into());
    }
    if regexp {
        RegexBuilder::new(source).case_insensitive(fold).multi_line(true)
            .build().map(Pattern::Regex).map_err(|why| format!("Invalid regexp: {why}"))
    } else {
        Ok(Pattern::Literal { chars: source.chars().collect(), fold })
    }
}
```

Tre decisioni stanno in questa funzione:

- *Il modello vuoto è rifiutato.* Troverebbe una corrispondenza in ogni posizione: una ricerca non si muoverebbe mai e una sostituzione non finirebbe. Rifiutandolo qui nessun chiamante deve ricordarsi il controllo. Un'espressione che *può* corrispondere al vuoto (`x*`) resta invece ammessa, ed è compito di chi cicla scavalcare le corrispondenze vuote (`Match::is_empty`) (→ RIC-1).
- *`multi_line(true)`*: `^` e `$` indicano inizio e fine di *riga*, come si aspetta chi scrive un'espressione in un editor. Senza, `^;;` troverebbe al massimo una corrispondenza in tutto il buffer.
- *`fold`* arriva dal chiamante, che lo legge dalla variabile Lisp `case-fold-search` (non legata = vero). Per i letterali il confronto ignora le maiuscole con `chars_equal`, che confronta le forme minuscole dei due caratteri.

9.2.1 *`search_forward` e `search_backward`*

`search_forward(text, from, limit)` restituisce la prima corrispondenza che *comincia* in `from` o dopo e *finisce* entro `limit`. Il limite è ciò che confina una sostituzione alla regione: il chiamante passa la fine della regione invece di quella del buffer.

- *Letterale*: prova ogni inizio da `from` a `limit - len` con `matches_at`, che legge i caratteri uno per uno con `BufferTrait::at`. Non alloca nulla: un passaggio, nessuna copia.
- *Espressione*: copia il buffer in una `String` (il motore vuole una `&str`), converte `from` in byte e chiama `captures_at`. Si usa `captures_at` e non una fetta del testo perché la ricerca deve vedere ciò che precede il punto di partenza: altrimenti ogni ripresa sembrerebbe l'inizio di una riga per `^` e per i look-behind. La corrispondenza trovata è scartata se finisce oltre il limite, anche quando ne esisterebbe una più corta che ci sta (→ RIC-4).

da `text/search.rs` — estratto semplificato

```
Pattern::Regex(_) => {
    let mut best = None;
    let mut at = floor;
    while let Some(found) = self.search_forward(text, at, to) {
        at = if found.is_empty() { found.end + 1 } else { found.end };
        best = Some(found);
    }
    best
}
```

`search_backward(text, to, floor)` cerca l'ultima corrispondenza che finisce entro `to` e comincia da `floor` in poi. Per i letterali prova gli inizi all'indietro. Per le espressioni il motore sa cercare solo in avanti,

quindi cammina tutte le corrispondenze da `floor` e tiene l'ultima: ogni passo è una `search_forward`, che ricopia il buffer, e il costo totale cresce con il quadrato del testo (→ RIC-2).

9.2.2 `scan`: tutte le corrispondenze in un passaggio

Occur, grep e la lettura dell'output di un compilatore vogliono *tutte* le corrispondenze, con riga, colonna e testo della riga. Chiamare `search_forward` in un ciclo costerebbe una copia del buffer per corrispondenza. `Pattern::scan(haystack, limit)` lavora su una `&str` già pronta e fa un passaggio solo, grazie a un `Cursor` che avanza sempre in avanti e conta caratteri e righe man mano:

da `text/search.rs` — estratto semplificato

```
struct Cursor<'a> {
    haystack: &'a str, byte: usize, chars: usize, line: usize, line_start: usize,
}
fn advance_to(&mut self, byte: usize) {          // byte >= self.byte, sempre
    for (at, c) in self.haystack[self.byte..byte].char_indices() {
        self.chars += 1;
        if c == '\n' { self.line += 1; self.line_start = self.byte + at + c.len_utf8(); }
    }
    self.byte = byte;
}
```

Per ogni corrispondenza (in byte, da `next_match`) `scan` porta il cursore al suo inizio, legge riga e colonna, prende il testo della riga (memorizzato, così venti corrispondenze sulla stessa riga non la ricostruiscono venti volte) e converte inizio e fine in caratteri. Poi riprende dalla fine della corrispondenza, o un carattere dopo se era vuota. Il risultato è una `Scan`: il vettore dei `Found` e `truncated`, vero se si è fermata per `limit`. La colonna invece è ricalcolata contando dall'inizio della riga a ogni corrispondenza (→ RIC-4).

Campo di `Found`	Significato
`start`, `end`	Offset in caratteri nell'intero testo.
`line`	Riga di inizio, contata da 1 (la numerazione di `goto-line`).
`column`	Colonna in caratteri, contata da 0 (come `current-column`).
`line_text`	La riga intera senza `\n`: mostrarla non richiede di rileggere il testo.
`groups`	Ciò che ha catturato ogni gruppo, 0 = l'intera corrispondenza; vuoto per un letterale.

`in_line()` ed `end_in_line()` danno la posizione della corrispondenza dentro `line_text`, che è ciò che serve a una vista per evidenziarla.

9.2.3 Le espressioni di `string-match`

`string-match` e `string-match-p` non costruiscono un `Pattern`: chiedono a `compiled(pattern)` l'espressione già compilata. Ogni thread tiene la sua tabella, dalla sorgente all'espressione (`Arc<Regex>`), e la svuota tutta quando arriva a `CACHE_LIMIT`, 512 voci.

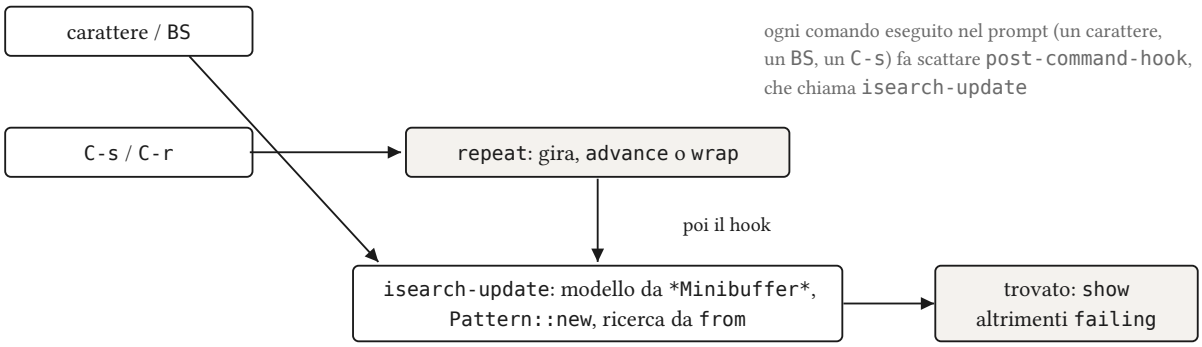
9.3 La sessione di ricerca incrementale: `Isearch`

La parte di interfaccia è descritta nel Sezione 7: `C-s` apre il prompt in `isearch-mode` e il `post-command-hook` di quel modo chiama `isearch-update` dopo ogni comando. Qui interessa lo stato che passa da un tasto all'altro.

da `text/isearch.rs` — estratto semplificato

```
pub struct Isearch {
    pub buffer: String,          // il buffer cercato (il corrente è il minibuffer)
    pub origin: usize,           // dove era il punto: ripristinato da C-g
    pub direction: Direction,    // Forward | Backward
    pub regexp: bool,
    pub from: usize,             // da dove parte la prossima scansione
    pub found: Option<Match>,    // la corrispondenza mostrata
    pub failing: bool,           // l'ultima scansione non ha trovato nulla
    pub wrapped: bool,           // la ricerca ha fatto il giro
}
```

Il campo chiave è `from`. Scrivere un carattere *non* lo sposta: allungare il modello rifà la ricerca dallo stesso punto, quindi la corrispondenza cresce dove l'utente sta guardando. Solo la ripetizione lo sposta.



Le regole dei metodi, tutte prese dal comportamento di Emacs:

- `advance()` sposta `from` a `found.start + 1` andando avanti (a `found.end - 1` andando indietro), non alla fine della corrispondenza: così anche le corrispondenze sovrapposte sono raggiungibili. Ripetere la ricerca di `aa` in `aaa` trova anche quella che comincia in 1.
- `wrap(len)` riparte dall'altro capo del buffer e imposta `wrapped`. Viene chiamato solo quando si ripete una ricerca che sta *già* fallendo: il primo `C-s` a vuoto mostra «Failing I-search», il secondo è la decisione di fare il giro.
- Cambiare direzione non avanza: `from` diventa l'inizio (o la fine) della corrispondenza corrente, perché `C-s C-r` deve ripercorrere le stesse corrispondenze senza saltarne una.
- `report(pattern)` compone il testo per l'area dei messaggi: `[Failing ][Wrapped ]` seguito da `I-search`, `I-search backward`, `Regexp I-search` o `Regexp I-search backward`, poi `: ` e il modello.

`isearch-update` gestisce anche i casi limite. Con il prompt vuoto riporta il punto a `origin` e toglie il mark. Un'espressione che non si compila (`[a-` è a metà di `[a-z]`) è trattata come una ricerca fallita, non come un errore: il carattere successivo la renderà valida. Quando trova, `show` mette il punto sull'estremità lontana della corrispondenza e il mark su quella vicina, così la corrispondenza è la regione e la disegna lo stesso codice che disegna una selezione. `isearch-exit` (Invio) disattiva il mark e lascia il punto dov'è; `isearch-abort` (`C-g`, `ESC`) riporta il punto a `origin` (→ RIC-6).

9.4 La sessione di sostituzione: `Replace`

`replace-string`, `replace-regexp` e `query-replace` creano una `Replace` con `begin_replace` (`editor/replace.rs`). Il campo d'azione è la regione se il mark è attivo, altrimenti dal punto alla fine del buffer: `origin` e `limit` sono i due estremi. Poi `seek` cerca la prima corrispondenza e un overlay della categoria `replace-match` la evidenzia.

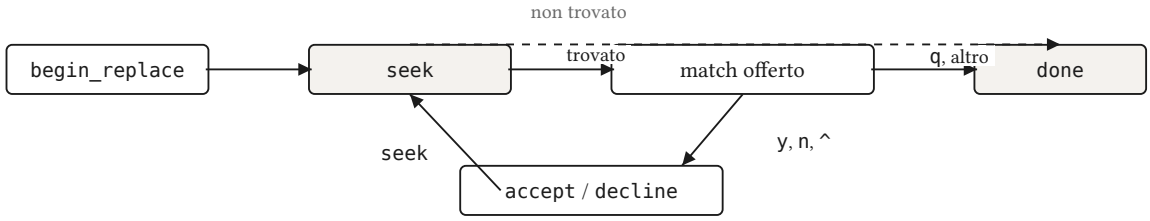
da `text/replace.rs` — estratto semplificato

```
pub struct Replace {
    pub buffer: String, pub pattern: Pattern,
    pub replacement: String,     // il modello, con \1 \& ancora dentro
    pub origin: usize, pub from: usize,
    pub limit: usize,            // spostato da ogni sostituzione
    pub found: Option<Match>, pub replaced: usize,
    pub history: Vec<Step>,      // da dove partiva ogni sostituzione accettata
    pub preserve_case: bool, pub done: bool,
}
```

Chi la guida chiama un verbo per ogni risposta. La vista predefinita (`default_view` in `primitives/replace.rs`) installa una keymap transitoria con cinque tasti: `y` (`replace-this`), `n` (`replace-skip`), `!` (`replace-rest`), `^` (`replace-back`, che usa `unaccept` e l'undo del buffer) e `q` (`replace-done`). La keymap è di tipo `OnUnbound::Release`: qualunque altro tasto toglie la domanda e poi fa ciò che fa di solito, ma la sessione resta aperta e la corrispondenza evidenziata (→ RIC-8). Un'interfaccia grafica userebbe pulsanti; nessuna delle due si esprime come un ciclo che legge una risposta.

Verbo	Effetto sulla sessione
`seek(text)`	`found = search_forward(text, from, limit)`; se non trova nulla, `done = true`.
`accept(new_len)`	La corrispondenza è stata sostituita da `new_len` caratteri. Registra `Step { from: start, limit }` in `history`; sposta `limit` della differenza di lunghezza; riprende da `start + new_len` (o `start + 1` se corrispondenza e sostituzione erano entrambe vuote) (→ RIC-1). Incrementa `replaced`.
`decline()`	Salta la corrispondenza: riprende dalla sua fine, o un carattere dopo se era vuota.
`unaccept()`	Riprende l'ultimo `Step`: `from` e `limit` tornano com'erano, `replaced` cala. Il testo lo rimette a posto il chiamante con l'undo del buffer, perché la sessione non sa nulla dei buffer.

Il limite si sposta perché una sostituzione più lunga o più corta sposta tutto ciò che segue, compresa la fine della regione. Senza questo aggiornamento una sostituzione confinata alla regione ne uscirebbe, o si fermerebbe prima. La `history` serve a tornare indietro: dopo una sostituzione di lunghezza diversa il testo da solo non dice più dove era cominciata.



Il testo da inserire lo calcola, per ogni corrispondenza, `expand_replacement`, che riceve il modello di sostituzione, la corrispondenza, il testo effettivamente trovato e `preserve_case`:

- `\1` … `\9` sono i gruppi catturati; un gruppo che non ha partecipato, o qualsiasi gruppo con un modello letterale (che non ne ha), diventa la stringa vuota;
- `\0` e `\&` sono l'intera corrispondenza, presa dal testo (`matched`) e non dal gruppo 0, così funzionano anche con i letterali;
- `\\` è una barra rovesciata; una barra seguita da qualunque altra cosa resta com'è, così un percorso Windows si può scrivere senza raddoppiare le barre.

Con `preserve_case` (il valore di `case-replace` quando la ricerca ignora le maiuscole) il risultato viene poi riscritto secondo `Casing::of(matched)`:

Corrispondenza	`Casing`	`foo` diventa
`BAR`	`Upper`: tutte le lettere maiuscole (almeno due)	`FOO`
`Bar`, `B`	`Capitalised`: la prima maiuscola, le altre minuscole	`Foo`
`bar`, `bAr`, `fooBar`, `42`	`AsWritten`: minuscole, miste o nessuna lettera	`foo`

Sul lato dell'editor, `replace_this` cancella la corrispondenza e inserisce l'espansione con le due porte (`delete_range`, `insert_text`), chiude un gruppo di undo (→ RIC-3), chiama `accept` e poi cerca la successiva. `replace_rest` (il `!` di `query-replace`, e tutto `replace-string`) ripete `replace_this` con un tetto di `len + 1` giri, che è l'unica cosa che ferma il caso delle corrispondenze vuote (→ RIC-1).

9.5 Il kill ring

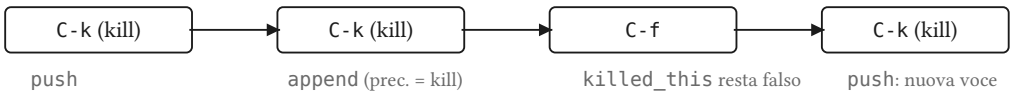
da `text/kill_ring.rs` — estratto semplificato

```
pub struct KillRing {
    entries: VecDeque<String>,  // la più recente in testa
    max: usize,                 // DEFAULT_KILL_RING_MAX = 60
    index: usize,               // da dove legge yank; azzerato da ogni kill
}
pub fn append(&mut self, text: String, direction: Direction) {
    if text.is_empty() { return; }
    match self.entries.front_mut() {
        Some(front) => {
            match direction {
                Direction::Forward => front.push_str(&text),
                Direction::Backward => front.insert_str(0, &text),
            }
            self.index = 0;
        }
        None => self.push(text),
    }
}
```

Un anello invece di un appunto singolo: uccidere del testo non distrugge quello che si stava per incollare. Le operazioni:

- `push(text)` aggiunge una voce in testa e azzera `index`; il testo vuoto è ignorato, così una kill di niente non allontana ciò che si voleva incollare; poi `trim` scarta le voci oltre `max`.
- `append(text, direction)` unisce il testo alla voce più recente: in coda per una kill in avanti, in testa per una all'indietro. Una serie di `C-k` diventa così un'unica voce che si legge nell'ordine giusto, e una serie di `M-DEL` non viene assemblata al contrario.
- `current()` è ciò che inserisce `yank`; `rotate()` avanza `index` di uno (con giro) ed è ciò che usa `yank-pop`; `nth(n)` legge senza muovere l'anello.
- `set_max` (`set-kill-ring-max`) riduce subito l'anello; se `index` finisce oltre la fine torna a 0.

La decisione tra `push` e `append` non è dell'anello ma del compartimento `KillYank` (`managers/kill_yank.rs`), che tiene due coppie di flag: `killed_last` / `killed_this` e `yanked_last` / `yanked_this`. Una kill accoda se anche il comando precedente era una kill; `roll_over`, chiamato all'inizio di ogni comando, sposta «questo» in «precedente» e azzera «questo». Allo stesso modo `yank-pop` è permesso solo se il comando precedente era uno yank, e `last_yank` dice quale tratto di testo sostituire.



tra un comando e l'altro `roll_over`: `killed_last = killed_this; killed_this = false`

Se la variabile `clipboard-sync` è vera (la imposta `clipboard.lisp`), ogni kill lascia anche la voce corrente in `pending_clipboard`. La `snapshot` successiva la prende (`take_pending_clipboard`) e il TUI la manda al terminale con una sequenza OSC 52. Prenderla invece di leggerla evita di rimandarla a ogni ridisegno.

L'ultimo rettangolo ucciso o copiato non va nell'anello: sta in un campo a parte di `KillYank`, una riga per stringa, e `yank-rectangle` lo legge da lì.

9.6 I rettangoli

Il punto e il mark sono due angoli; il rettangolo è il blocco di colonne che le righe tra loro hanno in comune. Non è la regione, che dalla metà di una riga alla metà di un'altra prende anche le righe intere in mezzo.

da `text/rectangle.rs` — estratto semplificato

```
pub struct Rectangle { pub top: usize, pub bottom: usize, pub left: usize, pub right: usize }
pub fn between(a: (usize, usize), b: (usize, usize)) -> Self {   // (riga, colonna)
    Self { top: a.0.min(b.0), bottom: a.0.max(b.0), left: a.1.min(b.1), right: a.1.max(b.1) }
}
```

`bottom` è incluso e `right` no: le righe sono un insieme di *cose*, le colonne un intervallo di *posizioni*. Un rettangolo largo zero è ammesso ed è utile: è una posizione su ogni riga, che è ciò che serve a `string-rectangle` per anteporre un prefisso a un gruppo di righe. Gli angoli possono essere dati in qualsiasi ordine.

Le colonne sono caratteri, e un tab è una colonna, come in tutto l'editor. Sullo schermo il terminale lo espande, quindi su una riga con un tab il blocco disegnato non è quello che il rettangolo prende (→ TUI-3).

`spans(text, rect)` traduce il rettangolo in un `Span` per riga, dall'alto:

da `text/rectangle.rs` — estratto semplificato

```
(rect.top..=rect.bottom.min(last_line)).map(|line| {
    let line_start = text.cursor_2d_to_1d(line, 0);
    let start = text.cursor_2d_to_1d(line, rect.left);    // limitato alla fine della riga
    let end = text.cursor_2d_to_1d(line, rect.right);
    let reached = start - line_start;
    Span { line, start, end, padding: rect.left - reached.min(rect.left) }
})
```

Tutto si regge sul fatto che `cursor_2d_to_1d` limita una colonna oltre la fine della riga alla fine della riga. Su una riga corta entrambi gli estremi cadono sulla fine e lo span è vuoto; `padding` dice quanti spazi mancano per raggiungere il bordo sinistro. Esempio con `left = 4`, `right = 7`:

Riga	Span	`padding`	Testo coperto
`0123456789`	4..7	0	`456`
`01234`	4..5	0	`4` (la riga finisce prima del bordo destro)
`01`	2..2	2	vuoto
(riga vuota)	0..0	4	vuoto

Che cosa fare delle righe corte lo decide l'operazione. Togliere testo (`kill-rectangle`, `copy-rectangle-as-kill`, `clear-rectangle`) prende ciò che c'è e non allunga la riga. Mettere testo

(`open-rectangle`, `yank-rectangle`, `string-rectangle`) prima aggiunge il padding, perché una colonna che non esiste è proprio ciò che quelle operazioni devono riempire. Gli span sono dall'alto perché è l'ordine di lettura; chi modifica li percorre dal basso, perché una modifica a una riga sposta tutti gli offset successivi. `text_of` restituisce il testo di ogni span, con una stringa vuota per le righe che non raggiungono il rettangolo, così il blocco mantiene la sua forma.

9.7 Le liste di risultati

Occur, grep e la lettura dell'output di un compilatore producono la stessa cosa: una lista di posti dove andare. `results.rs` la definisce una volta, così produrre la lista e percorrerla sono lavori separati.

Campo di `Entry`	Significato
`kind`	`"buffer"` o `"file"`: altrimenti `*scratch*` e un percorso si distinguerebbero solo indovinando.
`source`	Nome del buffer o percorso del file.
`line`, `column`	Riga da 1, colonna da 0.
`offset`	Offset in caratteri, per `goto-char`.
`text`, `start`, `end`	La riga intera e la posizione della corrispondenza al suo interno.
`at`	La riga dell'elenco su cui la voce è mostrata, da 1, quando il produttore la dice. La lista di una ricerca ha un'intestazione e poi una voce per riga, e non serve; l'uscita di un compilatore nomina i posti sulle righe dove il compilatore li ha scritti, e serve.

`Entry::new(kind, source, found)` copia i campi da un `Found` di `scan`; `rendered()` produce la forma `file:riga: testo` di ogni strumento alla grep. `Results` aggiunge il modello cercato, su che cosa (`over`), le voci, `truncated`, `done` (falso mentre una scansione di sfondo è in corso), il buffer dove è stato messo l'ultimo evidenziatore (`marked`, per toglierlo) e la voce corrente.

da `text/results.rs` — estratto semplificato

```
pub fn step(&self, step: isize) -> Option<usize> {
    if self.entries.is_empty() { return None; }
    let next = match self.current {
        None if step >= 0 => 0,                    // prima visita in avanti: la prima
        None => self.entries.len() - 1,            // prima visita indietro: l'ultima
        Some(current) => {
            let next = current as isize + step;
            if next < 0 || next as usize >= self.entries.len() { return None; }
            next as usize
        }
    };
    Some(next)
}
```

Il passo non fa il giro, di proposito: la fine della lista è un'informazione («era l'ultimo»), e ricominciare in silenzio farebbe correggere due volte il primo errore. Un insieme di risultati vive nel buffer che lo mostra, in `Buffer::data` sotto la chiave `"results"` (`RESULTS_KEY`): quando il buffer viene chiuso i risultati spariscono con lui, e rinominarlo non rompe nulla. Le voci conservano numeri di riga e offset, non marcatori che seguono il testo (→ RIC-5).

Dove una lista si mostra lo decide `editor/results.rs`. Accanto a ogni lista, nei dati del suo buffer, sta una `Presentation`: chi l'ha prodotta, da quale finestra e da quale buffer è stata chiesta, quale buffer ha preso il posto e in quale finestra si vede. `prepare_results_buffer` sceglie il buffer di un'esecuzione nuova:

con `results-reuse-buffer`, la lista più recente dello stesso produttore e dello stesso modo, fermando il lavoro che vi scriveva; altrimenti un buffer nuovo. `display_results` la mostra nella finestra con il focus, se `results-display-location` è `same-window`, o in una striscia in basso, riusando quella che mostra una lista dello stesso produttore (→ RIC-7). `select_results_target` sceglie la finestra in cui una visita apre il posto (la lista stessa con `results-visit-same-window`), e `quit_results` chiude la striscia o rimette il buffer che la lista aveva sostituito.

9.8 Le decisioni

1. **`text/` è una directory, con una regola d'ammissione.** Un file sta qui se lo potrebbe usare un programma che non è un editor. Messi alla radice del crate, accanto a `editor/` e `ui/`, questi file farebbero una lista di moduli senza un principio, e lo strato sarebbe soltanto descritto. Una directory lo rende visibile e controllabile: `layering_tests` fallisce se un file di qui nomina l'editor o l'interprete.
2. **Ricerca e sessioni stanno sotto l'editor.** La ricerca ha bisogno solo di un testo che dia i caratteri per posizione: né punto, né mark, né undo, né lock. Così si prova su un buffer qualunque, senza un editor intorno, e la ricerca incrementale, la sostituzione e occur non possono avere due idee diverse di che cosa sia una corrispondenza. Lo stesso vale per l'aritmetica di una sostituzione: da dove riparte, di quanto sposta ciò che segue, quando si ferma.
3. **Un'espressione cerca su una copia del testo.** Il motore (`regex`) lavora su una `&str`, quindi la ricerca con un'espressione copia il buffer a ogni chiamata, mentre quella letterale non copia nulla. La ricerca incrementale ricerca a ogni tasto, e su un buffer grande una ricerca con un'espressione costa una copia per tasto. È la versione semplice; quella che scala darebbe il testo a pezzi, senza copiarlo, alla ricerca a flusso di `regex_automata`.
4. **Le espressioni di `string-match` si compilano una volta.** Compilarne una costa circa sette decimi di millisecondo: niente quando un modo dichiara la sua grammatica, un blocco quando succede in un ciclo. I moduli chiamano `string-match` per ogni elemento: la sorgente dei completamenti del manuale toglie l'estensione a ogni pagina che trova, e senza la cache quattromila pagine sarebbero quattromila compilazioni, 2,7 secondi a ogni Tab sul thread che disegna. Con la cache le stesse chiamate costano venti millisecondi.
5. **La cache è per thread, e quando è piena si svuota tutta.** Per thread, perché trovare una corrispondenza non prenda un lock: Lisp gira sul thread dei comandi e su quello dell'esecutore, e una tabella condivisa li metterebbe l'uno sulla strada dell'altro nella chiamata più frequente che c'è. Svuotarla tutta invece di scartare le voci vecchie basta: le espressioni vengono dal sorgente dei moduli e sono qualche decina, e il limite serve solo a un programma che le *genera* dal suo input. Buttare tutto costa una compilazione per ognuna ancora in uso, cioè, nel caso peggiore, il comportamento senza la cache.
6. **Ricerca incrementale e sostituzione sono sessioni, non cicli.** Una ricerca incrementale è un ciclo sui tasti, e una sostituzione un ciclo sulle risposte; ma in rsedit non c'è una lettura bloccante di un tasto, perché i tasti arrivano uno per giro del ciclo degli eventi, e un pulsante di un'interfaccia grafica non è una lettura affatto. Il ciclo è quindi rovesciato, come la raccolta degli argomenti con `PendingCommand` (Sezione 6): la sessione tiene ciò che il ciclo terrebbe nelle sue variabili, e chi la guida chiama un metodo per ogni giro. La sessione non nomina buffer, finestre o lock, solo il nome del buffer e posizioni nel suo testo: si legge e si scrive senza tenere nulla, e descrive una ricerca in un buffer che non è il corrente, come è sempre mentre il prompt è aperto.
7. **La sostituzione segue le maiuscole di ciò che sostituisce.** Una ricerca che ignora le maiuscole e una sostituzione letterale trovano il posto giusto e ci scrivono la cosa sbagliata: sostituire `colour` con `color` in un documento trasforma il `Colour` a inizio frase in `color`, e ognuno va corretto a mano. È ciò che più si nota quando manca.
8. **Un anello solo per tutto l'editor.** Uccidere in un buffer e incollare in un altro è lo scopo del kill ring: l'anello sta sull'editor, in `KillYank`, e non sul buffer, così il testo passa da uno all'altro.

9. **Una riga corta resta nel rettangolo.** Una riga che finisce prima del bordo sinistro è comunque una delle righe del rettangolo, con uno span vuoto e il suo `padding`. Toglierla farebbe dipendere l'altezza del blocco da ciò che contiene.
10. **Il punto e l'evidenziazione della sostituzione stanno nell'editor, la domanda no.** Spostare il punto sulla corrispondenza serve a ogni vista, e una vista che dovesse farlo da sé potrebbe sbagliarlo; segnare la corrispondenza pure, perché una corrispondenza che non si vede è inutile in ogni vista. Ciò che resta alla vista è chiedere: nulla in `editor/replace.rs` mette una domanda sullo schermo o legge un tasto. Una vista che disegna da sé toglie l'overlay `replace-match`.

10 Modi

10.1 Il modo maggiore

Un buffer ha un solo modo maggiore, nominato in `current_mode`. Il modo è un `MajorMode` nel registro di `Modes`:

Campo	Contenuto
`keymaps`	Le associazioni del modo, consultate prima di quelle globali.
`grammar`	Regole e regioni di colorazione (`add-syntax-rule`, `add-syntax-region`).
`hooks`	Hook per nome (`post-command-hook`, `post-self-insert-hook`, `after-close-hook`, …), aggiunti con `(add-hook MODO NOME FUNZIONE)`.
`completion_functions`	Le fonti di `completion-at-point` per questo modo.
`syntax_table`	Delimitatori, stringhe, commenti, quote dei caratteri: ciò che legge lo scanner delle espressioni.

`Modes` contiene inoltre la keymap globale, i completamenti globali, la lista `auto_modes` (espressione regolare sul nome del file ⟶ modo, da `add-auto-mode`), la keymap transitoria, i tasti di ripetizione e gli hook globali, che valgono per ogni modo e si concatenano a quelli del modo.

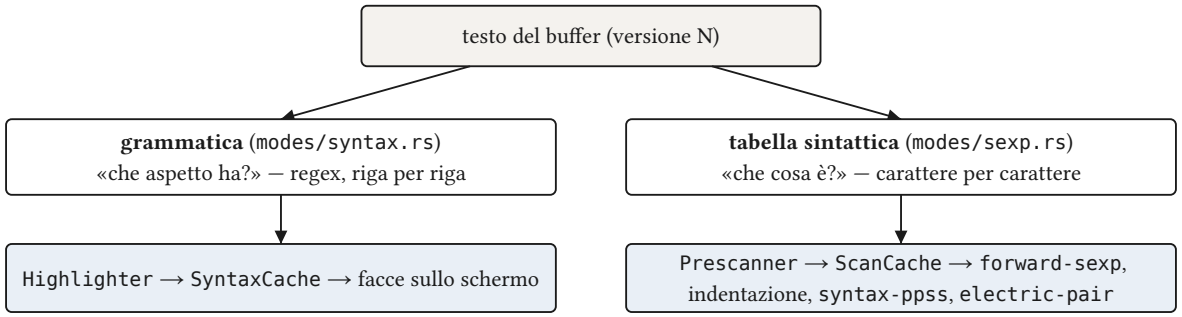
`make-mode` registra un `MajorMode` nuovo sotto il nome dato, anche se il nome esiste già: keymap, grammatica e hook definiti prima per quel nome vengono sostituiti (⟶ SIN-4).

Le proprietà dei simboli completano il modo senza toccare la struttura Rust; queste sono lette da `indent.lisp`:

```
(put 'rust-mode 'tab-width 4)
(put 'risp-mode 'indent-function 'lisp-indent-line)
(put 'rust-mode 'indent-region-style 'shift)
```

10.2 Due lessici sullo stesso testo

Il testo di un buffer viene letto in due modi diversi, per due domande diverse.



La *grammatica* colora una riga alla volta portando uno stato da una riga all'altra: lo stato è la pila delle regioni aperte (un commento a blocchi che continua, una stringa su più righe). Colorare la riga 5000 richiede solo lo stato all'inizio della riga 5000, che la cache conserva; dopo una modifica si ricolora dalla riga modificata in giù. In ogni riga tutti i modelli attivi sono provati insieme e *vince la corrispondenza più a sinistra* a parità di posizione le regioni vengono prima delle regole. Così `// this /* is not` è un commento di riga e non apre un commento a blocchi. Le espressioni regolari sono quelle della crate `regex`, senza lookahead,

lookbehind né backreference: per questo una regola può colorare solo un gruppo (`SyntaxRule::group`) e una regione può dichiarare una sequenza di escape.

La *tabella sintattica* dice quali caratteri aprono e chiudono liste, quali delimitano stringhe (`set-string-syntax`, anche multi-carattere come `r#"`), quali commenti (`set-comment-syntax`) e quale carattere quota un carattere (`set-char-quote`). Le due descrizioni sono indipendenti: aggiungere una forma di stringa a un modo significa aggiungerla in entrambe.

10.3 Lo scanner delle espressioni

`modes/sexp.rs` risponde alle domande sulla struttura del testo: dove finisce l'espressione dopo il punto (`forward-sexp`), dove comincia quella prima (`backward-sexp`), in quale lista, stringa o commento si trova una posizione (`syntax-ppss`, che l'indentazione e `electric-pair` usano). Contare le parentesi non basta: in `(message "close it with )")` tre parentesi su quattro sono testo. Lo scanner quindi fa un'analisi lessicale con quattro stati, e conta un delimitatore solo quando lo sta leggendo come codice. Gli stati e le transizioni:

Stato	Ci si entra da `Code` quando…	Si torna a `Code` quando…
`Code`	— (stato iniziale)	—
`Str { quote, start }`	si legge un carattere di classe `StringQuote`.	si rilegge la stessa virgoletta; un carattere `Escape` fa saltare anche il successivo.
`StyledStr { style, start }`	comincia un'apertura multi-carattere dichiarata con `set-string-syntax` (`r#"`, `"""`).	si legge la chiusura dello stile. Nessun escape: in una stringa grezza `\` è solo `\`.
`Comment { style, depth, start }`	comincia un'apertura di commento di `set-comment-syntax` (vince la più lunga).	riga: al `\n`. Blocco: alla chiusura con `depth` 0; se lo stile è `nestable` un'altra apertura incrementa `depth`.

Lo stato completo di una scansione è lo stato lessicale più una pila di `Frame`, uno per ogni lista aperta. Il fondo della pila è il buffer stesso e non viene mai tolto: una chiusura in più non svuota mai la pila.

da `modes/sexp.rs` — estratto semplificato

```
struct Frame {
    start: usize,              // inizio dell'espressione, prefisso incluso
    delimiter: usize,          // dove sta la parentesi
    children: VecDeque<Sexp>,  // le ultime `keep` espressioni completate qui
}
pub struct Scan<'a, B: BufferTrait> {
    text: &'a B, table: &'a SyntaxTable,
    pos: usize, state: State, stack: Vec<Frame>,
    prefix: Option<usize>,     // un carattere prefisso (' ` # ,) in attesa
    keep: usize,
}
```

`step()` fa un passo e dice se ha completato un'espressione (`Completed(depth, Sexp)`), se ha solo consumato caratteri (`Advanced`) o se il testo è finito (`End`). In stato `Code` l'ordine dei controlli conta:

1. Un'apertura di commento (la più lunga che corrisponde) entra in `Comment` e scarta un prefisso in attesa. Anche un commento completato conta come espressione, così `forward-sexp` lo scavalca.
2. Un'apertura di stringa multi-carattere viene prima dei caratteri singoli, perché contiene virgolette che altrimenti aprirebbero una stringa normale.
3. Un letterale di carattere (`'a'`, `'"'`) è un'unica espressione. Il `'` non ha una classe fissa: apre un carattere in `'a'`, una lifetime in `'static`, un apostrofo nella prosa; lo decide `SyntaxTable::char_literal_at` guardando ciò che segue.

4. Poi la classe del carattere: `Open` mette un `Frame` sulla pila; `Close` toglie quello in cima e completa la lista. Una chiusura che non corrisponde all'apertura chiude lo stesso, e una chiusura senza niente di aperto è testo: un file è sbilanciato per tutto il tempo in cui lo si scrive, e uno scanner che si fermasse lì sarebbe inutile proprio quando serve. `StringQuote` entra in `Str`; `Escape` salta due caratteri; `Prefix` ricorda la posizione del primo prefisso di una serie (`#'foo` comincia al `#`); `Symbol` consuma l'intera serie di costituenti e completa un'espressione; `Punctuation` avanza, e uno spazio stacca un prefisso in attesa.

Ogni espressione completata entra nei `children` del frame in cima, che ne tiene al massimo `keep`. È il limite che rende la scansione economica: `backward-sexp` con un conteggio N ha bisogno solo delle ultime N.

Funzione	Come usa la scansione
`forward(from, n)`	Scansione fino a `from`, poi passi finché non si completano `n` espressioni *allo stesso livello* che finiscono dopo `from`. Se la lista finisce prima, il punto non si muove: alla fine di una lista il comando non esce verso l'esterno (→ SIN-6).
`backward(from, n)`	Scansione fino a `from` con `keep = n`; il frame rimasto aperto è la lista in cui si trova il punto, e i suoi `children` sono i fratelli prima del punto. Restituisce l'inizio dell'N-esimo dal fondo, prefisso incluso.
`down(from)`	Scansione finché la profondità non supera quella di partenza: il punto è appena entrato in una lista.
`context_at(pos)`	Scansione fino a `pos`, poi `context()`: profondità, inizio e delimitatore della lista più interna, inizio della stringa o del commento in corso. È ciò che restituisce `syntax-ppss`.
`balance_point(pos)`	La prima posizione dopo `pos` in cui tutte le liste aperte in `pos` sono chiuse. Non chiede se il buffer è bilanciato alla fine: una funzione chiusa resta chiusa anche se quella sotto è ancora da finire.
`enclosing(pos)`	La lista più interna che contiene `pos`, per intero (fino alla fine del testo se non è chiusa); `None` al livello zero.

10.3.1 Riprendere da metà: `Resume` e il `Prescanner`

Andare all'indietro non si può fare localmente: se una `)` prima del punto sia codice o testo dipende da tutto ciò che la precede, perché una stringa aperta duecento righe sopra la rende testo. Quindi ogni scansione va in avanti, e può cominciare solo in una posizione di cui si conosce lo stato. All'inizio quella posizione era solo la 0; ora `Scan::begin` accetta un `Resume`, una fotografia dello stato presa prima:

da `modes/sexp.rs` — estratto semplificato

```
pub struct Resume { pos: usize, state: State, stack: Vec<Frame>, prefix: Option<usize> }
pub fn begin(text, table, before: usize, resume: Option<&Resume>, keep: usize) -> Self {
    if let Some(from) = resume.filter(|from| from.pos <= before) {
        return Self { pos: from.pos, state: from.state.clone(),
                      stack: from.stack.clone(), .. };          // riprende da lì
    }
    Self { pos: 0, state: State::Code,
           stack: vec![Frame { start: 0, delimiter: 0, .. }], .. }  // dall'inizio
}
```

Il controllo `from.pos <= before` è l'intero argomento di sicurezza: una fotografia è solo una scorciatoia per arrivare a una posizione, mai un sostituto. Una fotografia sbagliata o troppo avanti costa una scansione completa, non una risposta sbagliata. Le fotografie sono prese senza i `children` dei frame, che dipendono da `keep`: per questo `backward` non riprende mai e scansiona sempre dall'inizio del buffer.

Le fotografie le produce il `Prescanner`, un lavoro di sfondo come l'evidenziatore (Sezione 11). Ne prende una ogni `LINES_PER_CHECKPOINT` (64) righe, all'inizio delle righe 64, 128, 192… (`checkpoint_line`), e le salva nella `ScanCache` del buffer insieme alla versione del testo e al nome del modo. Una fotografia presa con la tabella di un altro modo sarebbe una risposta sbagliata che sembra giusta, e per questo il modo è controllato. Dopo una modifica alla riga L, `invalidate_from` tiene le fotografie delle righe fino a L (lo stato all'ingresso di una riga dipende solo dal testo prima) e scarta le altre; `record` accetta solo la fotografia successiva all'ultima valida. `Buffer::scan_resume(pos)` sceglie quella della riga multipla di 64 più vicina sotto `pos`, se versione e modo corrispondono.

10.4 Facce e temi

Una faccia è un `Face(u16)`: un indice in un registro globale. Le prime 14 sono predefinite (`default`, `region`, `mode-line`, `mode-line-inactive`, `window-separator`, `keyword`, `type`, `string`, `comment`, `function`, `builtin`, `line-number`, `line-number-current`, `replace-match`); un modulo crea una faccia nuova semplicemente nominandola (`'doc-comment` in `rust-mode.lisp`) e le dà un aspetto con `set-face`. Il `Theme` in `Runtime` associa a ogni faccia uno `Style` (colori di primo piano e sfondo, grassetto, corsivo, sottolineato, inverso). I colori sono richieste — un nome o `#rrggbb` — e il front-end decide come approssimarli (Sezione 12).

11 Il lavoro in background

Tutto il lavoro che l'editor fa lontano dal thread dei comandi passa da un solo oggetto, il *WorkManager* di `work/`: la colorazione, la prescansione, il controllo dei file, il salvataggio automatico, i lavori Lisp, i comandi di shell e le ricerche nei file. Il WorkManager li esegue, li elenca, li ferma e ne misura i tempi. Non conosce l'editor: lo stato su cui un lavoro lavora è un parametro di tipo, `C`, che nell'editor è `EditorState<B>` e nei test di `work/` è `()`.

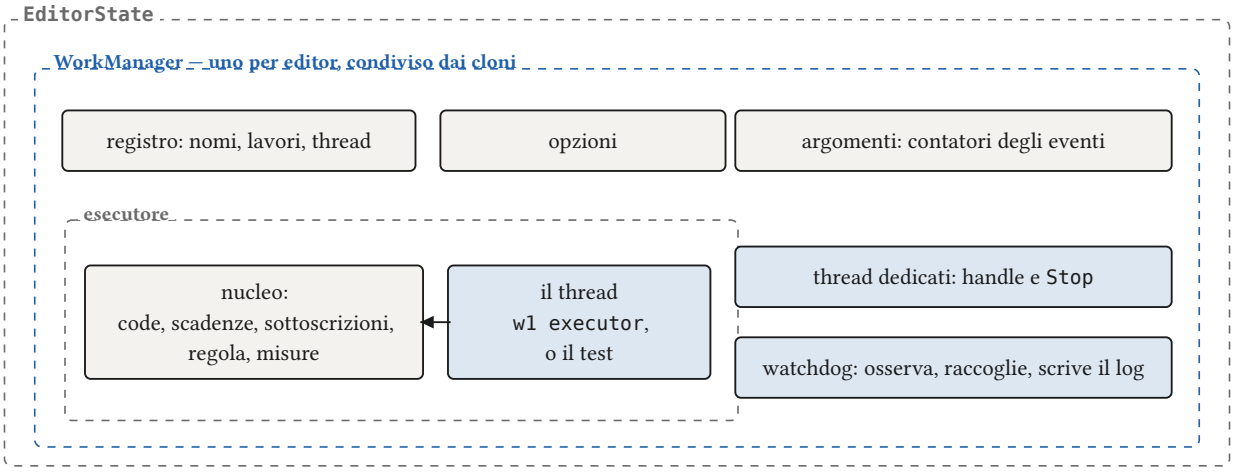
Le regole, in sei righe.

1. **Un solo punto d'ingresso.** Il lavoro asincrono passa dal WorkManager, ha un nome e un `WorkId`, e si può elencare e fermare.
2. **Due forme di lavoro.** I lavori a passi girano sull'esecutore; ciò che può bloccarsi, un processo o il disco, ha un thread dedicato.
3. **Un passo non aspetta mai.** Fa una quantità limitata di lavoro e dice come proseguire.
4. **Lo scheduler misura e segnala, non interrompe.** Un passo troppo lungo finisce nel log; la correzione si fa a mano.
5. **Gli argomenti non portano dati.** Dicono «vai a guardare»: la fonte della verità è lo stato.
6. **Tutto si prova senza thread.** Il nucleo riceve l'ora da un orologio che i test muovono a mano.

Le parole: nel codice un *job* (lavoro) è ciò che gira a passi, e *work* è un lavoro o un thread dedicato; *worker* resta solo nei nomi del Lisp, come `define-worker` e `worker-fuel`. Restano fuori dalla prima regola i thread che `(spawn ...)` avvia nell'interprete, e quello breve con cui un comando di shell fermato riceve `KILL` (Sezione 11.10).

11.1 Il WorkManager

Ce n'è uno per editor, condiviso da ogni clone di `EditorState`: clonarlo costa un `Arc`. La figura mostra che cosa possiede; in azzurro i thread, il resto sono dati che qualunque thread raggiunge attraverso il WorkManager.



I file di `work/` seguono le parti:

File	Che cosa contiene
`manager.rs`	`WorkManager`, la facciata: costruzione, `start`, `submit`, `spawn`, `cancel`, le domande al registro, gli hook, `shutdown` e la modalità manuale; `Shared`, ciò che i suoi cloni condividono.
`registry.rs`	`Registry`: che cosa esiste, per numero e per nome (Sezione 11.6).

File	Che cosa contiene
`executor.rs`	`Msg`, `Executor` ed `executor_main`: la casella dei messaggi e il ciclo che guida il nucleo (Sezione 11.5).
`watchdog.rs`	Il thread del watchdog e il suo turno (Sezione 11.8).
`core.rs`	`Core`, il nucleo: i lavori, le code, la regola, le scadenze e le misure (Sezione 11.3).
`job.rs`	Il contratto di un lavoro: `Job`, `Step`, `Exit`, `Wake`, `StepContext`, `Spec` e `WorkId` (Sezione 11.2).
`topic.rs`	`Topics` e `Topic`, gli argomenti (Sezione 11.4).
`threads.rs`	I thread dedicati: `Stop`, `Lives` e `start_thread` (Sezione 11.7).
`options.rs`, `report.rs`	Le opzioni e le righe `[BKG]` del log (Sezione 11.9).
`clock.rs`	`Clock`, `SystemClock` e `ManualClock` (Sezione 11.11).

11.1.1 Due modalità

`WorkManager::threaded` è quello dell'editor: un thread esegue il nucleo sull'orologio vero, da `start` a `shutdown`. `WorkManager::manual` è quello dei test, e `editor/boot.rs` lo sceglie quando la libreria è compilata per i test (`cfg!(test)`): nulla gira da solo, il nucleo legge un `ManualClock`, e il lavoro avanza quando il test chiama `run_until_idle` o `run_for` (Sezione 11.11). Il resto è uguale nelle due modalità, compreso il modo in cui entra un lavoro.

Una chiamata al WorkManager non tocca mai il nucleo. Cambia il registro, che risponde subito, e lascia un messaggio nella casella dell'esecutore, che lo legge tra un passo e l'altro. Così `submit` dall'interno di un passo è sicuro come da qualunque altro punto, anche in modalità manuale, dove l'esecutore è il thread del test.

11.1.2 Il ciclo di vita

1. **Nasce** in `EditorState::new`, prima dei compartimenti, perché i buffer ricevono da lui l'argomento `buffer-changed` (Sezione 11.4). Il `Log` dell'editor nasce ancora prima: le segnalazioni del WorkManager vanno lì.
2. **Parte** con `start(stato)`, appena lo stato esiste: il thread dell'esecutore riceve la sua copia di `EditorState`, e con lui parte il watchdog. Ciò che è stato sottomesso prima aspetta nella casella. In modalità manuale `start` non fa nulla.
3. **Riceve i lavori dell'editor**: evidenziatore e prescansione in `EditorState::new`, controllo dei file e salvataggio automatico in `create_global_env`, dopo `init.lisp`, perché il loro intervallo è un'impostazione (Sezione 11.10).
4. **Termina** con `shutdown`, che l'editor espone come `EditorState::shutdown(entro)`. Il front-end lo chiama all'uscita, con una guardia che lo chiama anche quando un panic attraversa `main` (`ShutDownOnExit` in `src/main.rs`, con mezzo secondo di attesa).

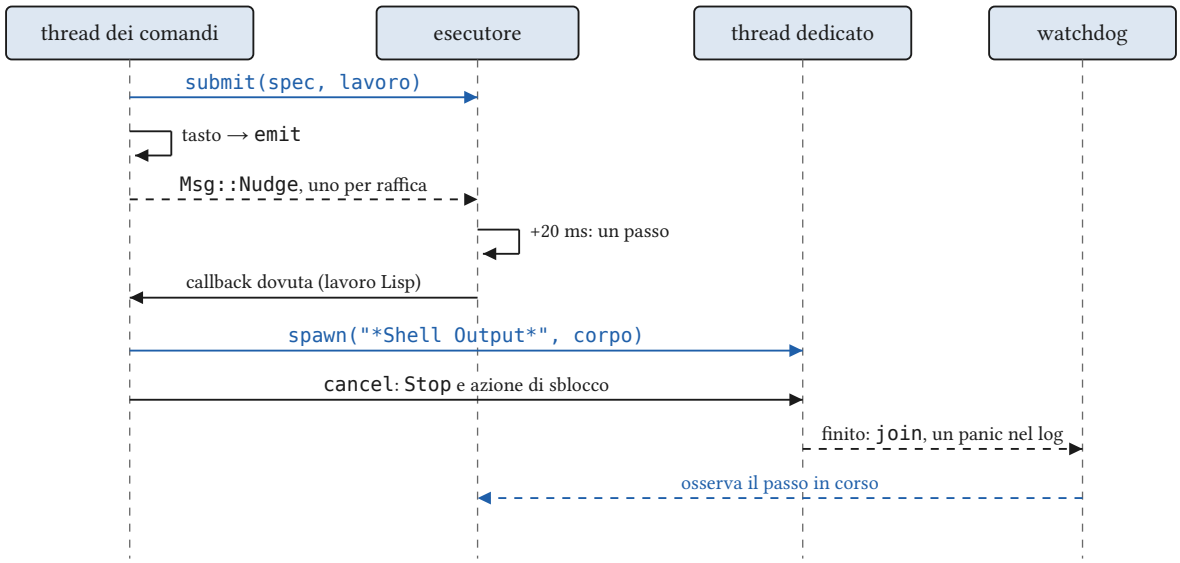
`shutdown` procede così:

1. chiude il registro: da qui `submit` e `spawn` rispondono `None`;
2. mette `Msg::Shutdown` nella casella con il lock del registro preso, così ogni lavoro accettato prima è già nella casella;
3. alza lo `Stop` di ogni thread dedicato ed esegue la sua azione di sblocco;
4. se nessun thread ha preso l'esecutore, in modalità manuale o prima di `start`, lo chiude sul thread del chiamante, e le `exit` dei lavori ricevono lo stato passato a `shutdown`; altrimenti lo chiude il thread dell'esecutore quando legge il messaggio. Ogni lavoro esce con `Exit::Shutdown`;
5. sveglia il watchdog, che si ferma, e aspetta che tutti i thread siano finiti, al massimo per `entro`;
6. fa `join` di ciò che è finito e abbandona il resto, nominandolo nel log: un thread che non si ferma, un passo che non finisce;

7. chiama un'ultima volta l'hook del watchdog, che porta sul file le righe appena scritte.

Una seconda chiamata non fa nulla.

11.1.3 Chi parla con chi



- **Il thread dei comandi** chiama il WorkManager: sottomette lavori, avvia thread dedicati, ferma, cambia opzioni. Le porte delle modifiche emettono gli argomenti, e una raffica di emissioni manda all'esecutore una sola sveglia. Le chiamate non aspettano: girano sul thread di chi chiama.
- **L'esecutore** esegue i passi uno alla volta. Un lavoro Lisp non chiama il suo modo: lascia una callback dovuta, che il thread dei comandi esegue tra un comando e l'altro (Sezione 11.10).
- **Un thread dedicato** lavora per conto suo, e si ferma con lo `Stop` e con l'azione che lo sblocca.
- **Il watchdog** osserva il passo in corso, raccoglie i thread finiti e scrive il file di log, così una segnalazione arriva sul file anche quando ciò che segnala è bloccato.

11.2 I lavori

Un lavoro implementa `Job<C>`:

da `work/job.rs` — estratto semplificato

```
pub trait Job<C>: Send + 'static {
    fn step(&mut self, cx: &mut StepContext<'_, C>) -> Result<Step, JobError>;
    fn exit(&mut self, _state: &C, _why: Exit) {}
}
```

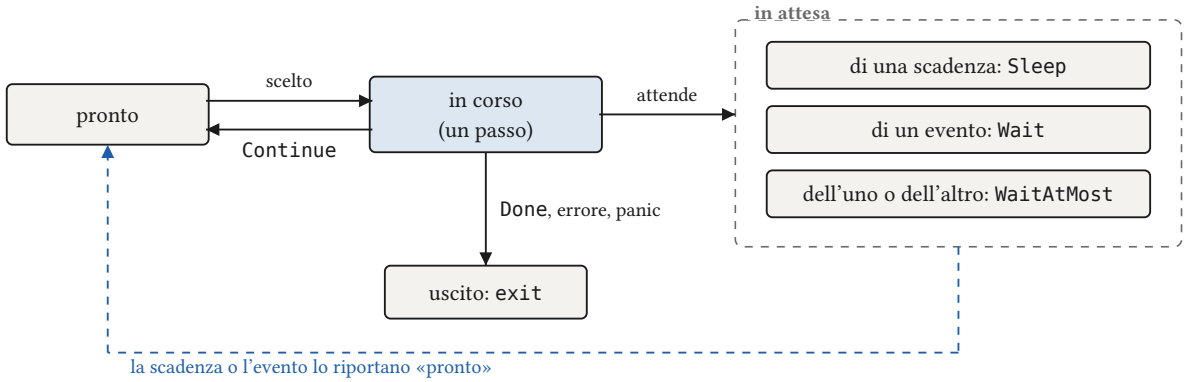
Lo `StepContext` porta lo stato (`state`), il motivo del passo (`woken_by`: `Start`, `Turn`, `Event` o `Timer`) e il numero del lavoro (`id`). Con `cx.worked(unità, su_che_cosa)` il passo dichiara quanto ha fatto, e con `cx.subscribe(argomenti)` cambia gli argomenti che lo svegliano. Il passo risponde con un `Step`:

`Step`	Quando viene il passo successivo
`Continue`	c'è altro da fare: il lavoro torna in fondo alla sua coda, e se nessun altro è pronto riparte subito
`Sleep(d)`	dopo `d`; gli eventi nel frattempo non lo svegliano, ma il passo dopo vede lo stato che annunciavano
`Wait`	al prossimo evento di uno dei suoi argomenti
`WaitAtMost(d)`	al prossimo evento, o dopo `d` al più tardi
`Done`	mai: il lavoro ha finito

Le regole di un passo:

- **Non aspetta mai.** Niente I/O che può bloccarsi, che va su un thread dedicato, e niente `sleep`: chi deve aspettare lo dice nell'esito.
- **Non tiene lock da un passo all'altro.** Copia sotto lock, lavora senza, e salva solo se la versione non è cambiata.
- **Non guarda l'orologio per decidere quanto fare.** Divide il lavoro in unità dei suoi dati (righe, file, carburante) e le dichiara con `cx.worked`; il tempo lo misura lo scheduler.
- **Non si interrompe.** `cancel` ha effetto alla fine del passo in corso.
- **Gli errori passano dal `Result`.** Lo scheduler li scrive nel log con il nome del lavoro, e il lavoro esce.

Un lavoro sottomesso è pronto, o in attesa della sua scadenza se la `Spec` dice `Start::After`. Poi passa da uno stato all'altro così:



`cancel` e `shutdown` portano a «uscito» da qualunque stato; un lavoro in corso ci arriva alla fine del passo. Il motivo dell'uscita è un `Exit`:

`Exit`	Quando
`Finished`	un passo ha risposto `Done`
`Failed`	un passo ha restituito un errore, o ha risposto `Wait` senza un argomento che possa svegliarlo
`Panicked`	un passo è andato in panic: il nucleo lo intercetta con `catch_unwind`
`Cancelled`	qualcuno l'ha fermato, o un lavoro con lo stesso nome l'ha sostituito
`Shutdown`	il WorkManager si chiude

`exit` arriva una volta sola, per qualunque motivo, anche a un lavoro fermato prima del primo passo: è il posto della pulizia, per esempio la callback `ON-DONE` di un lavoro Lisp. Gira sull'esecutore come un passo, ed è protetta e osservata come un passo.

Un lavoro si sottomette con una `Spec`, che ne dà il nome e il modo di eseguirlo: `Spec::new(nome)`, poi `class`, `unit` (l'unità di `cx.worked`, come le segnalazioni la stampano), `on` (un argomento) e `start`. Il nome è unico: sottomettere un lavoro con un nome già preso ferma ciò che lo porta, lavoro o thread dedicato. Il `WorkId` è un numero di tutto il processo e non si riusa: un numero tenuto dopo la fine del suo lavoro non può nominarne un altro.

11.3 Il nucleo

`Core` tiene i lavori e decide che cosa gira. Non ha un thread né un orologio suo: chi lo guida chiama `run_once` un turno alla volta, e il nucleo legge l'ora dal `Clock` che ha ricevuto. Un lavoro è in un solo posto alla volta (`Place`): pronto, nella coda della sua classe; in corso; in attesa di una scadenza; in attesa di un argomento, e forse anche di una scadenza. Un turno:

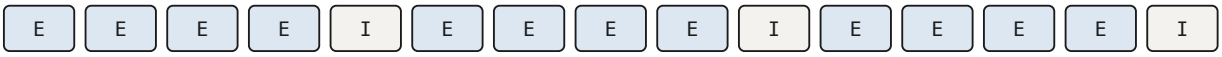
1. chiama `exit` sui lavori usciti, con lo stato che il turno porta;

2. rende pronti i lavori la cui scadenza è passata o i cui eventi sono maturi (`poll`), nell'ordine in cui lo sono diventati;
3. chiude le finestre di segnalazione scadute e controlla il carico dell'ultimo secondo (`tick`, Sezione 11.9);
4. sceglie un lavoro pronto (`choose`) ed esegue un suo passo, misurato (`run_step`);
5. chiama `exit` sui lavori che il passo ha fatto uscire.

Mentre fa il passo, il lavoro è fuori dal nucleo: il suo record non lo contiene, e un lavoro non può girare due volte insieme. Un lavoro fermato durante il passo trova il record sparito quando il passo finisce, ed esce allora. Mentre un passo o una `exit` è in corso, il nucleo lo scrive in `Underway` (il lavoro, l'inizio e un numero d'ordine), che altri thread leggono: il watchdog, per segnalare un passo che non finisce, e `WorkManager::jobs`, per elencare il lavoro come in corso. È proprio quando un passo non finisce che il nucleo non può rispondere da sé.

11.3.1 Chi passa prima

Le classi sono due. `Class::High` è per ciò che l'utente aspetta di vedere, colore e prescansione; `Class::Normal` è il default, e vi stanno il controllo dei file, il salvataggio automatico e tutti i lavori Lisp. Ogni classe ha la sua coda, e un lavoro entra in fondo alla sua ogni volta che diventa pronto. Passa prima la classe alta, ma dopo `high-steps` passi alti di fila (4 per default) tocca a un lavoro normale pronto. La regola si conta in passi e non in tempo, così è deterministica.



`E` = l'evidenziatore su un file enorme, che risponde sempre `Continue`; `I` = un lavoro Lisp con `(yield 'continue)`: un passo ogni cinque.

Quando l'evidenziatore ha finito, il lavoro Lisp ha l'esecutore tutto per sé.

11.3.2 Le scadenze

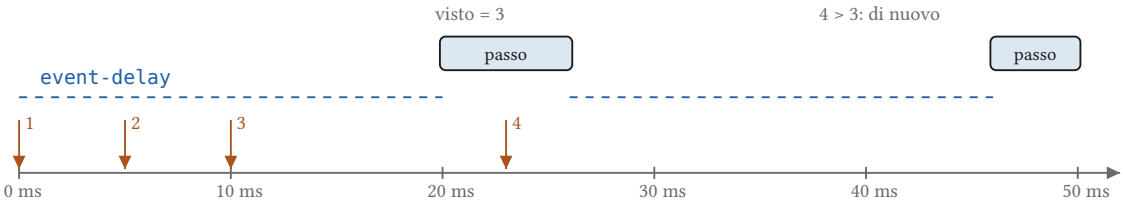
Un lavoro ha al più una scadenza, messa da `Sleep`, da `WaitAtMost` o da `Start::After`, e contata dalla fine del passo: un lavoro lento non si accoda a sé stesso, e un periodo saltato non si recupera. Nessuna scadenza va oltre un anno (`FURTHEST`): un `Instant` molto più lontano può non esistere, e sommarlo andrebbe in panic sull'esecutore, fuori da ogni passo.

11.4 Gli argomenti

Un argomento (`Topic`) è un evento con un nome, e non porta dati: è un contatore. Emetterlo (`emit`) costa un incremento atomico e, se l'esecutore non ha già una sveglia in arrivo, un `Msg::Nudge` nella sua casella; si può fare anche con il lock di un buffer preso. Un nome si risolve una volta (`WorkManager::topic`), e da lì l'argomento si emette senza toccare lock.

- **La sottoscrizione è persistente.** Un lavoro ha eventi in sospeso quando il contatore di un suo argomento ha superato l'ultimo valore che ha visto. Il nucleo aggiorna quel valore *prima* del passo: un evento che arriva durante il passo resta non visto, e risveglia il lavoro.
- **Il ritardo raccoglie le raffiche.** Un lavoro in attesa diventa pronto `event-delay` (20 ms) dopo che il nucleo ha notato il primo evento non visto, e gli eventi che arrivano nel frattempo si sommano a quello. Gli eventi arrivati durante un passo maturano dalla fine del passo.
- **Un `Wait` senza argomenti è un errore.** Nessuno potrebbe svegliare il lavoro: il nucleo lo segnala, e il lavoro esce con `Exit::Failed`.
- **Un passo può cambiare i suoi argomenti** con `cx.subscribe`: è ciò a cui si riduce un `(yield '(wait buffer-changed))` del Lisp. Un argomento nuovo conta come cambiato, perché il passo che lo nomina può aver guardato lo stato appena prima di un'emissione.
- **Un lavoro che dorme ignora gli eventi**, senza perderli: il passo successivo vede lo stato che annunciavano.

Un esempio: tre tasti in 15 ms producono un solo passo, e un quarto tasto che arriva durante il passo ne produce un secondo. Le frecce sono le emissioni, con il valore del contatore.



L'editor emette due argomenti, dichiarati in `editor/work.rs`; il Lisp ne emette di suoi con `emit-event`.

Argomento	Chi lo emette
`buffer-changed`	`edits::changed` (le due porte, undo, redo, `replace-back`), `Buffer::adopt_text`, il cambio di modo maggiore, un buffer che entra nella tabella
`syntax-changed`	`set_mode`, `edit_grammar`, `edit_syntax_table`

Il modulo dei buffer non conosce lo scheduler. Il compartimento `Buffers` riceve l'argomento `buffer-changed` già risolto e lo dà a ogni buffer che entra nella tabella; fino ad allora un buffer ha un argomento staccato (`Topic::detached`), che conta e non sveglia nessuno.

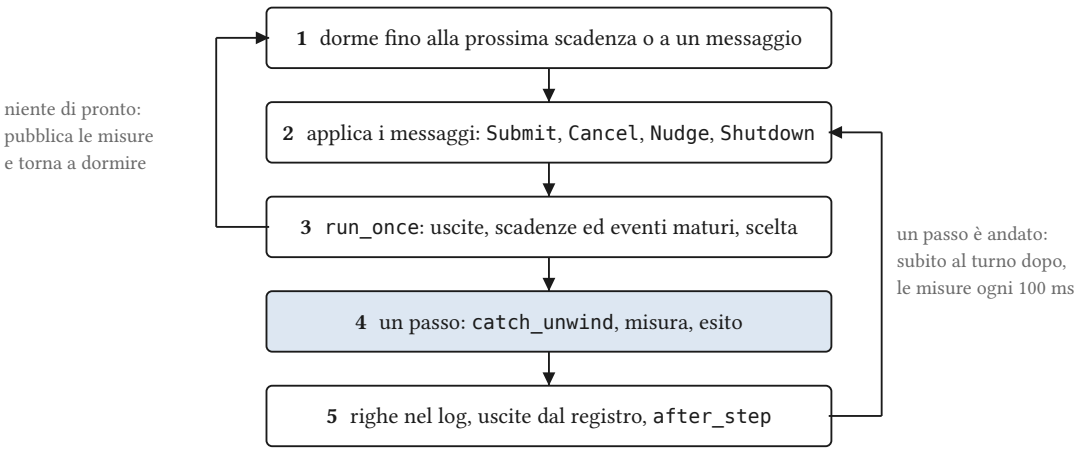
11.5 L'esecutore

L'esecutore (`Executor`) è il nucleo con l'altro capo della casella. Lo guida il thread `w1 executor` dell'editor, da `start` a `shutdown` (`executor_main`), o il test in modalità manuale; il numero nel nome dei thread è quello del WorkManager nel processo. Un turno applica i messaggi in attesa e poi esegue al più un passo:

da `work/executor.rs` — estratto semplificato

```
pub(super) fn turn(&mut self, shared: &Shared<C>, state: &C) -> bool {
    self.drain();
    if self.shut {
        return false;
    }
    shared.nudged.swap(false, Ordering::AcqRel);
    let stepped = self.core.run_once(state);
    self.settle(shared);
    if stepped {
        shared.stepped();
    }
    stepped
}
```

Il ciclo di `executor_main` ripete i turni finché un passo va, e dorme quando non c'è niente di pronto:



- **Le sveglie non si perdono.** Il flag `nudged` dice che un `Msg::Nudge` è già nella casella, e fa di una raffica di emissioni una sola sveglia. Il turno lo abbassa con uno scambio atomico *prima* che il nucleo guardi gli argomenti: un'emissione che il nucleo non vedrà trova il flag basso e manda una sveglia, e una che lo trova alto è avvenuta prima, e il nucleo la vede.
- **Dopo ogni passo, l'hook.** Quando il registro ha già tolto i lavori usciti, il turno chiama l'hook di `after_step`: l'editor lo usa per dire al front-end che lo schermo può essere cambiato (Sezione 11.10). L'hook gira tra due passi, quindi è breve e non aspetta; un suo panic va nel log.
- **Le misure si pubblicano.** Le misure dei lavori passano dal nucleo al registro ogni `PUBLISH_EVERY` (100 ms) mentre l'esecutore è occupato, e prima di dormire quando non lo è; in modalità manuale, alla fine di ogni guida.
- **Dorme fino alla prossima cosa da fare**: la scadenza più vicina, un evento che matura o la chiusura di una finestra di segnalazione (`next_wakeup`), oppure un messaggio, se arriva prima.

11.6 Il registro

Il registro (`Registry`, dietro il lock `registry` di `Shared`) è la risposta del WorkManager a «che cosa esiste», data subito e senza chiedere all'esecutore:

Campo	Che cosa tiene
`closed`	vero dopo `shutdown`: da lì non entra più nulla
`names`	un solo spazio di nomi per lavori e thread: un nome indica una cosa sola, e fermarlo per nome ferma quella, qualunque sia
`latest`	il lavoro più recente sottomesso con ogni nome, finché qualcuno non lo ferma; resta anche dopo la sua fine
`jobs`	i lavori, con ciò che l'esecutore ha pubblicato di loro (`WorkInfo`)
`threads`	i thread dedicati (`Dedicated`): nome, `Stop`, handle, avvio, richiesta di stop

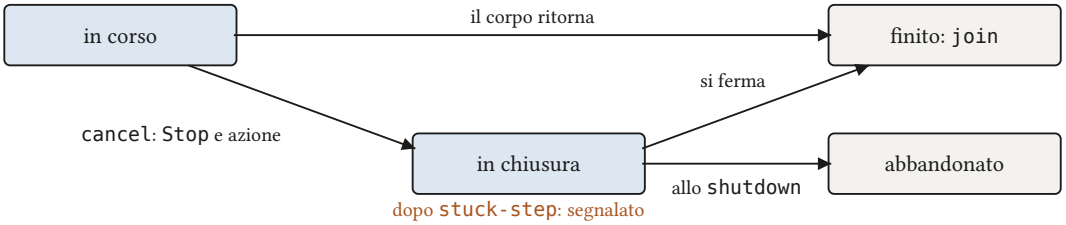
I messaggi per l'esecutore, tranne `Msg::Nudge`, partono con il lock del registro preso: la casella li tiene nell'ordine in cui il registro è cambiato, e dopo `Msg::Shutdown` non ne arriva un altro. Togliere un lavoro dal registro (`withdraw`) lascia un `Withdrawn` da completare senza lock: l'azione di sblocco di un thread gira dopo, sul thread di chi ha chiamato `cancel`, protetta come un passo.

Le domande che il registro risponde:

- `jobs()` elenca i lavori nell'ordine di sottomissione, con le misure dell'ultima pubblicazione; un lavoro che l'esecutore non ha ancora visto appare come è stato sottomesso. Il lavoro nel mezzo di un passo è elencato come in corso da ciò che dice `Underway`, perché dopo un passo che non finisce non si pubblica più nulla.
- `threads()` elenca i thread dedicati ancora vivi, dopo aver raccolto quelli finiti.
- `is_named(nome)` dice se un nome è preso. Lo usa `free_name_for_work`: un comando di shell prende il nome del buffer in cui scrive, e quel nome deve essere libero sia tra i buffer sia qui.
- `is_latest(nome, id)` dice se `id` parla ancora per il suo nome. È la domanda delle callback dovute (Sezione 11.10): un lavoro finito ha ancora diritto alla sua `ON-DONE`, uno sostituito o fermato no.

11.7 I thread dedicati

Un thread dedicato serve a ciò che può bloccarsi e non si divide in passi: leggere l'output di un processo, percorrere un albero di directory. `WorkManager::spawn(nome, corpo)` lo avvia con il nome `w1 nome` (Linux ne mostra 15 byte), lo mette nel registro accanto ai lavori e restituisce il suo `WorkId`; il corpo riceve lo `Stop` e il proprio numero.



- **Un thread non si uccide: gli si chiede di fermarsi.** `cancel` alza lo `Stop`, che il corpo controlla nei punti da cui può uscire, ed esegue l'azione registrata con `Stop::on_cancel`, che sblocca ciò su cui il corpo aspetta: per un comando di shell, fermare il processo chiude la pipe che il corpo legge. L'azione gira una volta sola, anche se è registrata dopo la richiesta, e gira sul thread che chiede lo stop: deve essere breve.
- **Il `join` non blocca nessuno.** Il watchdog, e chi chiede l'elenco dei thread, lo fa solo sui thread che risultano finiti (`is_finished`); un panic nel corpo torna dal `join` e va nel log.
- **Un thread che non si ferma**, per esempio una `stat` su un disco di rete appeso, resta elencato come in chiusura, viene segnalato dopo `stuck-step` e, allo `shutdown`, abbandonato.
- **`shutdown` sa quanto aspettare.** Ogni thread del WorkManager porta una quota di `Lives` dall'avvio alla fine; `shutdown` aspetta che il conto torni a zero, al più per il tempo che gli è dato.

L'editor non chiama `spawn` direttamente. `EditorState::spawn_work` passa al corpo l'editor, tiene il thread fuori dal thread dei comandi per tutta la sua vita (`WorkerTurn`, Sezione 11.10) e, quando il corpo ritorna, dice al front-end di ridisegnare. `spawn_writer` aggiunge un buffer di uscita: il thread prende il nome del buffer, il buffer sa chi scrive in lui prima che il corpo possa scrivere una riga, e chiudere il buffer ferma il thread.

11.8 Il watchdog

Il watchdog è un thread, `w1 watchdog`, che parte con l'esecutore, o con il primo thread dedicato se viene prima, e si sveglia ogni `watchdog-every` (250 ms). Guarda ciò che nessun altro può guardare: il passo in corso, che l'esecutore non può segnalare finché ci sta dentro. Un turno (`watch_turn`):

1. raccoglie i thread dedicati finiti (`reap`), e segnala quelli ancora vivi `stuck-step` dopo la richiesta di stop;
2. segnala il passo in corso se dura da `stuck-step` (1 s), e di nuovo ogni volta che la sua durata raddoppia: 1 s, 2 s, 4 s… Un turno in ritardo segnala una volta sola, con la durata vera, e il segno successivo è il primo raddoppio oltre quella. Il numero d'ordine di `Underway` distingue un passo dal successivo, anche dello stesso lavoro; una `exit` che non finisce è segnalata allo stesso modo, «in its exit»;
3. chiama per ultimo l'hook di `after_watch`, così ciò che il turno ha segnalato è già in ciò che l'hook scrive.

L'editor passa ad `after_watch` la funzione `Log::flush`: il file di log si scrive qui, e non da chi scrive una riga. In modalità manuale il watchdog non c'è: i thread finiti si raccolgono quando il test guida il WorkManager, e il test fa un turno del watchdog con `watch_once`.

11.9 Misure, segnalazioni e opzioni

Il nucleo misura ogni passo: quanto è durato, quanto il lavoro ha aspettato in coda e le unità dichiarate con `cx.worked`, da cui viene il costo per unità. Per ogni lavoro tiene i passi, il tempo totale e il passo più lungo; per l'esecutore, l'occupazione dell'ultimo secondo, misurata esattamente su una finestra che scorre. Quando una misura supera la sua soglia, una riga `[BKG]` va nel log. Nessuna segnalazione cambia ciò che un lavoro fa: un lavoro lento si corregge a mano, con le segnalazioni come prova.

Riga	Quando (opzione)	Che cosa riporta
`long step`	un passo dura più di `long-step`	lavoro, durata, su che cosa, unità e costo per unità

Riga	Quando (opzione)	Che cosa riporta
`late start`	un lavoro pronto aspetta più di `late-high` o di `late-normal`	di quanto, e dietro quale lavoro
`load`	l'esecutore è occupato per più di `busy-share` dell'ultimo secondo	la quota, e i tre lavori che la causano
`summary`	si chiude una finestra di `summary-every` che ha contato qualcosa	quante altre occorrenze, e la peggiore
`stuck step`	un passo o una `exit` è in corso da `stuck-step`, poi dal doppio	lavoro e durata; la scrive il watchdog
`thread`	un thread dedicato è vivo `stuck-step` dopo la richiesta di stop, o viene abbandonato allo `shutdown`	nome e tempo trascorso
`error`, `panic`	sempre	lavoro e messaggio; il lavoro esce

La soglia del ritardo cambia con la classe perché un lavoro normale, per progetto, aspetta fino a `high-steps` passi alti. Per ogni coppia di lavoro e regola la prima occorrenza va subito nel log e apre una finestra di `summary-every`; le successive nella finestra si contano soltanto, e alla chiusura un riepilogo dice quante erano e la peggiore. Una finestra che si chiude vuota finisce, e la prima occorrenza dopo un periodo tranquillo va di nuovo subito nel log. Errori e panic non hanno limite: il lavoro esce, e non può ripeterli.

```
[BKG] long step: highlighter, 912 ms on app.min.js (3 lines, 304 ms each)
[BKG] late start: file-watcher started 230 ms late, behind highlighter (6 steps)
[BKG] load: 64% of the last second (highlighter 41%, manpage-index 20%)
[BKG] summary: highlighter, 12 more long steps in the last 10 s (worst 912 ms)
[BKG] stuck step: manpage-index running for 2.0 s
[BKG] thread: *Shell Output* still running 1000 ms after it was asked to stop
```

Le opzioni stanno nel WorkManager, in valori atomici che il nucleo legge a ogni scelta: una modifica vale dal passo successivo, e chi la fa non aspetta lo scheduler. Il Lisp le legge con `work-option` e `work-options` e le cambia con `set-work-option`, che rifiuta un valore fuori dall'intervallo. I tempi sono in millisecondi.

Opzione	Default	Valori	Che cosa regola
`high-steps`	4	1–1000	passi alti di fila prima di uno normale
`event-delay`	20	0–60 000	il ritardo dopo il primo evento non visto
`long-step`	40	1–3 600 000	la soglia del passo lungo
`late-high`	40	1–3 600 000	la soglia del ritardo, classe alta
`late-normal`	200	1–3 600 000	la soglia del ritardo, classe normale
`stuck-step`	1000	1–3 600 000	la soglia del passo bloccato e del thread che non si ferma
`busy-share`	50 %	1–100	la soglia del carico
`summary-every`	10 000	1–3 600 000	la durata di una finestra di riepilogo
`watchdog-every`	250	1–60 000	ogni quanto si sveglia il watchdog

`work-list` elenca ciò che è in background, lavori e thread, con stato e misure. La pagina `background` del manuale dell'utente spiega le une e le altre a chi scrive in Lisp.

11.10 Gli utenti del WorkManager

11.10.1 I lavori dell'editor

Lavoro	Classe, unità	Quando gira	Che cosa fa a ogni passo
`highlighter`	alta, righe	a `buffer-changed` e `syntax-changed`: `Continue` finché un buffer è indietro, poi `Wait`	colora fino a 500 righe del primo buffer indietro, partendo da quello con il focus
`prescanner`	alta, righe	come l'evidenziatore	porta avanti di fino a 500 righe la `ScanCache` di un buffer, con una fotografia ogni 64 (Sezione 10)
`file-watcher`	normale, buffer	`watch-file-interval` (3 s) dopo l'avvio e dopo ogni passo	confronta con il disco al più 8 buffer; ricarica o segna
`auto-saver`	normale, file	`auto-save-interval` (30 s) dopo l'avvio e dopo ogni passo	scrive la copia di al più 4 buffer modificati

Il codice è in `modes/`: `highlighter.rs`, `prescan.rs`, `watcher.rs` e `autosave.rs`.

Evidenziatore e prescansione sono sottomessi in `EditorState::new`: un lavoro che aspetta un cambiamento non costa nulla. Controllo dei file e salvataggio automatico sono sottomessi in `create_global_env`, dopo `init.lisp`, perché l'interruttore che li spegne e il loro intervallo sono impostazioni del Lisp, che rileggono a ogni passo. Ognuno è sottomesso una volta sola: un lavoro dell'editor che va in panic non torna fino alla fine della sessione (→ BKG-2).

Il passo dell'evidenziatore è il modello per scriverne altri:

da `modes/highlighter.rs` — estratto semplificato

```
impl<B: BufferTrait> Job<EditorState<B>> for Highlighter {
    fn step(&mut self, cx: &mut StepContext<'_, EditorState<B>>) -> Result<Step, JobError> {
        match cx.state.highlight_one_turn() {
            Some((buffer, lines)) => {
                cx.worked(lines as u64, &buffer);
                Ok(Step::Continue)
            }
            None => Ok(Step::Wait),
        }
    }
}
```

`highlight_one_turn` copia sotto lock le righe da colorare, lo stato del lessico all'inizio della prima e la versione del buffer; colora senza lock; riprende il lock e salva solo se la versione e il modo sono ancora quelli copiati. Un carattere digitato nel frattempo rende inutile il turno, che il passo successivo rifà dalla riga cambiata. Lo stesso schema regge la prescansione, che però legge le righe sotto il lock di lettura perché la scansione è veloce, il controllo dei file e il salvataggio automatico. Il costo di una riga cresce più che linearmente con la sua lunghezza (→ SIN-2), e una regione di sintassi che corrisponde alla stringa vuota fa un passo che non finisce: il watchdog lo segnala, ma l'esecutore resta fermo (→ SIN-1).

11.10.2 I comandi di shell e le ricerche

`shell-command-start` e la ricerca nei file (`grep--scan`, in `primitives/results.rs`) girano su un thread dedicato avviato con `spawn_writer`, con il nome del buffer in cui scrivono: `*Shell Output*`, o `<2>`, `<3>`… se il nome è preso tra i buffer o tra i lavori (`free_name_for_work`). Un comando legge

stdout e stderr da una sola pipe, nell'ordine in cui il processo li scrive, e alla fine aggiunge una riga `--- exited N ---`. Lo ferma `stop-worker` con il nome del suo buffer, o la chiusura del buffer. Su Unix l'azione di sblocco manda `TERM` e `CONT` al gruppo del processo, e un thread breve, `stop shell`, avviato fuori dal WorkManager, manda `KILL` due secondi dopo a ciò che resta. La pagina `shell` del manuale dell'utente dice che cosa vede chi lo usa.

11.10.3 I lavori Lisp

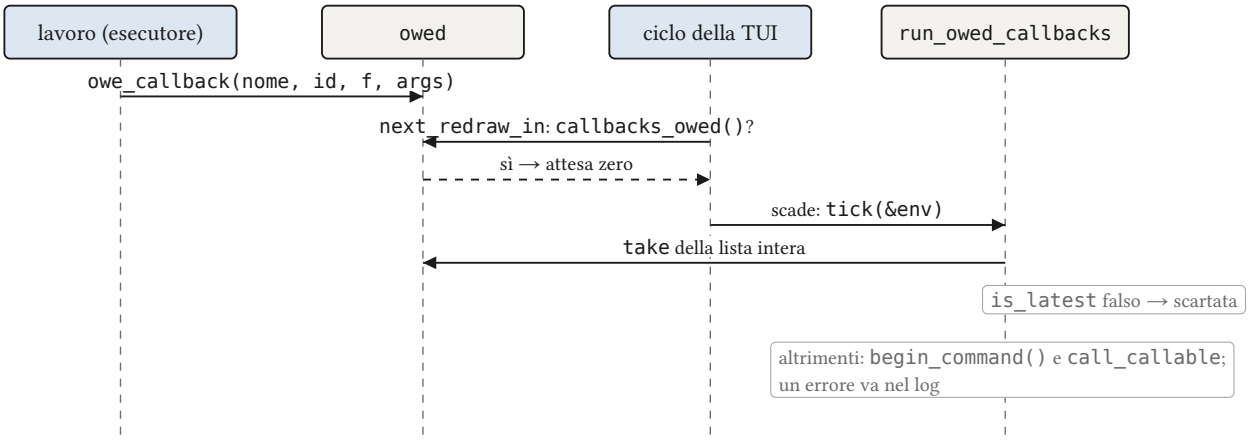
`define-worker` e `background-call` (`primitives/workers.rs`) sottomettono un `LispJob` (`primitives/lisp_job.rs`): classe normale, unità il carburante, nessun argomento finché un'attesa non ne nomina. Un passo riprende la fiber una volta, o chiama una volta la funzione di `background-call`, e ciò che la fiber passa a `yield` dice quando viene il passo dopo: `(yield)` dopo l'intervallo del lavoro, 40 ms se non è dato; `(yield 'continue)` appena tocca di nuovo a lui; `(yield '(sleep MS))`; `(yield '(wait TOPIC...))`; `(yield '(wait-at-most MS TOPIC...))`. La tabella completa, con gli esempi, è nella pagina `background`.

Ogni passo ha il suo carburante, `worker-fuel` (200 000 per default), letto quando il lavoro parte, impostato sul thread dell'esecutore all'inizio di ogni passo e rimesso com'era alla fine. Il carburante è per thread: un comando non accorcia il passo di un lavoro, e un lavoro non spende quello di un comando. Un passo che lo esaurisce finisce, e il lavoro esce come fallito; altrimenti un `(while t)` senza `yield` fermerebbe l'esecutore, e la colorazione con lui. Durante il passo `WorkerTurn` segna il thread come «in background» (`in_worker`), e le primitive che hanno senso solo sul thread dei comandi lo controllano: `minibuffer-read` rifiuta di aprire un prompt.

`ON-PROGRESS` e `ON-DONE` non girano sull'esecutore: il lavoro le lascia come callback dovute. `ON-PROGRESS` riceve il nome dopo ogni passo tranne l'ultimo. `ON-DONE` riceve il nome e `t` per `Exit::Finished`, `nil` per `Failed` e `Panicked`; dopo `Cancelled` e `Shutdown` non è chiamata, perché chi ha fermato il lavoro lo sa già.

11.10.4 Le callback dovute

Una callback chiesta da un lavoro va nella lista `owed` dell'editor (`owe_callback`), con il nome e il numero del lavoro. Il front-end lo sa da `next_redraw_in`, che risponde zero finché la lista non è vuota, e la esegue chiamando `tick`, che chiama `run_owed_callbacks`:



La lista si prende intera con un solo `take` prima di eseguire qualunque callback: una callback che ne chiede altre le aggiunge a una lista nuova, che girerà al prossimo `tick`, e il lock non resta mai preso mentre gira il Lisp. Una callback il cui lavoro non parla più per il suo nome (`is_latest`) si scarta. Le altre girano con un budget di carburante proprio (`begin_command`), sul thread che possiede l'interfaccia, tra un comando e l'altro: possono aprire buffer, scrivere messaggi o cambiare finestra come un comando, ma non sono comandi, e non cambiano `last-command`.

11.10.5 Quando ridisegnare

Quasi tutto ciò che è sullo schermo cambia perché l'utente ha fatto qualcosa; ciò che cambia un lavoro no. Per questo l'editor passa al WorkManager, come hook di `after_step`, `BackgroundChange::raise`. Alla fine di ogni passo, e quando un thread dedicato scrive nel suo buffer o finisce (`changed_in_background`), `raise` chiama una volta la funzione che il front-end ha registrato con `on_background_change`, e non di nuovo finché il front-end non prende una fotografia. La TUI la usa per svegliare il suo ciclo.

`next_redraw_in(&env, &frame)` è poi il minimo tra: la scadenza del messaggio nell'area messaggi (`echo-message-timeout`, 5 s; `nil` non scade mai); `TURN_INTERVAL` (40 ms) se la colorazione è in corso; il tempo che manca perché sia passato un `TURN_INTERVAL` dall'ultima fotografia, se un lavoro ha cambiato qualcosa da allora (`BackgroundChange::due_in`), così un lavoro che fa un passo al millisecondo non ridisegna il terminale a ogni passo; zero se ci sono callback dovute. `None` vuol dire che nulla è in sospeso e il front-end può bloccarsi sull'input. `drag_scroll_in()` aggiunge i 60 ms dello scorrimento automatico mentre si trascina una selezione oltre il bordo di una finestra.

11.11 Come si prova

Livello	Come
Il nucleo	`work_core_tests`: un `Core` guidato a mano su un `ManualClock`, senza thread. Un lavoro che deve durare 30 ms sposta l'orologio dall'interno del suo passo, e ogni test è esatto; un minuto di scheduler simulato richiede microsecondi.
Il WorkManager	`work_manager_tests`, `work_threads_tests` e `work_watchdog_tests`, con gli aiuti comuni di `work_support`. Quasi tutti usano la modalità manuale; i pochi con thread veri aspettano una condizione con una scadenza, mai un tempo fisso, e chiudono il WorkManager anche quando falliscono.
L'editor e il Lisp	L'editor dei test nasce in modalità manuale: i lavori girano solo con `ctx.work().run_until_idle(ctx)` o `run_for(ctx, durata)`, e i thread veri solo dove un test li vuole.

Gli altri strumenti:

- **Il watchdog in modalità manuale.** Il thread del test è l'esecutore, quindi un test che vuole osservare un passo lo fa eseguire su un altro thread (`drive_elsewhere`), sposta l'orologio e chiama `watch_once`.
- **Invarianti su sequenze casuali.** `random_sequences_keep_the_invariants` genera sottomissioni, `cancel`, emissioni ed esiti con un generatore scritto nel test, a partire da semi fissi, e controlla dopo ogni operazione che ogni lavoro stia in un solo posto, che `exit` arrivi una volta sola e che la regola di `high-steps` valga.
- **Nessun thread lasciato indietro.** Su Linux, `shutdown_leaves_no_thread_behind` conta in `/proc` i thread che portano il nome del WorkManager, e dopo `shutdown` non ne trova più.
- **Gli argomenti alla fonte.** `topic_source_tests` conta ciò che ogni fonte emette, e segue una modifica fino al lavoro che sveglia.

11.12 Le decisioni

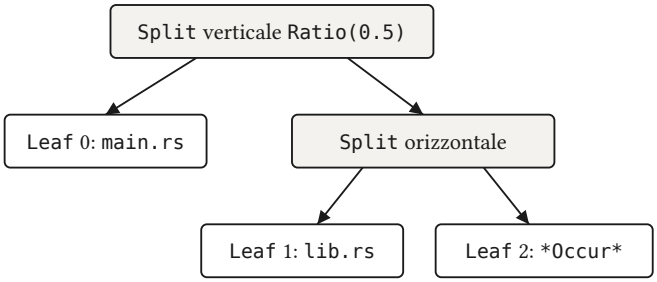
1. **Un solo WorkManager per tutto il lavoro asincrono.** Ogni lavoro, a passi o su un thread, entra dalla stessa porta, ha un nome e un numero, e si può elencare con `work-list` e fermare con `stop-worker`. Scartato: lasciare a ogni sottosistema i suoi thread, che nessuno elenca né ferma e che nessuno aspetta all'uscita.
2. **Lavori a passi su un esecutore, e un thread dedicato per ciò che si blocca.** Un passo non aspetta mai; ciò che deve aspettare ha un thread suo, che il WorkManager avvia e conosce. Comporta che un lavoro a passi si scriva come una macchina che riprende da dove era rimasta. Scartato: un thread per lavoro, che costa uno stack ciascuno e fa girare insieme lavori che prendono gli stessi lock.

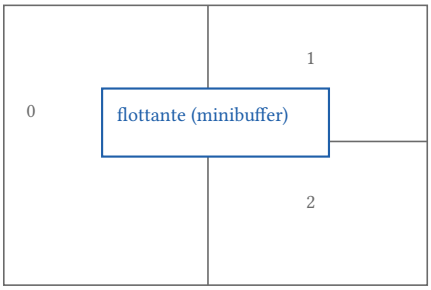
3. **Lo scheduler misura e segnala, non interrompe.** Un passo lungo finisce nel log con il suo costo per unità, e si corregge a mano. Scartati: un budget di tempo per passo, che renderebbe il lavoro dipendente dalla macchina e i test non deterministici, e l'interruzione di un passo, che Rust non sa fare in sicurezza. Il carburante dei lavori Lisp non è un budget di tempo: conta i passi del valutatore, uguali su ogni macchina.
4. **Gli argomenti non portano dati.** Un argomento dice «vai a guardare», e ciò che serve sta nello stato, dove nasce. Un messaggio con dati va consegnato uno per uno, invecchia, e quando una coda si riempie va perso: non può essere la fonte della verità. Un contatore costa un incremento, si emette anche sotto il lock di un buffer, e di una raffica fa un numero cresciuto.
5. **Il nucleo non ha thread né orologio.** Chi lo guida gli dà i turni, e l'ora gli arriva da un `Clock`: così si prova tutto senza thread, e in modo esatto. Scartato: un nucleo con il suo thread, che i test potrebbero solo aspettare.
6. **La regola delle classi si conta in passi.** Un passo normale ogni `high-steps` passi alti di fila, e non una quota di tempo: la regola è deterministica e si prova senza orologio.
7. **Il nucleo scandisce i suoi lavori invece di tenere un heap delle scadenze.** I lavori sono decine, non migliaia, e una scansione non può andare fuori passo con i lavori come un heap con voci superate. Un heap si aggiunge se le misure un giorno lo chiedono.
8. **Una chiamata non tocca il nucleo.** Il registro risponde subito, e il resto arriva all'esecutore come messaggio: così `submit` dall'interno di un passo non aspetta l'esecutore, che è il thread stesso che chiama. Comporta che registro e nucleo tengano due viste dei lavori, e che `jobs()` mostri le misure dell'ultima pubblicazione.
9. **Nomi unici, e il nuovo sostituisce il vecchio.** Rivalutare un modulo, o premere due volte lo stesso tasto, non duplica un lavoro. Le callback del lavoro sostituito si scartano con `is_latest`, che il registro cambia insieme al lavoro: due sottomissioni con lo stesso nome, o una sottomissione e uno stop, non possono lasciarlo a indicare il lavoro sbagliato.
10. **Un thread non si uccide: gli si chiede di fermarsi.** Rust non ha un modo sicuro di uccidere un thread, e uno ucciso mentre tiene il lock di un buffer bloccherebbe l'editor al primo accesso. Lo `Stop` e l'azione di sblocco coprono ogni caso tranne il thread che non controlla e non si sblocca, che viene segnalato e abbandonato all'uscita.
11. **`shutdown` è esplicito.** L'esecutore tiene una copia di `EditorState`, che contiene il WorkManager: è un ciclo di riferimenti, e `Drop` non lo rompe. Per questo il front-end chiama `shutdown` all'uscita, con una guardia che lo fa anche dopo un panic, e lo chiama anche ogni test che avvia dei thread.
12. **Il watchdog scrive il file di log.** Una scrittura che si blocca, per un disco pieno o una home di rete, ferma il watchdog e nessun altro, e una segnalazione arriva sul file anche quando ciò che segnala è bloccato. Chi scrive una riga nel log non scrive mai il file.
13. **Le callback dei lavori girano sul thread dei comandi.** Un lavoro non chiama il suo modo: lascia una nota, che il thread dei comandi legge dove leggerebbe un tasto. Se un lavoro chiamasse il modo, del Lisp scritto per il thread dei comandi girerebbe in un istante qualunque su un altro thread: a metà di un inserimento, dentro un hook che scorre una lista. Nessuna cura nella callback lo ripara, perché a essere sbagliata non è la callback.

12 Lo schermo

12.1 Finestre

Le finestre affiancate formano un albero (`LayoutNode`): una foglia è una `Window` (id, nome del buffer, scorrimento orizzontale e verticale, punto proprio quando non ha il focus, larghezza del margine), un nodo interno è una divisione orizzontale o verticale con una `Division` (`Ratio`, `FirstFixed`, `SecondFixed`). Sopra l'albero stanno le finestre *flottanti* (`FloatingWindow`: una finestra più un rettangolo, un bordo facoltativo, un titolo e la finestra a cui restituire il focus quando si chiude): il minibuffer, il `*Backtrace*`, i popup dei moduli.





Le foglie si dividono lo spazio; le flottanti sono disegnate sopra, con il loro rettangolo.

Il compartimento `Windows` tiene l'albero, le flottanti, l'id con il focus, il prossimo id, lo stato di un trascinamento del mouse e l'ultimo clic (per il doppio clic entro 400 ms). Le operazioni che tolgono una finestra rispondono con un `WindowRemoved` (`No`, `Yes`, `Refocus(id)`) che dice alla facciata se e verso quale finestra spostare il focus.

12.2 `snapshot`: dalla memoria alla fotografia

`snapshot(&env, larghezza, altezza)` costruisce il `FrameSnapshot`. Prende i lock in un ordine fisso e non fa I/O; il disegno avviene dopo, senza lock.

1. Abbassa il segnale del lavoro di sfondo (`BackgroundChange::taken`): ciò che i lavori hanno cambiato fin qui è in questa fotografia.
2. Legge dalle variabili Lisp e dai campi piccoli tutto ciò che serve *prima* di toccare finestre e buffer: `echo-message-timeout`, il tema, l'input in sospeso (`pending_input`), il messaggio della keymap transitoria, `mode-line-format`, `window-separator`, la configurazione del margine (`display-line-numbers`), se la colorazione è in corso.
3. Legge l'area messaggi, vuota se il messaggio è scaduto.
4. Con `windows` in scrittura (lo scorrimento può dover seguire il punto) e `buffers` in lettura, attraverso `windows_mut_with_buffers`, compone ogni foglia dell'albero nel rettangolo del frame meno l'ultima riga, raccogliendo i separatori; poi compone le flottanti.
5. Consuma il testo destinato agli appunti (`take_pending_clipboard`).

Per ogni foglia la composizione fa: scorrimento per mantenere visibile il punto; layout delle righe visibili, compreso il testo virtuale; margine dei numeri di riga (assoluti o relativi, larghezza minima `display-line-numbers-width`, 3 per default, massimo 20); evidenziazioni nell'ordine *sintassi* → *overlay* → *regione* → *testo virtuale*, perché le ultime vincono; posizione del cursore relativa alla finestra; riga di stato. Le finestre flottanti non hanno margine né riga di stato, e ricevono solo regione e testo virtuale (→ TUI-6).

Campo di `FrameSnapshot`	Contenuto
`views`	Un `RenderableWindowView` per finestra: rettangolo, nome del buffer, titolo, focus, cursore relativo, righe di testo, evidenziazioni (riga, colonne, faccia), riga di stato, bordo, margine e sua larghezza.
`separators`	Rettangoli e carattere delle linee tra finestre affiancate.
`echo_message`, `prompt`	Il messaggio e l'eventuale offerta della keymap transitoria, che lo sostituisce.
`pending_input`	`C-u 4 C-x-`: ciò che l'editor sta leggendo.
`theme`	Gli stili delle facce (`Arc<Theme>`).
`focused_window_id`, `width`, `height`	Come dice il nome.
`colouring_pending`	La colorazione non ha finito: ridisegnare tra 40 ms.
`clipboard`	Testo che il front-end deve mettere negli appunti di sistema, se presente.

La riga di stato segue `mode-line-format` (predefinito `" %* %b  %m %s  L%l C%c  %p "`): `%b` nome del buffer, `%f` file (o nome), `%m` modo, `%s` lo stato che il modo tiene nel buffer (`mode-line-state`, come il `NORMAL` di vim), `%l` riga (da 1), `%c` colonna (da 0), `%p` posizione (`All`, `Top`, `Bot` o percentuale), `%*` `**` se modificato e `--` altrimenti, `%%`.

12.3 Il renderer della TUI

`render_to` disegna una fotografia in quest'ordine: nasconde il cursore e pulisce lo schermo; disegna i separatori (prima delle finestre, perché una flottante deve poterli coprire); per ogni vista disegna bordo e titolo, ogni riga del rettangolo riempita fino alla larghezza (così una flottante è opaca), il margine cella per cella, le evidenziazioni sopra il testo, la riga di stato sotto il rettangolo; poi l'area messaggi nell'ultima riga, l'input in sospeso allineato a destra nella stessa riga, il cursore *per ultimo* e infine, se c'è, la sequenza OSC 52 per gli appunti.

I colori sono risolti dal front-end secondo la profondità del terminale: 24 bit con `COLORTERM` che contiene `truecolor` o `24bit`; il cubo a 256 colori (o la scala di grigi, se più vicina) con `TERM` che contiene `256color`; altrimenti la voce più vicina dei 16 colori della tavolozza, nell'ordine di `NAMED_COLORS`.

Gli appunti in uscita passano da OSC 52 perché l'editor non sa di avere un terminale e un programma esterno (`xclip`, `pbcopy`) non funzionerebbe via SSH. In entrata non c'è nulla da leggere: il terminale consegna gli appunti con l'incolla «bracketed», che diventa `insert-pasted-text`.

12.4 Il mouse

Con `mouse-mode` attivo la TUI chiede al terminale di riportare il mouse. `handle_mouse_event` traduce l'evento in un `Hit` (testo di una finestra, riga di stato, separatore, flottante, niente), e da lì in un comando: clic per spostare il punto e il focus, doppio clic, trascinamento per selezionare (con scorrimento automatico ogni 60 ms oltre il bordo), trascinamento di un separatore o di una riga di stato per ridimensionare, rotella per scorrere la finestra sotto il puntatore. Risponde `true` solo se ha cambiato qualcosa. Le domande sulle finestre — che cosa c'è sotto il puntatore, il trascinamento in corso, l'ultimo clic — le fa a `windows.rs`; il punto di un clic diventa una colonna del buffer in `buffer_column`, che compone la sola riga cliccata, perché il testo virtuale fa di una colonna sullo schermo un'altra colonna nel buffer.

12.5 Le decisioni

1. **Una fotografia sola, presa tutta insieme.** I lavori, i thread dei comandi di shell e quelli di `spawn` cambiano lo stato mentre il front-end disegna: un renderer che leggesse sei lock in sei istanti comporrebbe un frame mai esistito. `snapshot` legge tutto sotto i lock, senza I/O, e il disegno viene dopo, senza lock.

2. **`snapshot` prende le finestre in scrittura.** Comporre una finestra ne sposta lo scorrimento per tenere visibile il punto: disegnare cambia lo stato. Va bene finché disegna un thread solo; per comporre più frame in parallelo, quel raccordo dovrebbe diventare un passo dopo ogni comando.
3. **L'editor dice al front-end quando ridisegnare.** Quasi tutto cambia per un evento, ma un messaggio scade, la colorazione arriva, un lavoro riempie un buffer: il front-end chiede `next_redraw_in` e riceve una durata sola, e un secondo front-end ha lo stesso comportamento chiedendo la stessa cosa. `None` è la risposta comune, e un editor fermo non costa nulla.
4. **Le flottanti non hanno margine né riga di stato.** Una flottante è un prompt o una striscia di scelte, poche righe lette e chiuse: numerarle non dice nulla, e il bordo dice già che cosa sono.
5. **Il mouse calcola dove, un comando decide che cosa.** Dove cade un clic dipende dal layout e dallo scorrimento di ogni finestra, e lo calcola l'editor; che cosa significa passa dai comandi come un tasto: un passo di undo, un `post-command-hook`, un nome che `M-x` e il log conoscono, un legame che il Lisp può cambiare.
6. **Un trascinamento continua nella finestra in cui è cominciato.** Il seguito e la fine di un trascinamento si risolvono prima di chiedere che cosa c'è sotto il puntatore: dopo poche righe il puntatore è spesso su un'altra finestra o fuori dal frame, e una selezione che cambiasse buffer a metà non sarebbe una selezione.
7. **`mouse-mode` vale per ogni front-end.** `handle_mouse_event` lo controlla da sé, non solo dove il terminale accende la cattura: una finestra grafica, che riceve sempre il mouse, gli ubbidisce allo stesso modo.

13 Macro di tastiera

Una macro registra *tasti*, non comandi: `C-u 5`, un tasto letto da `read-key-sequence` e perfino un tasto non associato fanno parte di ciò che l'utente ha premuto. `handle_key_event` chiama `record_keys` nei punti in cui un tasto viene consumato. La riproduzione (`replay_keys`) rimanda i tasti a `handle_key_event`, con un `OneGroup` per ripetizione, e si ferma al primo comando fallito, confrontando `command_errors` prima e dopo ogni tasto. Una macro che richiama macro può annidarsi fino a 16 livelli (`MAX_DEPTH`).

13.1 Le decisioni

Tre decisioni sulla macro stanno con il suo compartimento (Sezione 4.8): una macro registra tasti, registrazione e riproduzione stanno insieme, le riproduzioni annidate hanno un tetto. Le altre sono della facciata.

1. **La riproduzione passa dalla porta dei tasti.** Ogni tasto registrato torna a `handle_key_event`, la stessa porta di un tasto del terminale: un secondo dispatcher sarebbe un secondo insieme di regole su argomenti prefisso, keymap transitorie, catture e prompt, tenuto in passo a mano. Il prezzo è il rientro, ed è già pagato: il carburante conta la profondità e solo lo scope più esterno ricarica, così i comandi di una riproduzione spendono il budget della riproduzione.
2. **Una riproduzione si ferma al primo comando fallito.** Ogni tasto dopo un errore sarebbe premuto in uno stato che la registrazione non ha mai visto. `run_command_form` non restituisce l'errore — un tasto non ha un chiamante a cui restituirlo —: `report_error` incrementa `command_errors`, che la riproduzione confronta prima e dopo ogni tasto. Un conteggio, che nessuno deve azzerare.
3. **Nessun tasto interrompe una riproduzione.** Il ciclo non legge tasti: è dentro un comando, e il thread che consegnerebbe `C-g` è quello che lo esegue. Una macro che richiama sé stessa la ferma il tetto di profondità, non il budget: ogni livello è una vera chiamata attraverso il gestore dei tasti, e lo stack finirebbe molto prima, interrompendo il processo. Per lo stesso motivo una riproduzione disegna un frame solo, quando il comando restituisce.
4. **Un gruppo di undo per ripetizione.** Ogni ripetizione ha il suo timbro e il suo `OneGroup`: venti ripetizioni sono venti passi, e una macro andata storta alla diciannovesima si riporta indietro fino lì.

14 Le primitive dell'editor

14.1 Il registro

Ogni modulo di `primitives/` ha una funzione `install(into: &Registry)` che registra le sue primitive con `into.function(nome, puntatore, DOC)` o, per i comandi, `into.command(nome, puntatore, specifiche, DOC)`. `install_primitives` chiama tutti i moduli. La macro `primitive!` genera la firma richiesta da `LispPrimitive`; `args.rs` contiene gli aiuti per leggere gli argomenti (nomi come simbolo o stringa, numeri, posizioni).

Ogni docstring comincia con la firma, nella forma `(nome ARG):` seguita da una frase: è ciò che mostrano i comandi di aiuto. `doc_faithfulness_tests` confronta ciò che l'aiuto racconta con l'editor reale; `primitive_shape_tests` verifica le forme condivise dalle primitive, come il controllo dell'arità.

Modulo	N.	Contenuto
`edits`	47	Inserire, cancellare, uccidere, incollare, undo, movimenti per carattere, parola, riga, paragrafo ed espressione.
`io`	31	File: aprire, salvare, rileggere, recuperare, rinominare; percorsi e directory; variabili d'ambiente; `quit` e `save-some-buffers`.
`buffers`	17	Creare, cambiare, chiudere buffer; contenuto; sola lettura; dati appesi (`buffer-put`, `buffer-get`).
`region`	17	Mark e regione, `keyboard-quit`, kill ring (`kill-region`, `yank`, `yank-pop`, `kill-new`).
`modes`	15	`make-mode`, `add-hook`, regole e regioni di sintassi, `add-auto-mode`, tabella sintattica, `syntax-ppss`.
`verbs`	14	Maiuscole, spazi, righe vuote, unione e duplicazione di righe, trasposizioni, `zap-to-char`.
`macros`	12	Registrazione e riproduzione delle macro, contatore.
`minibuffer`	12	`minibuffer-read`, conferma, annullamento, completamento, storia.
`replace`	12	Sostituzione e `query-replace`.
`windows`	12	Dividere, chiudere, scegliere, scorrere finestre; `display-buffer-at-bottom`.
`general`	11	`eval-file`, `define-key`, `define-repeat-key`, `repeat`, `log`, `backtrace`, `regexp-opt`, keymap transitorie.
`results`	11	Liste di posizioni (`occur`, compilazione) e `next-error`.
`commands`	10	`call-interactively`, `M-x`, `register-command` e il registro dei comandi.
`isearch`	10	Ricerca incrementale.
`rectangle`	10	Operazioni sui rettangoli.
`completion`	8	`completion-at-point`, le sue fonti, `fuzzy-filter`.
`match_data`	7	`string-match` e i dati della corrispondenza.
`comments`	6	Commentare e decommentare.
`help`	6	Ciò che serve all'aiuto: `key-binding`, `where-is`, `keymap-bindings`, `read-key-sequence`. I comandi `describe-*` sono in `help.lisp`.
`mouse`	6	Comandi del mouse.
`overlays`	5	Overlay.
`theme`	5	Facce e stili.
`ask`	4	Domande sì/no con keymap transitoria.
`shell`	4	Comandi di shell.

Modulo	N.	Contenuto
`virtual_text`	4	Testo virtuale.
`workers`	4	`define-worker`, `background-call`, `stop-worker`, `running-workers`.
`ui`	3	`recenter` e finestre flottanti.
`scan`	1	`scan-buffer`: tutte le corrispondenze di un modello in un buffer, come voci di una lista di risultati.

Il totale è 304, a cui si aggiungono le 86 primitive di base dell'interprete. L'elenco completo dei nomi è nell'appendice (Sezione 18.3).

15 I moduli Lisp

Quasi tutto il comportamento visibile è in `core/lisp/`: le associazioni dei tasti, i comandi di alto livello, i modi dei linguaggi, l'interfaccia dei completamenti. `build.rs` copia i file in `target/<profilo>/data/lisp` a ogni compilazione, da un *elenco esplicito*: un modulo nuovo va aggiunto lì, altrimenti non arriva accanto all'eseguibile e `eval-file` non lo trova.

Modulo	Che cosa aggiunge
`commands`	`defcommand`, `save-match-data` e altre macro di base.
`debug`	`message`, `report-error`, il buffer `*Backtrace*`, `debug-on-error`.
`common-keymaps`	Le associazioni di Emacs: movimento, kill, file, buffer, finestre, `C-g`.
`indent`	Tab, `indent-region`, `newline-and-indent`, `electric-indent-mode`.
`minibuffer`	Tasti e comportamenti del prompt.
`rust-mode`, `risp-mode`, `cc-mode`	Modi per Rust, per il Lisp dell'editor e per C/C++.
`dired`	Una directory in un buffer (`C-x d`).
`completion`	La striscia dei completamenti (`*completion-read-function*`).
`preview`	Anteprima di `string-rectangle`.
`clipboard`	Copia le uccisioni negli appunti di sistema.
`electric-pair`	Chiusura automatica di parentesi e virgolette.
`buffer-list`	`C-x b` e `C-x C-b`.
`shell`	`M-!`.
`manpage`	Pagine di manuale.
`help`	`C-h f`, `C-h k`, `C-h b`, `describe-prefix-keys`.
`occur`	`M-s o`, `M-s g`.
`compile`	`C-c c` e `M-g n`.
`theme`	`C-c t`: scelta del tema.
`completion-at-point`	Le cinque fonti di `C-M-i`.

Le variabili che l'editor legge da Lisp:

Variabile	Default	Effetto
`echo-message-timeout`	5	Secondi di permanenza di un messaggio; `nil` per sempre.
`minibuffer-width`, `minibuffer-height`	60, 3	Dimensioni del prompt.
`mode-line-format`	vedi sopra	Riga di stato.
`window-separator`	`"|"`	Carattere tra finestre; `nil` per uno spazio.
`display-line-numbers`	`nil`	`t` assoluti, `'relative` relativi.
`display-line-numbers-width`	3	Cifre minime del margine.
`mouse-mode`	`nil` (`t` in `init.lisp`)	Il mouse all'editor invece che al terminale.
`after-resize-hook`	`nil`	Funzioni chiamate con larghezza e altezza.

Variabile	Default	Effetto
`frame-width`, `frame-height`	—	Dimensioni correnti, scritte dal front-end.
`lisp-path`	`data/lisp`	Dove `eval-file` cerca i moduli.
`help-key`	`"C-h"`	Il tasto che, dopo un prefisso, chiede che cosa lo segue.
`worker-fuel`	200.000	Passi per turno di un worker.
`watch-files`, `watch-file-interval`	`t`, 3	Il watcher dei file.
`auto-save`, `auto-save-interval`, `auto-save-directory`	`t`, 30, —	L'autosalvataggio.
`history-length`	100	Voci per storia di prompt.
`*minibuffer-read-function*`	`default-minibuffer-prompt`	Chi apre i prompt.
`*key-capture-function*`	`nil`	Chi riceve la prossima sequenza invece di eseguirla.

16 I test

I test dell'editor stanno in `core/src/tests/`, uno o più file per funzionalità (circa ottanta), più `tests/perf/` per le misure e i test del front-end in `src/tests/`. Costruiscono un editor con `create_global_env`, caricano i moduli Lisp che servono con `include_str!` e valutano Lisp o simulano tasti con `handle_key_event`. `isolate_config_for_tests` punta `RSEDIT_CONFIG_DIR` a una directory temporanea del processo, una volta sola, così i test non scrivono nella configurazione di chi li esegue.

Alcuni file meritano di essere conosciuti prima di cambiare il codice che proteggono: `layering_tests` (i due confini), `deadlock_tests` (i lock attraverso Lisp), `doc_faithfulness_tests` e `primitive_shape_tests` (le docstring), `edit_protection_tests` (sola lettura), `undo_tests`, `frame_snapshot_tests`, `shipped_lisp_tests` (ogni modulo distribuito, caricato nell'ordine di `init.lisp`: nell'editor reale un errore di caricamento finisce solo nel log). `perf::suite` misura tempi reali con più thread e può fallire su macchine lente o condivise (→ DOC-4).

17 Ricette

17.1 Una primitiva o un comando in Rust

1. Scegli il modulo di `primitives/`; scrivi la docstring come `const`, con la firma nella prima riga.
2. Scrivi la funzione con `primitive!`. Leggi lo stato con le closure della facciata (`with_current_buffer_mut`, `windows`, …) e *non* chiamare Lisp con un lock preso: copia, rilascia, poi chiama.
3. Modifica il testo solo con `insert_text` e `delete_range`, e controlla il loro risultato.
4. Se scorri qualcosa di grande, addebita il carburante (`ctx.consume_fuel`).
5. Registra con `into.function` o, per un comando, con `into.command` e le specifiche degli argomenti.
6. Aggiungi i test in `core/src/tests/` e aggiorna la tabella di Sezione 14.

17.2 Un comando in Lisp

```
(defcommand count-words-region (start end) ("r")
  "Report how many words the region holds."
  (message "%d words" (length (split-string (buffer-substring start end)))))
(define-key nil "M-=" 'count-words-region)
```

`define-key` con `nil` lega nella keymap globale, con un simbolo di modo nella keymap di quel modo. Un comando legato senza argomenti passa da `call-interactively`, che gli chiede la regione.

17.3 Un modo maggiore

Il modello è `rust-mode.lisp`:

```
(make-mode 'toy-mode)
(add-syntax-region 'toy-mode "\"" "\"" 'string "\\\\.")
(add-syntax-rule 'toy-mode "#.*" 'comment)
(add-syntax-rule 'toy-mode "\\b(let|if|else)\\b" 'keyword)
(set-comment-syntax 'toy-mode '(("#")))
(put 'toy-mode 'tab-width 2)
(add-auto-mode "\\.toy$" 'toy-mode)
```

Le regioni prima delle regole; ciò che può attraversare una riga è una regione, il resto una regola. Ricorda che `make-mode` sostituisce un modo esistente.

17.4 Un lavoro di sfondo

In Lisp: `background-call` con una fiber che fa `(yield)` tra un pezzo e l'altro e una callback `ON-DONE` per il risultato, oppure `define-worker` con una fiber che aspetta il suo argomento, `(yield '(wait TOPIC))`. La pagina `background` ha gli esempi.

In Rust:

1. Scrivi un tipo che implementa `Job<EditorState<B>>`. Ogni `step` fa una quantità limitata di lavoro: copia sotto lock, lavora senza, ricontrolla la versione prima di salvare. Dichiara le unità con `cx.worked`, e risponde `Continue` se resta altro, `Wait` o `Sleep` se no. Il modello è `Highlighter` (Sezione 11.10).
2. Dagli una `Spec`: un nome unico, la classe (`High` solo se l'utente aspetta di vederne il risultato), l'unità, gli argomenti che lo svegliano, e `Start::After` se il primo passo non serve subito.
3. Sottomettilo con `work().submit(spec, Box::new(lavoro))` dove lo stato esiste: in `EditorState::new`, o in `create_global_env` se legge impostazioni del Lisp, che allora rilegge a ogni passo.

4. Ciò che può bloccarsi non è un lavoro: va su un thread dedicato, con `spawn_work`, o con `spawn_writer` se scrive in un buffer, e con `Stop::on_cancel` per l'azione che lo sblocca.
5. Aggiungi la riga alla tabella di Sezione 11.10, e i test in modalità manuale (Sezione 11.11).

17.5 Un modulo Lisp nuovo

Scrivi `core/lisp/nome.lisp`, aggiungilo all'elenco di `core/build.rs`, aggiungi `(eval-file "nome")` a `DEFAULT_INIT_LISP` in `editor/boot.rs` (e, a mano, al tuo `init.lisp`), e un test che lo carichi.

17.6 Un nuovo front-end

Un front-end deve: tradurre gli eventi nei tipi di `rsedit_core`; chiamare `handle_key_event`, `handle_paste`, `handle_mouse_event` e `resize`; comunicare `frame-width` e `frame-height`; in un ciclo, chiedere `snapshot` e disegnarlo senza tenere nulla dell'editor; aspettare al massimo `next_redraw_in` (e `drag_scroll_in`) e chiamare `tick` quando l'attesa scade; mettere `frame.clipboard` negli appunti. Tutto ciò che c'è da sapere su colori, bordi e cursore è nella fotografia; `src/tui.rs` è l'esempio completo.

18 Appendice

18.1 Glossario

Buffer testo con punto, mark, storia e modo; può visitare un file.
Punto la posizione del cursore in un buffer, in caratteri.
Mark, regione una seconda posizione; la regione è tra mark attivo e punto.
Comando una funzione registrata con le sue specifiche di argomenti.
Keymap transitoria una keymap consultata prima di tutte, finché non viene tolta.
Compartimento un gruppo di campi dello stato sotto un solo lock, in `managers/`.
Facciata `EditorState`, l'unico che tiene più lock di compartimento insieme.
Fotografia il `FrameSnapshot`, ciò che il front-end disegna.
Lavoro ciò che il WorkManager esegue a passi; implementa `Job`. Nel codice è un *job*; *work* è un lavoro o un thread dedicato, e *worker* resta solo nei nomi del Lisp (`define-worker`, `stop-worker`, `running-workers`, `worker-fuel`).
Passo una chiamata a `Job::step`, cioè una quantità limitata di lavoro.
Argomento un evento con un nome e senza dati, cioè un contatore che cresce a ogni emissione.
Esecutore il thread del WorkManager che esegue i passi, uno alla volta.
Thread dedicato un thread del WorkManager per un lavoro che può bloccarsi, fermato con il suo `Stop`.
Watchdog il thread del WorkManager che osserva il passo in corso e scrive il file di log.
Callback dovuta una chiamata chiesta da un lavoro di sfondo ed eseguita dal thread dei comandi.
Epoca il timbro del comando corrente, con cui l'undo raggruppa le modifiche.

18.2 Mantenere aggiornato questo manuale

- Nuovo campo di `EditorState` o nuovo compartimento → Sezione 4 (tabelle e ordine dei lock).
- Cambiamento del percorso di un tasto → Sezione 5; di `call-interactively` → Sezione 6.
- Nuova primitiva o nuovo modulo di `primitives/` → Sezione 14 e l'elenco in appendice (lo script che lo genera cerca `into.function(` e `into.command(`).
- Nuovo lavoro dell'editor → la tabella di Sezione 11.10; nuova opzione o nuova segnalazione → Sezione 11.9.
- Nuovo campo della fotografia o cambiamento del renderer → Sezione 12.
- Nuovo modulo Lisp o nuova variabile letta da Rust → Sezione 15.
- Difetto corretto → aggiorna la descrizione, togli il rimando al rapporto e la scheda da `rapporto-problemi.typ`.

Gli estratti di codice vanno riletti quando cambia la funzione da cui vengono. I diagrammi usano gli aiuti di `style.typ` descritti nel manuale dell'interprete. Dopo ogni modifica conviene compilare e guardare le pagine.

18.3 Elenco delle primitive dell'editor

Generato da `into.function(` e `into.command(` nei file di `primitives/`. Un asterisco indica un comando, raggiungibile con `M-x`.

Modulo	Primitive
`ask` (4)	`y-or-n`, `yes-or-no`, `ask--answer`, `ask--typed`
`buffers` (17)	`buffer-put`, `buffer-get`, `buffer-substring`, `switch-to-buffer`, `current-buffer`, `buffer-create`, `close-buffer`, `kill-buffer`*, `kill-buffer-without-saving`*, `buffer-string`, `clear-buffer`, `with-current-buffer`, `set-buffer-read-only`, `buffer-read-only-p`, `buffer-modified-p`, `buffer-file-name`, `major-mode`

Modulo	Primitive
`commands` (10)	`register-command`, `commandp`, `command-args`, `all-commands`, `execute-extended-command`, `command-completions`, `command-execute-prompt`*, `all-buffer-names`, `call-interactively`, `pending-command`
`comments` (6)	`comment-dwim`*, `comment-line`*, `comment-indent`*, `comment-region`*, `uncomment-region`*, `comment-syntax`
`completion` (8)	`completion-at-point`*, `fuzzy-filter`, `completion-at-point-choose`, `add-completion-function`, `set-completion-functions`, `completion-functions`, `buffer-words`, `bounds-of-thing-at-point`
`edits` (47)	`delete-region`, `self-insert`, `insert`, `insert-pasted-text`*, `insert-newline`*, `delete-backward-char`*, `backward-char`*, `forward-char`*, `previous-line`*, `delete-char`*, `kill-line`*, `kill-whole-line`*, `kill-word`*, `backward-kill-word`*, `kill-paragraph`*, `backward-kill-paragraph`*, `undo`*, `redo`*, `undo-boundary`*, `set-undo-limit`*, `beginning-of-line`*, `end-of-line`*, `forward-word`*, `backward-word`*, `forward-sexp`*, `backward-sexp`*, `kill-sexp`*, `backward-kill-sexp`*, `up-list`*, `backward-up-list`*, `down-list`*, `current-column`, `current-indentation`, `previous-indentation`, `indent-line-to`, `forward-paragraph`*, `backward-paragraph`*, `beginning-of-buffer`*, `end-of-buffer`*, `goto-line`*, `next-line`*, `point`, `point-min`, `point-max`, `goto-char`, `line-number-at-point`, `current-line`
`general` (11)	`eval-file`, `define-key`, `define-repeat-key`, `repeat`*, `log`, `regexp-opt`, `all-logs`, `backtrace`, `set-echo-message`, `set-transient-keymap`, `clear-transient-keymap`
`help` (6)	`key-binding`, `keys-with-prefix`, `keymap-bindings`, `where-is`, `command-specs`, `read-key-sequence`
`io` (31)	`quit`*, `quit-without-saving`*, `save-some-buffers`*, `save-some-buffers--answer`, `quit--confirm`, `find-file`*, `data-directory`, `getenv`, `setenv`, `path-separator`, `list-dir`, `directory-files-recursive`, `read-file-to-string`, `match-list`, `expand-file-name`, `file-name-as-directory`, `directory-file-name`, `file-name-directory`, `file-name-nondirectory`, `save-buffer`*, `save-buffer--overwrite`, `recover-file`*, `recover-file--adopt`, `revert-buffer`*, `revert-buffer--reread`, `write-file`*, `file-exists-p`, `file-directory-p`, `directory-entry-count`, `delete-file`, `rename-file`
`isearch` (10)	`isearch-forward`*, `isearch-backward`*, `isearch-forward-regexp`*, `isearch-backward-regexp`*, `isearch-update`, `isearch-exit`, `isearch-abort`, `isearch-repeat-forward`, `isearch-repeat-backward`, `isearch-active-p`
`macros` (12)	`kmacro-start-macro`*, `kmacro-end-macro`*, `kmacro-cancel-macro`*, `kmacro-call-macro`*, `kmacro-end-and-call-macro`*, `kmacro-insert-counter`*, `kmacro-set-counter`*, `kmacro-counter`, `kmacro-recording-p`, `kbd-macro-keys`, `define-kbd-macro`, `kmacro-call-keys`
`match_data` (7)	`string-match`, `string-match-p`, `match-beginning`, `match-end`, `match-string`, `match-data`, `set-match-data`

Modulo	Primitive
`minibuffer` (12)	`minibuffer-cleanup`, `minibuffer-confirm`, `minibuffer-cancel`, `minibuffer-complete`, `minibuffer-choose-completion`, `default-minibuffer-prompt`, `minibuffer-read`, `completion-invalidate`, `minibuffer-history`, `clear-minibuffer-history`, `history-previous`*, `history-next`*
`modes` (15)	`make-mode`, `add-hook`, `add-syntax-rule`, `add-syntax-region`, `add-auto-mode`, `syntax-ppss`, `balance-point`, `bounds-of-enclosing-list`, `set-syntax-pairs`, `set-string-syntax`, `set-char-quote`, `set-syntax-entry`, `set-comment-syntax`, `syntax-class`, `matching-delimiter`
`mouse` (6)	`mouse-set-point`*, `mouse-start-selection`*, `mouse-drag-to`*, `mouse-resize`*, `mouse-mode-toggle`*, `mouse-scroll`*
`overlays` (5)	`make-overlay`, `delete-overlay`, `remove-overlays`, `overlays-at`, `overlay-face`
`rectangle` (10)	`rectangle-mark-mode`*, `kill-rectangle`*, `delete-rectangle`*, `copy-rectangle-as-kill`*, `yank-rectangle`*, `open-rectangle`*, `clear-rectangle`*, `string-rectangle`*, `rectangle-bounds`, `killed-rectangle`
`region` (17)	`set-mark`*, `keyboard-quit`*, `deactivate-mark`*, `exchange-point-and-mark`*, `mark-whole-buffer`*, `kill-region`*, `kill-ring-save`*, `yank`*, `yank-pop`*, `set-kill-ring-max`*, `mark`, `use-region-p`, `region-beginning`, `region-end`, `kill-new`, `current-kill`, `kill-ring-length`
`replace` (12)	`query-replace`*, `query-replace-regexp`*, `replace-string`*, `replace-regexp`*, `replace-this`, `replace-skip`, `replace-rest`, `replace-back`, `replace-done`, `replace-abandon`, `replace-match`, `replace-count`
`results` (11)	`occur--scan`, `grep--scan`, `next-error`*, `previous-error`*, `results-count`, `results-entry`, `results-state`, `results-visit`, `results-buffer`, `results-select`*, `results-put`
`scan` (1)	`scan-buffer`
`shell` (4)	`shell-command-start`*, `shell-command-to-string`, `parse-overstrike`, `shell-command-running-p`
`theme` (5)	`set-face`*, `face-style`, `reset-faces`, `list-faces`, `list-colors`
`ui` (3)	`recenter`*, `make-floating-window`, `close-floating-window`
`verbs` (14)	`upcase-word`*, `downcase-word`*, `capitalize-word`*, `delete-horizontal-space`*, `just-one-space`*, `delete-blank-lines`*, `back-to-indentation`*, `join-line`*, `duplicate-line`*, `transpose-lines`*, `transpose-chars`*, `transpose-words`*, `zap-to-char`*, `zap-to-char--do`
`virtual_text` (4)	`make-virtual-text`, `clear-virtual-text`, `delete-virtual-text`, `virtual-text-at`
`windows` (12)	`split-window-below`*, `split-window-right`*, `delete-window`*, `delete-other-windows`*, `other-window`*, `selected-window`, `select-window`, `count-windows`, `scroll-up-command`*, `scroll-down-command`*, `display-buffer-at-bottom`, `window-buffer`

Modulo	Primitive
`workers` (4)	`define-worker`, `stop-worker`, `running-workers`, `background-call`